



POLYTECHNIC UNIVERSITY OF BARI

DEPARTMENT OF ELECTRICAL AND INFORMATION ENGINEERING

MASTER'S DEGREE IN COMPUTER SCIENCE ENGINEERING
CURRICULUM IN ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

FINAL DISSERTATION IN
FORMAL LANGUAGES AND COMPILERS

NEURO-SYMBOLIC SEMANTIC SEGMENTATION FRAMEWORK FOR AUTONOMOUS SPACE EXPLORATION

SUPERVISOR:

PROF. FLORIANO SCIOSCIA

CO-SUPERVISORS:

PROF. MICHELE RUTA

PROF. SAVERIO IEVA

ENG. DAVIDE LOCONTE

CANDIDATE:

GERMANO PANSINI

ACADEMIC YEAR 2024–2025



LIBERATORIA ALLA CONSULTAZIONE DELLA TESI DI LAUREA DI CUI ALL'ART.4 DEL REGOLAMENTO DI ATENEO PER LA CONSULTAZIONE DELLE TESI DI LAUREA (D.R. n. 479 del 14/11/2016).

Il sottoscritto PANSINI GERMANO matricola 589281
Corso _____ di _____
INGEGNERIA INFORMATICA MAGISTRALE (AI & DATA SCIENCE)
autore della presente tesi di Laurea dal titolo
Neuro Symbolic Semantic Segmentation Framework for Autonomous Space Exploration

Parola chiave Neuro-symbolic AI; Lunar robotics; Semantic segmentation; Description Logics; Ontologies; Tiny-ME reasoner; USD digital twin; Autonomous navigation.

Abstract This thesis presents a neuro-symbolic navigation pipeline for autonomous robotic exploration of lunar environments. A SegFormer-based semantic segmentation model trained on the LuSNAR dataset is combined with the SpEx ontology and the Tiny-ME embedded Description Logic reasoner to transform multi-sensor observations into symbolic scene descriptions and task-oriented explanations. The resulting ABoxes are embedded and hash-validated inside semantic USD digital twins, enabling high-fidelity 3D inspection and simulation. The experimental evaluation analyses segmentation accuracy, reasoning capabilities, and runtime overhead to assess the feasibility of the proposed approach on resource-constrained robotic platforms.

☒ Autorizza

☐ Non autorizza

la consultazione della presente tesi, fatto divieto a chiunque di riprodurre in tutto o in parte quanto in essa contenuto.

Bari, 03/12/2025

Firma _____

La/il sottoscritta/o,

Cognome: PANSINI

Nome: GERMANO

Matricola: 589281

Data Seduta Laurea: 17 / 12 / 2025.

dichiara

che la tesi cartacea consegnata in data odierna è conforme a quella caricata su esse3, quindi, approvata

dal Relatore Prof. SCIOSCIA FLORIANO

(in stampatello)

Bari, 03 / 12 / 2025

Firma

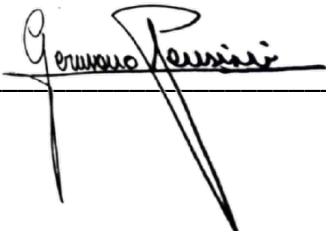


TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES	1
-----------------	---

ACRONYMS AND ABBREVIATIONS	3
----------------------------	---

<i>Introduction and Thesis Objectives</i>	8
---	---

1 Theoretical Foundations and State of the Art in Neuro-Symbolic AI	12
1.1 Symbolic and Sub-Symbolic Artificial Intelligence	12
1.1.1 From Logic-Based Knowledge to Learned Representations	13
1.1.2 Hybrid Neuro-Symbolic Paradigms and Design Axes	14
1.2 Knowledge Representation Systems, OWL and Lightweight Encodings	15
1.2.1 TBox and ABox for KRSs	15
1.2.2 OWL, Profiles and Binary Encodings	16
1.3 Description Logics and the Attributive Language with unqualified Number restrictions ($\mathcal{ALN}$) Fragment	16
1.3.1 Constructors and Expressivity	16
1.3.2 Roles, Datatypes and Numeric Constraints	17
1.3.3 Normal Forms and Structural Reasoning	17
1.4 Reasoning Services in Description Logics	18
1.4.1 Standard Services	18
1.4.2 Non-Standard Services: Abduction, Contraction and Re- lated Operators	19
1.5 Tiny-ME for Embedded Semantic Reasoning	20
1.5.1 Architecture, C API and Internal Representation	20
1.5.2 Complexity Profile and Resource Usage	21
1.5.3 Integration with Cowl and Streamed ABox Updates	22
1.6 Ontologies for Autonomous Robotics	22
1.6.1 IEEE 1872–2015 and CORA	22
1.6.2 Reference Frames, Units and Spatial Relations	23
1.6.3 Guidelines for Attributive Language with unqualified Num- ber restrictions and concrete Domains ($\mathcal{ALN}(\mathcal{D})$)-Compliant TBox Design	23

1.7	State of the Art in Neuro-Symbolic Perception and Navigation . . .	24
1.7.1	Ontology-Guided Segmentation and Multimodal Fusion . . .	24
1.7.2	Geospatial Knowledge Graphs and Neuro-Symbolic Methods	25
1.7.3	Terrain Semantic Segmentation for Planetary Rover Vision .	25
1.8	Critical Synthesis and Research Gaps	26
2	Datasets and Simulation for Planetary Exploration	28
2.1	Requirements for Semantic Datasets in Neuro-Symbolic Navigation	29
2.1.1	Semantic Coverage, Class Imbalance, and Environmental Variability	29
2.1.2	Symbolic Compatibility and ABox/TBox Mappability	30
2.1.3	Simulation and USD/KG Integration	31
2.2	Comparative Analysis and Cataloging Methodology	32
2.2.1	Dataset Template and Qualitative Indicators	32
2.2.2	Omniverse Simulability and Integration Effort	32
2.2.3	Suitability for Symbolic Reasoning and KB Construction . .	33
2.3	Analyzed Datasets (DS01–DS10)	34
2.3.1	DS01 – SSCBench	34
2.3.2	DS02 – AI4Mars	35
2.3.3	DS03 – S ⁵ Mars	35
2.3.4	DS04 – HM3D	35
2.3.5	DS05 – TartanAir	36
2.3.6	DS06 – RELIS-3D	36
2.3.7	DS07 – M2DGR	36
2.3.8	DS08 – Replica	36
2.3.9	DS09 – HSSD-200	37
2.3.10	DS10 – LuSNAR (Overview)	37
2.3.11	Cross-Dataset Summary	38
2.4	Selection of the Reference Benchmark	38
2.4.1	Comparative Evaluation of Martian, Terrestrial, and Lunar Datasets	38
2.4.2	Rationale for Selecting LuSNAR	39
2.5	Detailed Characterisation of the LuSNAR Dataset	40
2.5.1	Dataset Structure and Multi-Sensor Organisation	41
2.5.2	Semantic Taxonomy and Label Encodings	42
2.5.3	Neuro-Symbolic Metadata Derived from LuSNAR	42
2.6	OmniLRS and NVIDIA Isaac Sim as Simulation Environments . . .	43
2.6.1	USD-based Lunar Digital Twin and ROS 2 Integration . . .	43
2.7	Summary and Role in the Overall Architecture	44
3	System Architecture and Design	45
3.1	Design Objectives and Constraints	45
3.2	High-Level System Overview	46
3.3	Perception Subsystem	48
3.3.1	LuSNAR streams	48

3.3.2	SegFormer semantic segmentation	48
3.3.3	3D reconstruction and semantic projection	48
3.4	Knowledge and Reasoning Subsystem	49
3.4.1	Design of the TBox	49
3.4.2	Rover profiles and goal requests	51
3.4.3	Dynamic ABox construction	52
3.4.4	Tiny-ME and the <code>libusdsem</code> library	52
3.5	End-to-End Data and Control Flow	53
3.5.1	Offline training	53
3.5.2	Neuro-symbolic processing and USD export	53
3.5.3	Decision-support interface	54
3.6	ROS 2 and USD/Isaac Sim Integration	54
3.6.1	Lightweight ROS 2 graph	54
3.6.2	USD stages and Isaac Sim	56
3.7	Summary	56
4	Implementation of the Neuro-Symbolic Pipeline	57
4.1	Development Environment and Core Dependencies	57
4.1.1	Hardware Platform	57
4.1.2	System Environment and Reproducibility	58
4.1.3	USD, Isaac Sim, and OmniLRS Environment	58
4.1.4	Build System and Project Layout	59
4.2	LuSNAR Dataset Preparation	60
4.2.1	Scene Selection and Frame Indexing	60
4.2.2	Semantic Mask Conversion and Normalization	60
4.2.3	Data Augmentation	60
4.3	SegFormer-Based Semantic Segmentation	61
4.3.1	Architecture and Configuration	61
4.3.2	Loss Function and Optimisation	61
4.3.3	Inference and Output Products	62
4.4	The <code>libusdsem</code> Library: Bridging USD, SpEx, and Tiny-ME	62
4.4.1	Design Goals and Interface	62
4.4.2	Logical Language Validation: <code>check_aln()</code>	63
4.4.3	Mapping Between USD Prims and SPEX Individuals	63
4.4.4	Integration with Tiny-ME	64
4.5	ROS 2 Architecture and Node-Level Design	64
4.5.1	Overall ROS 2 Graph	64
4.5.2	Perception Nodes	65
4.5.3	ABox Builder Node	65
4.5.4	Matchmaking and Reasoning Node	66
4.5.5	USD Bridge and visualization Node	66
4.5.6	Navigation Interface Node	67
4.5.7	Node Life-Cycle and Synchronization	67
4.6	Summary	68

5	Experimental Results	69
5.1	Dataset and Preprocessing	69
5.1.1	Lunar Segmentation, Navigation and Reconstruction (LuS-NAR) Dataset and Camera Choice	69
5.1.2	Label Processing and Image Resizing	70
5.1.3	Final Dataset Structure	70
5.2	Training Setup	71
5.2.1	Network Architecture and Initialization	71
5.2.2	Loss Functions and Class Balancing	71
5.2.3	Data Augmentation	71
5.2.4	Optimization and Early Stopping	72
5.3	Evaluation Protocol and Metrics	72
5.3.1	Scene-Wise Segmentation Testing and Output Artifacts . . .	72
5.3.2	Segmentation Metrics	73
5.3.3	Segmentation Latency and Overhead Decomposition	73
5.3.4	End-to-End Pipeline Benchmarking	74
5.4	Segmentation Results	75
5.4.1	Global Segmentation Performance	75
5.4.2	Per-Class Analysis	76
5.4.3	Computational Efficiency of SegFormer	78
5.4.4	Qualitative Analysis and Limitations on LuSNAR Triptychs	80
5.4.5	Model Selection for Neuro-Symbolic Integration	83
5.5	End-to-End Pipeline Timings and 3D Semantics	84
5.5.1	Per-Stage Runtime Decomposition	84
5.5.2	Implications for Robot Operating System 2 (ROS 2) Deployment	85
5.5.3	Reasoning Output Structure and Matchmaking Scores . . .	86
5.5.4	Hash-Based Validation of Embedded ABoxes	89
5.5.5	Qualitative goal-resource matchmaking examples	90
5.6	Summary of Experimental Evidence	93
6	CONCLUSIONS AND FUTURE WORK	94
6.1	Summary of Contributions	94
6.2	Limitations	95
6.3	Future Directions	96
	BIBLIOGRAPHY	100
A	Dataset Catalog for Planetary and Robotic Benchmarks	104
A.1	Dataset Cataloging Template	104
A.2	Catalog Entries DS01–DS10	105

LIST OF TABLES

1.1	Standard Description Logic (DL) reasoning services relevant for Tiny Matchmaking Engine (Tiny-ME).	18
1.2	Non-standard DL reasoning services provided by Tiny-ME.	19
2.1	Summary of the ten catalogd datasets (see Appendix A for the full catalog).	38
5.1	Global performance on the LuSNAR test set (Moon_3, Moon_5, Moon_7).	75
5.2	Per-class IoU and accuracy on the LuSNAR test set.	76
5.3	Computational efficiency at 1024×1024 resolution.	78
5.4	Per-stage timings and memory usage for the neuro-symbolic pipeline on LuSNAR Moon_3 (SegFormer-B0, SpEx mode, 10 frames). . . .	84
A.1	Compact dataset cataloging template used throughout Appendix A.2.	104
A.2	Catalog entry for DS01 – Semantic Scene Completion Benchmark for Autonomous Driving (SSCBench).	105
A.3	Catalog entry for DS02 – A Dataset for Terrain-Aware Autonomous Driving on Mars (AI4Mars).	105
A.4	Catalog entry for DS03 – Semi-Supervised Semantic Segmentation Mars Dataset (S5Mars).	106
A.5	Catalog entry for DS04 – Habitat-Matterport 3D Dataset (HM3D).	107
A.6	Catalog entry for DS05 – AirSim Simulation Dataset for Visual SLAM (TartanAir).	107
A.7	Catalog entry for DS06 – RELLIS-3D Off-Road Robotics Dataset (RELLIS-3D).	108
A.8	Catalog entry for DS07 – Multi-sensor and Multi-scenario SLAM Dataset for Ground Robots (M2DGR).	108
A.9	Catalog entry for DS08 – Replica Photorealistic Indoor Dataset (Replica).	109
A.10	Catalog entry for DS09 – Habitat Synthetic Scenes Dataset 200 (HSSD-200).	109
A.11	Catalog entry for DS10 – LuSNAR.	110

LIST OF FIGURES

3.1	Block-level architecture of the neuro-symbolic pipeline.	47
3.2	Core structure of the SpEx knowledge base.	49
3.3	Specialization of the core hierarchy.	50
3.4	Logical structure of the <code>:BetaPickupGoalAndCapabilityPattern</code> class.	51
3.5	ROS 2 integration graph. Nodes wrap existing components: Seg- Former inference, ABox builder, Tiny-ME reasoner, mesh recon- struction, USD export and validation.	55
5.1	Global metrics comparison between SegFormer-B0 and SegFormer- B1 (mIoU, mAcc, overall accuracy).	76
5.2	Per-class metrics for Rocks: IoU and accuracy for SegFormer-B0 and SegFormer-B1.	77
5.3	Per-class metrics for Mountains: IoU and accuracy for SegFormer- B0 and SegFormer-B1.	77
5.4	Per-class metrics for Impact craters: IoU and accuracy for SegFormer- B0 and SegFormer-B1.	78
5.5	Forward-pass latency (ms) at 1024×1024 for SegFormer-B0 and SegFormer-B1.	79
5.6	End-to-end latency with I/O (ms) at 1024×1024 for SegFormer-B0 and SegFormer-B1.	80
5.7	Triptych 1693290477807724288 (RGB GT SegFormer-B0), Moon_3 test frame.	81
5.8	Triptych 1693290361863365888 (RGB GT SegFormer-B0), Moon_3 test frame.	81
5.9	Triptych 1693290392722639872 (RGB GT SegFormer-B0), Moon_3 test frame.	81
5.10	Triptych 1693290369163568640 (RGB GT SegFormer-B0), Moon_3 test frame.	82

ACRONYMS AND ABBREVIATIONS

$\mathcal{ALN}(\mathcal{D})$ Attributive Language with unqualified Number restrictions and concrete Domains , 8–10, 13, 17, 21–23, 25, 26, 29, 46, 49

$\mathcal{ALN}$ Attributive Language with unqualified Number restrictions , 12, 14–17, 20, 30–34, 40, 43, 44, 62, 63, 66, 68, 83, 86, 94, 106

ABox Assertional Box 9–11, 15, 16, 19, 21, 22, 24, 28, 30–32, 40–48, 51–54, 56, 59, 62–69, 74, 75, 80, 82, 83, 85, 86, 89, 90, 93–96, 98, 105–107, 109

AdamW Adaptive Moment Estimation with Decoupled Weight Decay 62, 72

ADE20K ADE20K Scene Understanding Dataset 61

AI Artificial Intelligence 8, 11–13, 25, 35, 37, 107, 109

AI4Mars A Dataset for Terrain-Aware Autonomous Driving on Mars 1, 29–31, 33, 35, 38, 40, 105

AirSim AirSim 32, 36, 37, 39, 107, 110

API Application Program Interface 15, 20, 62, 64

BPA Ball-Pivoting Algorithm 86

CA Concept Abduction 9, 10, 19–21, 27, 34, 43, 63, 66, 87, 88, 93, 94

CB Concept Bonus 19–21, 27

CC Concept Contraction 9, 10, 19–21, 27, 34, 43, 63, 66, 87, 88, 93, 94

CCov Concept Covering 19–21, 27, 34, 63

CD Concept Difference 19–21, 27

CNF Conjunctive Normal Form 17, 18

CORA Core Ontology for Robotics and Automation 22–24, 49

CPU Central Processing Unit 21, 57, 58, 73, 75, 84, 85, 95, 98

CS Concept Satisfiability 19, 87

CSV Comma-Separated Values 60, 72, 107

DEM Digital Elevation Model 41, 106

DL Description Logic 1, 8–11, 13–19, 23–26, 29, 30, 93, 95

DOI Digital Object Identifier 104

EL OWL 2 Expressive and Lightweight 16

glTF Graphics Language Transmission Format 31–33, 35–37, 39, 107, 109, 110

GPS Global Positioning System 32, 36, 104, 105, 108

GPU Graphics Processing Unit 21, 57, 58, 61, 65, 71, 73, 79, 98

GT Ground Truth 107

HDF5 Hierarchical Data Format 5 62

HDR High Dynamic Range 109

HM3D Habitat-Matterport 3D Dataset 1, 31, 33, 35, 37, 38, 107

HSSD-200 Habitat Synthetic Scenes Dataset 200 1, 31, 33, 37, 38, 109

HSV Hue Saturation Brightness 71

Hz Hertz 86

IMU Inertial Measurement Unit 9, 10, 31, 32, 36, 37, 39, 41, 46, 48, 50, 52–54, 60, 65, 69, 70, 74, 82, 86, 94, 104–110

IoT Internet of Things 19, 27

IoU Intersection over Union 73, 75–77, 80, 82, 83

IR Infrared 108

IROS IEEE/RSJ International Conference on Intelligent Robots and Systems 105

IsaacSim NVIDIA Isaac Sim 8, 10, 28, 31–33, 39, 43, 44

JPEG Joint Photographic Experts Group 108

JSON JavaScript Object Notation 72, 86, 88, 89, 93, 94, 106, 107, 109

KB Knowledge Base 104–110

KG Knowledge Graph 9–11, 13–16, 18, 19, 21, 22, 25, 26, 28, 32–38, 40, 47, 57, 59, 63, 64, 96–98

KITTI-360 KITTI-360 Autonomous Driving Dataset 105

KRL Knowledge Representation Language 13

KRS Knowledge Representation System 13, 15

LiDAR Light Detection and Ranging 9, 10, 32, 36, 37, 39–41, 43, 44, 46, 48, 65, 69, 70, 104, 105, 107, 108, 110

lr Learning Rate 62

LuSNAR Lunar Segmentation, Navigation and Reconstruction , 1, 8–11, 28–33, 36–50, 53, 54, 56, 57, 59–62, 65, 68–72, 74, 75, 82, 84, 86, 88, 89, 93–95, 97, 110

M2DGR Multi-sensor and Multi-scenario SLAM Dataset for Ground Robots 1, 30, 36, 38, 108

mAcc Mean Accuracy 40, 73, 75, 76

Mastcam Mast Camera 105, 106

Matplotlib Matlab + Plotting Library 58

MER Mars Exploration Rover 105

mIoU Mean Intersection over Union 40, 72, 73, 75, 76, 83, 93, 94

MLP Multi-Layer Perceptron 61

MSL Mars Science Laboratory 105, 106

NAVCAM Navigation Camera 105

NumPy Numerical Python 58

nuScenes nuScenes Autonomous Driving Dataset 105

NVIDIA NVIDIA Corporation 21, 57, 58

Odometry Odometry 31, 104–109

OmniLRS Omniverse Lunar Robotics Simulator 28, 31, 32, 39, 43, 44, 59, 68

Omniverse NVIDIA Omniverse Platform 8, 10, 28, 29, 32, 33, 37, 43, 44, 104–107, 109, 110

OpenCV Open Source Computer Vision Library 58

OWA Open World Assumption 20

OWL Web Ontology Language 9–12, 14, 16, 21–26, 31, 52, 53, 62, 83, 95, 96, 98

OWL Functional OWL 2 Functional-Style Syntax 16, 66

OWLLink OWL API Communication Protocol 16

Pandas Panel Data and Python Data Analysis 58

PDS Planetary Data System 106

PFM Portable Float Map 110

PNG Portable Network Graphics 60, 106–110

ProtocOWL ProtocOWL 10, 16, 22, 45, 46, 48, 52–54, 56, 62, 64, 66, 74, 84, 85, 89, 94, 95

PTex Ptex Texture Mapping Format 109

QL OWL 2 Query Language 16

QoS Quality of Service 65

RAM Random Access Memory 21, 57, 84, 85

RCC8 Region Connection Calculus 8 23

RDF Resource Description Framework 16

RELLIS-3D RELLIS-3D Off-Road Robotics Dataset 1, 29–31, 33, 36, 38, 108

Replica Replica Photorealistic Indoor Dataset 1, 31, 33, 36–38, 109

RGB Red Green Blue 15, 32, 35–37, 39, 41, 44, 46, 48, 54, 60, 65, 69, 70, 72, 74, 104–110

RL OWL 2 Rule Language 16

ROS 2 Robot Operating System 2 , 10, 11, 32, 33, 43, 45, 46, 48, 52–54, 56–59, 62, 64, 65, 67–69, 85, 86, 90, 93, 96, 98

RTK Real-Time Kinematic 108

RTX NVIDIA Ray Tracing GPU Architecture 57

S5Mars Semi-Supervised Semantic Segmentation Mars Dataset 1, 25, 29–31, 33, 35, 38, 40, 106

SciPy Scientific Python 58

SegFormer SegFormer 9, 11, 44, 46, 48, 53, 54, 56–62, 65, 68, 69, 71, 72, 74, 75, 78, 80, 82–86, 94, 95, 97–99

SHA-256 Secure Hash Algorithm 256-bit 46, 48, 53

SHOIN(D) SHOIN with Datatypes 17

SLAM Simultaneous Localisation and Mapping 29, 36, 37, 39–41, 65, 67, 107–110

SPEx Symbolic Planetary Exploration 8–11, 29, 39, 42–44, 46–48, 51, 57, 59, 62–64, 66–68, 74, 84, 86, 94, 96, 97, 99, 110

SROIQ(D) SROIQ with Datatypes 16, 17

SSC Semantic Scene Completion 34, 38, 105

SSCBench Semantic Scene Completion Benchmark for Autonomous Driving 1, 31, 34, 38, 105

SWoE Semantic Web of Everything 16, 20, 22

TartanAir AirSim Simulation Dataset for Visual SLAM 1, 30, 31, 33, 36, 38, 39, 107

TBox Terminological Box 9, 10, 12, 15–22, 24, 28, 30–32, 40–42, 44, 46–53, 56, 62–64, 66, 67, 74, 83, 86, 88, 94, 96, 97, 99, 105–107, 109

Tiny-ME Tiny Matchmaking Engine 1, 8–13, 15–25, 27, 29, 30, 33, 34, 40, 41, 43, 44, 46, 47, 49, 52–54, 56–59, 62–64, 66–69, 74, 83, 85–88, 90, 91, 93–98

TXT Plain Text File Format 110

UE4 Unreal Engine 4 32, 33, 36, 37, 39, 107, 110

UGV Unmanned Ground Vehicle 108

URL Uniform Resource Locator 104

USD Universal Scene Description 8, 10, 11, 28, 29, 31–41, 43–50, 52–54, 56–59, 62–69, 74, 75, 85, 86, 89, 90, 93–99, 104–110

XML Extensible Markup Language 16

Zarr Zarr Array Storage Format 62

Introduction and Thesis Objectives

Autonomous robotic exploration of planetary surfaces imposes strict requirements on perception, decision-making, and resource management. Future lunar missions are expected to rely increasingly on robotic systems operating for extended periods with limited supervision, under uncertain sensing conditions and tight energy and compute budgets. In this setting, purely sub-symbolic approaches based on deep neural networks provide powerful perception capabilities but offer limited support for encoding prior knowledge, enforcing safety constraints, or explaining decisions, as highlighted in recent terrain-segmentation surveys and applications to Mars and lunar robotics [1, 2, 3, 4]. Conversely, symbolic methods based on logical knowledge representation and reasoning provide explicit semantics and verifiable inferences [5], but lack the scalability and predictive power of deep learning. This tension motivates the neuro-symbolic framework developed in this thesis, in line with recent work on neuro-symbolic Artificial Intelligence (AI) and knowledge-graph reasoning [6, 7].

The main objective of this work is to design, implement, and evaluate an end-to-end **neuro-symbolic pipeline** for autonomous ground navigation in lunar environments integrating: (i) a lightweight deep model for semantic segmentation of lunar terrain, inspired by recent Mars and lunar segmentation networks [4, 8, 9, 10]; (ii) the novel Symbolic Planetary Exploration (SPEx) ontology in the *Attributive Language with unqualified Number restrictions and concrete Domains* ($\mathcal{ALN}(\mathcal{D})$) Description Logic (DL) for the planetary exploration domain; (iii) an embedded DL reasoner, *Tiny-ME*¹, offering both standard and non-standard reasoning services [11, 12, 13]; and (iv) a Universal Scene Description (USD) native digital twin of the environment for simulation and semantic 3D visualisation [14, 15, 16, 17]. A central technical goal is the construction of validated **semantic USD scenes**, where robotic space terrain exploration sensor data, ontology-based symbolic knowledge, and automated reasoning results are consistently embedded in per-frame USD assets. The pipeline is instantiated and evaluated on the multi-sensor lunar dataset *LuSNAR*² [4]. The resulting semantic USD representations are, by construction, compatible with simulators such as *NVIDIA Omniverse Platform (Omniverse)*³ and *NVIDIA Isaac Sim (IsaacSim)*⁴,

¹<https://swot.sisinflab.poliba.it/tinyme/>

²<https://github.com/zqyu9/LuSNAR-dataset>

³<https://www.nvidia.com/it-it/omniverse/>

⁴<https://developer.nvidia.com/isaac/sim>

which can mirror LuSNAR scenes, following current trends towards high-fidelity simulators for robotic perception and navigation [18, 19, 20, 21].

From a knowledge-representation perspective, the thesis introduces the **SPEx** ontology for planetary exploration, aligned with lightweight DL requirements [5] and with robotics-oriented standards such as the IEEE Ontology for Robotics and Automation⁵ [22, 23] and previous work on knowledge-based sensing and acting in robots [24]. SPEx models space terrain elements such as lunar surface patches, rocks, craters, and mountains; rover subsystems and sensors; quantitative descriptors derived from perception (slope, roughness, distances, object sizes); qualitative abstractions of those descriptors into symbolic categories; and task-specific concepts for traversability, hazards, and scientific interest. The ontology is explicitly tailored to the $\mathcal{ALN}(\mathcal{D})$ fragment of the Web Ontology Language (OWL) supported by Tiny-ME [11, 12], so that reasoning tasks remain tractable on embedded platforms while still providing a sufficiently rich conceptual vocabulary for navigation. Methodologically, its development follows established ontology engineering guidelines [5].

On the perception side, the thesis adopts a **SegFormer**⁶-based semantic segmentation module trained on LuSNAR images, in line with recent planetary and remote-sensing pipelines that combine transformer backbones and multi-sensor information [8, 10, 25]. The model produces pixel-wise terrain labels (regolith, rock, crater, mountain, sky), optionally complemented by confidence maps. Depth data and Light Detection and Ranging (LiDAR) point clouds provide geometric structure, following recent multi-modal navigation benchmarks [26, 27, 28]. A dedicated **Assertional Box (ABox) builder** module integrates segmentation masks, depth, and Inertial Measurement Unit (IMU) readings to extract object instances and compute geometric descriptors, mapping numeric values into the qualitative bins defined in the SPEx Terminological Box (TBox). Similar ontology-guided combinations of symbolic knowledge and low-level visual cues have been explored in remote sensing and medical imaging [29, 30, 31]. The resulting per-frame ABoxes, expressed in *OWL Functional Syntax*, [32] provide a logically consistent description of the scene and rover state.

Decision support and explainability are delegated to **Tiny-ME**, a multiplatform DL reasoner for embedded and resource-constrained systems [11, 12]. For each frame, Tiny-ME receives a merged Knowledge Graph (KG) comprising the SPEx TBox as background knowledge, dynamic rover profiles, goal specifications (rock pickup, obstacle avoidance, energy-saving navigation), and the perception-derived scene ABox. Tiny-ME performs both standard and non-standard reasoning services – most notably *Concept Abduction* and *Concept Contraction* [5, 11] – to compute graded compatibility scores and explanatory terms, indicating *e.g.* why a given terrain patch is safe, why a rock is suitable for a particular rover, or why a crater should be avoided.

⁵<https://github.com/srfiorini/IEEE1872-owl/tree/master>

⁶Simple and Efficient Design for Semantic Segmentation with Transformers: https://huggingface.co/docs/transformers/model_doc/segformer

At the implementation level, the thesis realizes the full neuro-symbolic loop as a dataset-driven pipeline operating on LuSNAR logs and Omniverse scenes, providing a C/C++ interface between USD stages, the SPEx KG, and Tiny-ME. The pipeline exports per-frame semantic USD assets that integrate geometric and semantic primvars, world-aligned poses, and embedded ABox fragments in *ProtocOWL* format [33], validated through byte-wise and hash-based checks inspired by recent USD-to-KG frameworks [17]. This yields self-contained semantic USD frames suitable for high-fidelity simulators such as IsaacSim, enabling semantic 3D inspection and what-if analysis. The thesis focuses on the robustness of this offline pipeline; a ROS 2⁷ deployment architecture is outlined at the node-graph level as a potential pathway toward a future on-board closed-loop implementation.

The thesis is driven by the following research questions:

- **RQ1:** *How can a lightweight $\mathcal{ALN}(\mathcal{D})$ -compliant ontology and an embedded DL reasoner be coupled with a deep segmentation backbone and a semantic USD workflow to support navigation in planetary environments while respecting resource constraints [11, 12, 13]?*
- **RQ2:** *To what extent can non-standard reasoning services (Concept Abduction (CA), Concept Contraction (CC), and related operators) provide meaningful, task-oriented explanations and graded compatibility scores for navigation decisions under perceptual uncertainty [5, 6]?*
- **RQ3:** *How large is the runtime overhead introduced by the symbolic and semantic USD layers, and is the resulting neuro-symbolic pipeline compatible with real-time execution on edge platforms, when compared with purely neural or purely symbolic baselines [11, 1, 4]?*

The main contributions of the thesis can be summarized as follows:

- **SPEx ontology for lunar exploration:** the design of the SPEx ontology covering terrain types, hazards, rover capabilities, and goal templates, explicitly tailored to the $\mathcal{ALN}(\mathcal{D})$ OWL 2 fragment supported by Tiny-ME [5, 11, 12];
- **LuSNAR-centred neuro-symbolic data model:** the definition of a LuSNAR-based data model in which semantic segmentation outputs, depth, LiDAR, and IMU measurements are systematically converted into OWL ABoxes and mapped to the qualitative bins of the SPEx TBox, following ontology-engineering best practices [5] and drawing inspiration from prior ontology-guided vision pipelines [25, 29, 30];
- **Semantic USD integration:** the development of `libusdsem`, a C/C++ bridge between USD scenes, the SPEx KG, and Tiny-ME, enabling automatic the construction and hash-based validation of per-frame semantic USD assets that embed ABox fragments and reasoning outputs in a manner consistent with the underlying ontology [17];

⁷<https://www.ros.org/>

- **ROS 2 deployment architecture:** the design of a ROS 2-based perception–reasoning loop with Tiny-ME, expressed as a deployment-ready architecture in which SegFormer (SegFormer)-based perception, ABox construction, reasoning, and USD-based visualisation are organised as ROS 2 nodes and topics;
- **Experimental evaluation:** an evaluation quantifying segmentation accuracy, inference latency, reasoning overhead, and the cost of semantic USD generation, and assessing the feasibility of the proposed pipeline on edge-class hardware.

The remainder of the thesis is structured as follows. **Chapter 1** reviews the theoretical foundations and the state of the art in neuro–symbolic AI, DLs, OWL, and lightweight embedded reasoning, with a focus on Tiny-ME and ontologies for autonomous robotics. **Chapter 2** surveys datasets and simulation environments for planetary exploration, introduces the LuSNAR dataset, and motivates its selection as the reference benchmark. **Chapter 3** presents the overall architecture and design of the neuro–symbolic pipeline, including the SPEx KG, the static and dynamic ABoxes, and their interaction with perception and USD/ROS 2. **Chapter 4** details the implementation of the end-to-end pipeline, covering the SegFormer-based perception module, the ABox builder, the `libusdsem` interface, and USD export. **Chapter 5** reports the full experimental evaluation. Finally, **Chapter 6** presents the conclusions and discusses potential directions for future work.

Chapter 1

Theoretical Foundations and State of the Art in Neuro-Symbolic AI

Chapter 1 outlines the theoretical foundations and prior work that motivate the *neuro-symbolic framework* adopted in this thesis for autonomous navigation in planetary environments. The discussion progresses from core principles of symbolic and sub-symbolic intelligence to specialised reasoning mechanisms and domain ontologies.

Section 1.1 introduces the distinction between symbolic and sub-symbolic paradigms and presents the main forms of neuro-symbolic integration that inspire the architecture adopted here. **Section 1.2** reviews essential notions of knowledge representation systems, description logics and OWL, emphasising the lightweight $\mathcal{ALN}$ family that underpins the reasoning layer. **Section 1.3** then examines the structure and expressivity of the $\mathcal{ALN}$ fragment and its datatype extension, providing the formal basis on which the reasoning services of **TinyME** operate. Building on this foundation, **Section 1.4** discusses both standard and non-standard reasoning services, clarifying how they contribute to classification, explanation and semantic matchmaking under embedded constraints. The discussion expands in **Section 1.6**, which surveys robotics-oriented ontologies and outlines design guidelines for $\mathcal{ALN}$ -compatible TBoxes tailored to navigation tasks. **Section 1.7** reviews recent neuro-symbolic approaches to perception and scene understanding, with specific attention to remote sensing and planetary rover vision. Finally, **Section 1.8** synthesises the surveyed literature and identifies open research challenges that motivate the overall architecture adopted in this thesis.

1.1 Symbolic and Sub-Symbolic Artificial Intelligence

Symbolic AI represents knowledge explicitly, using discrete and machine-interpretable structures such as logical formulae, production rules, frames and ontologies. A domain expert defines a vocabulary of concepts, relations and constraints in a formal language with model-theoretic semantics, typically a fragment of first-order logic

or a DL [5].

1.1.1 From Logic-Based Knowledge to Learned Representations

Classical *Knowledge Representation Systems (KRSs)* are built around three components [5]:

- a *Knowledge Representation Language (KRL)* with precisely defined syntax and semantics;
- a *Knowledge Graph* containing axioms and assertions;
- one or more sound and complete *reasoning procedures* with known complexity properties. In DLs-based KRSs, reasoning tasks such as *satisfiability*, *subsumption*, *classification* and *instance checking* are defined with respect to the set of interpretations (models) that satisfy the KG.

The main advantage of symbolic AI lies in this explicit semantics. Each assertion has a well-defined meaning in terms of models, and inference steps can be traced back to sound proof rules. This is crucial in domains where safety, explainability and regulatory compliance are essential, such as autonomous robotics and safety-critical control. Symbolic models are also modular and reusable: once a domain ontology is available, it can be shared across applications and aligned with standard vocabularies.

Sub-symbolic AI, by contrast, represents knowledge implicitly in the parameters of machine learning models. Deep neural networks, in particular, learn high-dimensional feature representations and decision boundaries directly from data. Convolutional Neural Networks and Vision Transformers have become the *de facto* standard for visual perception tasks, including semantic segmentation in terrestrial and planetary scenarios [1]. Their strength lies in approximating complex input–output mappings that are difficult or impossible to specify analytically.

The price, however, is limited interpretability. Internal representations are numerical patterns rather than logical statements; they lack explicit model-theoretic semantics. Enforcing hard constraints (for example, that hazardous regions must be non-traversable, or that two rigid objects cannot occupy the same space) typically requires indirect mechanisms such as losses, regularizers or post-processing, and is not guaranteed in general. In many deep learning pipelines, perception and decision-making are treated as black-box optimisation problems, which complicates formal verification and robustness analysis to huara.

The *neuro-symbolic* paradigm seeks to combine the complementary strengths of these two approaches. Neural networks provide flexible, data-driven perception and pattern recognition; symbolic components supply structured domain knowledge, logical inference and outcome explainability. In the framework considered in this thesis, the neural part is a lightweight semantic segmentation model, while the symbolic part is an $\mathcal{ALN}(\mathcal{D})$ -ontology with simple numeric constraints, processed by the Tiny-ME inference engine [11].

1.1.2 Hybrid Neuro-Symbolic Paradigms and Design Axes

Existing *neuro-symbolic systems* can be roughly classified according to how tightly neural and symbolic components are coupled and how information flows between them [6].

Loosely coupled pipelines. In the most common pattern, neural models produce outputs (labels, detections, attributes) that are converted into symbolic assertions and processed by an ontology reasoner. Information flows essentially in one direction: from perception to reasoning. Many *ontology-guided image understanding* systems adopt this architecture. In remote sensing, convolutional networks provide pixel-wise labels that are subsequently refined by OWL reasoners enforcing ontological constraints and correcting inconsistent labellings [25, 29]. In medical imaging, decisions from multiple networks are fused and checked against a domain ontology before issuing a final diagnosis [30]. This pattern is compatible with existing DL reasoners and is also assumed here.

Symbol-guided learning. A second family injects symbolic knowledge into the learning process. Logical rules are encoded as regularisation terms in the loss function or translated into differentiable approximations, so that constraint violations are penalised during training [6, 7]. Neural parameters and symbolic constraints are optimised jointly. These approaches can improve data efficiency and generalisation but usually require powerful hardware and differentiable approximations of reasoning, which is misaligned with embedded, real-time settings.

End-to-end differentiable reasoning. A third class of methods replaces explicit symbolic reasoning with neural architectures operating over vector embeddings of symbols, often described as *neural theorem provers* or *differentiable knowledge graph reasoners* [6]. Such methods can scale to large KGs, but typically sacrifice formal guarantees like soundness and completeness. Training and inference costs are usually too high for resource-constrained robots.

Designing a neuro-symbolic system therefore requires trade-offs along several axes:

- **Expressivity vs. tractability:** highly expressive logics, such as those underlying OWL 2 DL, enable rich modelling but come with high worst-case complexity [5]. Lightweight fragments such as the $\mathcal{ALN}$ family restrict the language in exchange for predictable performance [11].
- **Accuracy vs. explainability:** deep networks excel at perceptual accuracy, whereas symbolic reasoners support traceable explanations. A pipeline architecture delegates low-level interpretation to neural models and uses the ontology for higher-level semantic classification and decision support.
- **Offline vs. real-time reasoning:** many existing neuro-symbolic pipelines run reasoning offline on pre-computed segmentations [25, 29]. Autonomous

navigation, in contrast, requires inference under tight timing budgets. Tiny-ME is explicitly designed for embedded use and offers a C Application Program Interface (API) for integration inside control loops [24, 11].

Given these considerations, the work in this thesis adopts a *loosely coupled architecture*: the segmentation network processes Red Green Blue (RGB)-D data and produces symbolic evidence; Tiny-ME, operating over an $\mathcal{ALN}$ -family ontology with simple concrete datatypes, performs standard and non-standard reasoning to support navigation decisions.

1.2 Knowledge Representation Systems, OWL and Lightweight Encodings

1.2.1 TBox and ABox for KRSs

DLs-based KRSs distinguish between *terminological* and *assertional* knowledge [5]:

- the *TBox* (terminological box) contains axioms that define and relate concepts and roles. It plays a role analogous to a schema or ontology;
- the *ABox* (assertional box) contains ground facts about individuals, typically derived from perception, sensors or other data sources.

In an autonomous navigation scenario, the TBox encodes terrain types, hazards, traversability criteria and robot capabilities, while the ABox is continuously updated with individuals representing cells, obstacles, landmarks and dynamically perceived entities. This separation fits naturally with planetary exploration, where conceptual structure evolves slowly but perceptual evidence changes at high frequency.

A KRS is expected to satisfy several properties [5]:

- *soundness*: all derived conclusions are logically entailed by the KG;
- *completeness*: all entailed conclusions can be derived by the implemented procedures;
- *decidability*: the main reasoning tasks terminate with a definite answer;
- acceptable *computational complexity* given the target platform.

The choice of DL fragment and algorithms in Tiny-ME is shaped by these requirements.

1.2.2 OWL, Profiles and Binary Encodings

The **OWL** is the W3C standard for publishing and sharing ontologies on the Web. OWL 2 DL corresponds, modulo syntactic differences, to the DL SROIQ with Datatypes (SROIQ(D)) and supports features such as role hierarchies, transitivity, inverse roles and qualified cardinality restrictions [5]. While this expressivity is valuable, it entails high worst-case complexity for many reasoning tasks.

To address performance concerns, OWL 2 defines three **profiles**: OWL 2 Expressive and Lightweight (EL), OWL 2 Query Language (QL) and OWL 2 Rule Language (RL). Each profile restricts the available constructors and role axioms so that core reasoning tasks are tractable in polynomial time for typical implementation strategies. EL targets large taxonomies; QL is suited for ontology-based data access; RL favours rule-based implementations. The spirit of the approach in this thesis is similar: adopt a fragment with controlled expressivity and tractable reasoning, in this case the $\mathcal{ALN}$ family with simple numeric constraints, and design a reasoner closely aligned with that fragment [11].

OWL ontologies can be serialised in several concrete syntaxes (Resource Description Framework (RDF)/Extensible Markup Language (XML), Turtle, OWL 2 Functional-Style Syntax (OWL Functional), Manchester Syntax). For embedded and distributed architectures, however, compact binary encodings and low parsing overhead are crucial. The Cowl library provides a binary representation for an OWL 2 subset tailored to constrained devices [13]. It is designed to:

- minimise memory usage during parsing and reasoning;
- support streaming and incremental updates for both TBox and ABox;
- preserve a clean mapping to OWL 2 semantics to maintain interoperability.

On the protocol side, OWL API Communication Protocol (OWLink) offers an HTTP-based interface to OWL reasoners. In the context of the **Semantic Web of Everything (SWoE)**, Tiny-ME uses lightweight OWL data streams encoded with ProtocOWL (ProtocOWL), a compact binary format and frame-based protocol for semantic data interchange, to exchange KG fragments and reasoning requests with external components [11]. This enables the reasoner to be deployed either locally on-board or as a service on more capable nodes, without changing the logical core.

1.3 Description Logics and the $\mathcal{ALN}$ Fragment

1.3.1 Constructors and Expressivity

DLs describe domains using **concepts** (unary predicates), **roles** (binary predicates) and, in many cases, **datatypes** (concrete domains) [5]. The $\mathcal{ALN}$ logic provides a restricted but useful set of constructors. Concepts in $\mathcal{ALN}$ are built from:

- atomic concepts A , the universal concept $\top$ and the bottom concept $\perp$;

- conjunction $C \sqcap D$;
- negation of atomic concepts $\neg A$;
- universal restrictions $\forall R.C$, constraining all R -successors to instances of C ;
- unqualified number restrictions $(\geq nR)$ and $(\leq nR)$, bounding the number of R -successors of an individual.

The $\mathcal{ALN}(\mathcal{D})$ extension introduces *concrete domains* and *datatype roles*, which allow concepts to constrain numeric attributes such as slope, roughness, distance or energy consumption [11]. In Tiny-ME, datatype predicates are usually interpreted as intervals or simple inequalities over real numbers. Compared with more expressive DLs such as SHOIN with Datatypes (SHOIN(D)) or SROIQ(D), $\mathcal{ALN}(\mathcal{D})$ cannot express complex role axioms or qualified number restrictions. Nevertheless, it is sufficient to model many taxonomies and local constraints relevant for embedded reasoning in robotics, while enabling structural reasoning algorithms with favourable complexity [11, 5].

1.3.2 Roles, Datatypes and Numeric Constraints

Roles in DL-based ontologies are typically divided into:

- *object properties*, relating individuals (e.g., adjacency relations between cells, observation relations between objects and sensors);
- *datatype properties*, relating individuals to concrete values (e.g., numeric attributes such as slope or distance).

In pure $\mathcal{ALN}$, number restrictions constrain the cardinality of object properties:

$$(\geq nR), \quad (\leq mR).$$

In $\mathcal{ALN}(\mathcal{D})$, datatype constraints add bounds or conditions on concrete attribute domains. A practical representation, used in Tiny-ME, is to restrict datatype predicates to intervals and simple inequalities, so that constraints can be processed efficiently by interval-based algorithms [11]. This is adequate for many navigation-related thresholds.

In autonomous navigation, object properties model qualitative spatial structure (adjacency, support, containment), while datatype properties encode quantitative measurements derived from sensors. An $\mathcal{ALN}(\mathcal{D})$ ontology provides a single formalism to combine both facets.

1.3.3 Normal Forms and Structural Reasoning

An important feature of $\mathcal{ALN}(\mathcal{D})$ is the existence of convenient *normal forms* for concepts and TBox axioms. Tiny-ME exploits a *Conjunctive Normal Form (CNF)*

tailored to its structural inference algorithms [11]. Informally, a CNF-normalised concept can be written as:

$$C \equiv \bigwedge_i A_i \sqcap \bigwedge_j \neg B_j \sqcap \bigwedge_k \forall R_k. C_k \sqcap \bigwedge_\ell (\geq n_\ell R_\ell) \sqcap \bigwedge_m (\leq m_m R_m) \sqcap \bigwedge_p d_p,$$

where A_i and B_j are atomic concepts, C_k are recursively normalized concepts, and d_p denote datatype constraints.

Normalization typically involves:

- **unfolding** of defined concepts, replacing names with their definitions;
- **flattening** nested conjunctions and simplifying trivial constructs;
- removal of redundancies and detection of obvious contradictions.

Once concepts are in CNF, **subsumption** can be reduced to checking syntactic and numeric inclusions: $C \sqsubseteq D$ holds if every constraint in the normal form of D is matched by an equal or stronger constraint in the normal form of C [5, 11]. This structural approach avoids general tableaux procedures and leads to polynomial-time algorithms for suitably restricted TBoxes (e.g., acyclic or primitive) [5].

When concrete domains are added, complexity depends on the allowed datatype operations. Tiny-ME restricts these so that datatype reasoning remains tractable and compatible with embedded hardware [11]. This design is central to the feasibility of real-time reasoning in the proposed framework.

1.4 Reasoning Services in Description Logics

This section recalls the most relevant standard and non-standard DL reasoning services for the proposed framework.

1.4.1 Standard Services

Table 1.1: Standard DL reasoning services relevant for Tiny-ME.

Service	Informal description
Concept Satisfiability	Checks whether a concept description is logically consistent with the TBox, i.e., whether it can have at least one instance.
Subsumption	Determines whether all instances of a concept C must also be instances of a concept D , given the TBox.
Classification	Computes the subsumption hierarchy of named concepts, yielding a taxonomy of the domain.
Instance Checking	Verifies whether a given individual is necessarily an instance of a concept, given the KG.
Realisation	Finds the most specific named concepts that an individual instantiates, providing fine-grained semantic types.

Standard DL reasoning services form the backbone of ontology-based systems [5], as summarised in Table 1.1. The most relevant ones are:

- **Concept Satisfiability (Concept Satisfiability (CS))**: given a concept C and a TBox $\mathcal{T}$, determine whether C has a non-empty extension in some model of $\mathcal{T}$;
- **Subsumption**: given concepts C and D , decide whether $C \sqsubseteq D$ in all models of $\mathcal{T}$;
- **Classification**: compute the subsumption hierarchy induced by all named concepts in the TBox;
- **Instance Checking**: given an individual a and a concept C , decide whether a is an instance of C in all models of the KG;
- **Realisation**: for an individual a , identify the most specific named concepts in $\mathcal{T}$ that a instantiates.

In the context of this thesis, CS and subsumption are mainly used at design time to validate and maintain the TBox. At run time, instance checking and realisation are crucial: they map ABox assertions derived from perception into higher-level semantic categories, such as distinguishing traversable cells from hazards. A compact summary of these services is reported in Table 1.1.

1.4.2 Non-Standard Services: Abduction, Contraction and Related Operators

Table 1.2: Non-standard DL reasoning services provided by Tiny-ME.

Service	Informal description
Concept Abduction (CA)	Computes minimal additional assumptions needed for an offered concept to satisfy a request.
Concept Contraction (CC)	Identifies the minimal part of a request that must be retracted to restore consistency with an offered concept.
Concept Covering (Concept Covering (CCov))	Selects a set of offered concepts whose conjunction best approximates a request concept.
Concept Bonus (Concept Bonus (CB))	Rewards offered concepts that contribute additional, desirable properties beyond the strict request requirements.
Concept Difference (Concept Difference (CD))	Describes what remains of a request after it has been covered by a set of offered concepts.

Tiny-ME supports a set of **non-standard reasoning services** originally developed for semantic matchmaking in e-commerce scenarios and later applied to Internet of Things (IoT) and robotics [24, 11]. In the following, these operators will

be referred to by the acronyms CA, CC, CCov, CB and CD, as summarised also in Table 1.2. These services are particularly valuable in Open World Assumption (OWA) settings with incomplete or noisy information. The most relevant ones are:

- **CA**: given a request Q and a resource R , compute a minimal hypothesis H such that $R \sqcap H \sqsubseteq Q$; intuitively, H captures what is missing in R for it to satisfy the request;
- **CC**: when $Q \sqcap R$ is unsatisfiable, decompose $Q \equiv K \sqcap G$ so that $K \sqcap R$ is satisfiable and G is minimal; G represents the part of the request that must be relaxed to restore compatibility;
- **CCov**: select a set of requests that jointly approximate a request as closely as possible;
- **CB**: a *concept bonus* operator that augments the matching process with additional positive evidence, rewarding resources that provide desirable properties beyond the strict requirements of the request;
- **CD**: quantify what part of a request remains uncovered after the covering step, describing the residual semantic gap.

These non-standard services (CA, CC, CCov, CB, CD) provide the logical basis for reasoning about gaps, relaxations and trade-offs between ideal and feasible behavior.

1.5 Tiny-ME for Embedded Semantic Reasoning

Tiny-ME is a multiplatform reasoning engine designed for the SWoE, with explicit support for constrained devices and heterogeneous deployment environments [11]. It is implemented in standard C11, has minimal external dependencies and targets an $\mathcal{ALN}$ -family fragment with simple concrete datatypes.

1.5.1 Architecture, C API and Internal Representation

Internally, Tiny-ME stores concepts in a normalised structural form, often tree- or DAG-based, whose nodes represent atomic concepts, negations, universal restrictions, number restrictions and datatype constraints [11]. During TBox loading, the reasoner:

- unfolds definitions and normalises all concept descriptions;
- builds indexes and hash structures to accelerate subsumption and comparison;
- performs preliminary consistency checks.

The public *C API* exposes primitives to:

- initialize and configure the reasoner, including datatype domains and memory parameters;
- load TBox axioms and ABox assertions from in-memory structures or byte streams;
- invoke standard reasoning services;
- invoke non-standard services (CA, CC, CCov, CB, CD) with configurable penalty functions.

This low-level interface allows Tiny-ME to be integrated into embedded firmware, robotics middleware and higher-level languages. In the context of this thesis, it provides the semantic reasoning layer on board the rover platform.

ProtocOWL [33] is a compact encoding format and transmission protocol for OWL 2 KGs, enabling dynamic trade-off between compression ratio and encoder memory usage. Through the *Cowl*¹ [13] OWL manipulation library, Tiny-ME supports reading and writing OWL 2 TBoxes and ABoxes in ProtocOWL natively.

1.5.2 Complexity Profile and Resource Usage

Tiny-ME is engineered to operate within tight resource budgets [11]. Its design includes:

- restriction to the $\mathcal{ALN}(\mathcal{D})$ family, enabling *structural reasoning* with polynomial-time complexity for core tasks;
- careful control of dynamic memory allocations, often using pre-allocated pools and bounded data structures;
- data layouts tuned for cache locality and predictable control flow.

Empirical evaluations reported in [11] show that Tiny-ME can handle KGs with hundreds of concepts and individuals on devices with limited Random Access Memory (RAM), including microcontroller-class hardware, with reasoning times compatible with real-time loops. Earlier work illustrates the feasibility of employing non-standard reasoning in the control loops of mobile robots [24].

In this thesis, Tiny-ME is intended to run on single-board computers with embedded Graphics Processing Unit (GPU) platforms such as NVIDIA Corporation (NVIDIA) Jetson boards. The neural segmentation model uses the GPU, while Tiny-ME runs on the Central Processing Unit (CPU) with modest computational footprint. This parallelization of labor in heterogeneous computing architectures is essential for maintaining real-time constraints for the target scenarios.

¹<https://swot.sisinflab.poliba.it/cowl/>

1.5.3 Integration with Cowl and Streamed ABox Updates

In distributed systems and multi-robot scenarios, KG fragments must be exchanged efficiently between components. Cowl provides a compact OWL 2-based encoding with reduced parsing overhead, suitable for low-resource hardware [13]. It supports:

- binary serialisation of both TBox axioms and ABox assertions;
- streaming and incremental updates, allowing the KG to evolve over time;
- direct correspondence with standard OWL axioms.

Tiny-ME can consume Cowl-encoded ontologies via ProtocOWL, which defines lightweight messages and reasoning operations for SWoE deployments [11]. This architecture enables different deployment strategies:

- fully on-board reasoning, where perception and reasoning share the same hardware;
- offloaded reasoning, where a robot streams ABox updates to a more powerful node hosting Tiny-ME or a compatible reasoner;
- hybrid configurations with local reasoning for time-critical tasks and remote reasoning for more complex analysis.

Even when Tiny-ME is used via direct in-process calls, the same conceptual pattern applies: perception modules generate and stream ABox updates to the reasoner, which returns inferred classifications or warnings.

1.6 Ontologies for Autonomous Robotics

Robots act in continuous, metric space, yet logics such as $\mathcal{ALN}(\mathcal{D})$ provide a qualitative abstraction. Ontologies for robotics must therefore connect symbolic concepts with geometric models.

1.6.1 IEEE 1872–2015 and CORA

The *IEEE Std 1872–2015* defines a family of ontologies for Robotics and Automation (R&A), collectively known as Core Ontology for Robotics and Automation (CORA) [22]. CORA provides a shared vocabulary for describing robots, environments, tasks and capabilities, with the aim of promoting interoperability and reuse. It comprises:

- a **core ontology** specifying foundational classes such as *Robot*, *Environment*, *Sensor*, *Actuator*, *Task* and *Capability*;
- **domain-specific ontologies** extending the core for particular application domains (industrial robots, service robots, autonomous vehicles, etc.).

CORA is formulated in OWL and follows a modular design. Aligning domain ontologies with CORA eases integration with other systems and datasets. For planetary rovers, for example, concepts such as *LunarRover* or *TraversableLunarTerrain* can be modelled as subclasses of appropriate CORA classes, preserving compatibility with general R&A frameworks [23].

1.6.2 Reference Frames, Units and Spatial Relations

For connecting symbolic concepts with geometric models, IEEE 1872 and related works emphasize three aspects [22, 23]:

- **reference frames:** world frames, robot bases, sensor frames and task-specific frames must be represented to give meaning to positions and orientations;
- **measurement units:** semantic descriptions should be explicit about physical units (metres, degrees, radians, etc.) to avoid ambiguity and support consistency checks;
- **spatial relations:** qualitative relations such as *inside*, *adjacentTo*, *above* or *withinDistance* are essential to express navigation constraints and task conditions.

In DL ontologies, reference frames can be modelled as individuals, and spatial relations as object properties. Numeric attributes like slope or distance are represented by datatype properties such as *hasSlope* or *hasDistance*. Thresholds relevant to navigation are encoded via datatype constraints in concept definitions. Popular frameworks include *Region Connection Calculus 8 (RCC8)* [34], a set of 8 possible relationships between spatial regions.

For planetary exploration, the ontology must be compatible with coordinate systems used both in simulation world coordinates and on-board localisation frames. Geometric computation is not replaced by the ontology; instead, the ontology provides a semantic layer that reasons about the implications of measured quantities.

1.6.3 Guidelines for $\mathcal{ALN}(\mathcal{D})$ -Compliant TBox Design

Ontology engineering in $\mathcal{ALN}(\mathcal{D})$ follows standard methodologies [5] but under stricter constraints imposed by Tiny-ME. The design principles adopted in this thesis include:

- **controlled vocabulary size:** the number of primitive and defined concepts is limited to keep reasoning fast and memory usage bounded;
- **simple role structure:** complex role axioms and role chains are avoided; relations such as adjacency are either pre-computed or encoded as simple object properties without additional constraints;

- **local constraints:** universal and number restrictions refer to local properties of individuals and their immediate neighbours; global constraints are handled outside the DL layer if required;
- **static vs. dynamic separation:** the TBox captures static domain knowledge (terrain taxonomies, hazard definitions), while dynamic information (sensor readings, rover pose, time-varying conditions) resides in the ABox and is updated incrementally;
- **alignment with standards:** where possible, key concepts are aligned with CORA and IEEE 1872–2015 [22, 23].

These guidelines ensure that the resulting TBox is compatible with Tiny-ME, can be processed repeatedly in real time and remains maintainable as domain knowledge evolves.

1.7 State of the Art in Neuro-Symbolic Perception and Navigation

1.7.1 Ontology-Guided Segmentation and Multimodal Fusion

Several works integrate ontological reasoning with image segmentation and classification, especially in remote sensing and medical imaging.

Li *et al.* combine DeepLab-based segmentation with an OWL 2 ontology to refine pixel-wise labels in remote sensing [25]. The CNN produces initial land-cover predictions, which are mapped to ontological concepts. An OWL reasoner then enforces semantic constraints derived from the class hierarchy and additional domain knowledge, correcting inconsistent labels and regularising the segmentation.

Zand *et al.* propose OBSIS, a framework where multiple mixture-model and CRF-based segmenters produce intermediate label distributions over superpixels [29]. These distributions are combined and constrained by an ontology capturing semantic relations that are difficult to encode directly in CRF potentials. Reasoning helps to resolve conflicts between modalities and to enforce global consistency.

Yahyaoui *et al.* use an ontology to guide multimodal fusion in medical image classification [30]. Separate networks process different imaging modalities, and their outputs are fused through an ontology representing diseases, anatomical structures and clinical relationships. Ontological reasoning filters inconsistent predictions and improves robustness.

Comber *et al.* extend ontology-driven geographic object-based image analysis (O-GEOBIA) by incorporating machine learning to automatically derive classification rules [31]. An ontology defines geographic object types, while learned rules link feature patterns to classes. The ontology both constrains rule generation and structures the interpretation of results.

These works demonstrate that combining sub-symbolic perception with symbolic knowledge can improve semantic coherence and interpretability. However, they typically rely on expressive OWL 2 DL ontologies, operate offline and assume server-grade hardware. They do not aim at lightweight logics such as $\mathcal{ALN}(\mathcal{D})$ or embedded reasoners.

1.7.2 Geospatial Knowledge Graphs and Neuro-Symbolic Methods

KGs provide a flexible way to represent structured information about scenes, emphasising relational patterns and graph algorithms. Potnis *et al.* propose a conceptual framework for geospatial KG-infused neuro-symbolic AI in remote sensing [7]. Geographical entities such as roads, land parcels and rivers are nodes in a KG, with edges encoding semantic and spatial relations. Neural models extract candidate labels and relations; KG reasoning and graph embeddings refine predictions and capture higher-order dependencies.

DeLong *et al.* survey neuro-symbolic approaches for reasoning over KGs [6]. They classify methods into logic-based, embedding-based and hybrid approaches and analyse trade-offs between semantic faithfulness and scalability. Logic-based systems, such as DL reasoners, provide strong guarantees but usually target moderate-sized KGs; embedding-based systems scale to large KGs but have weaker semantic guarantees. The Tiny-ME-based framework proposed in this work lies on the logic-based side.

Large-scale benchmarks for 3D semantic scene completion in autonomous driving [28] show the potential for representing complex scenes with rich semantic structure. Although such benchmarks do not explicitly employ ontologies or KGs, they highlight the value of dense semantic representations that could be connected to logical reasoning.

1.7.3 Terrain Semantic Segmentation for Planetary Rover Vision

Planetary exploration has motivated several works on semantic segmentation of Martian terrain. These methods are predominantly sub-symbolic but provide the perceptual foundation for neuro-symbolic pipelines in this domain.

S5Mars introduces a semi-supervised learning strategy for Martian terrain segmentation based on sparsely annotated images [9]. The framework uses consistency regularisation and domain-specific augmentations to exploit unlabeled data, alleviating annotation scarcity.

MarsSeg [8] proposes an encoder–decoder architecture with multi-level feature extractors and dedicated modules (*e.g.*, aggregation and pyramid structures) to capture multi-scale context in Mars images [8]. The approach explicitly addresses class imbalance through loss design and architectural choices.

SegMarsViT leverages lightweight Vision Transformers with cross-scale feature fusion to achieve an effective trade-off between accuracy and computational cost [10]. It is explicitly designed for deployment in resource-constrained environments, making it particularly relevant for on-board rover use.

Kuang *et al.* provide a systematic review of semantic terrain segmentation methods for planetary rovers [1]. The survey covers classical image-processing pipelines and modern deep learning architectures, and analyses open challenges such as class imbalance, domain shift and the gap between simulated and real data.

Barrett *et al.* develop a deep-learning terrain classification system based on orbital imagery to support landing site assessment for planetary missions [2]. The authors define a taxonomy of terrain classes grounded in geomorphological criteria and train convolutional networks to classify pixels accordingly.

Kuang *et al.* also propose a rock segmentation pipeline that combines synthetic data from a game engine with real rover images [3]. A modified U-Net++ model is trained on synthetic data and adapted to real imagery, demonstrating the value of simulation in training planetary perception systems.

These works provide high-quality sub-symbolic terrain understanding but treat semantic classes as labels rather than as formally defined concepts in an ontology. Logical reasoning on top of segmentation outputs is not performed; the semantics of classes remains implicit in the network weights and dataset documentation.

1.8 Critical Synthesis and Research Gaps

The literature reviewed in this chapter reveals several trends and corresponding gaps relevant to neuro-symbolic perception and navigation.

1. Ontology-guided segmentation and multimodal fusion in remote sensing and medical imaging show that symbolic knowledge can regularize neural predictions and improve semantic coherence [25, 29, 30, 31]. However, these systems rely on expressive OWL 2 DL ontologies, operate mainly offline and assume server-grade hardware. They do not target embedded platforms or lightweight fragments such as $\mathcal{ALN}(\mathcal{D})$.
2. Geospatial KGs and neuro-symbolic methods illustrate the benefits of structured graph representations and hybrid reasoning for scene understanding [7, 6, 28]. Their focus is on scalability to large KGs rather than on strict real-time and resource-constrained on-board robots.
3. Planetary terrain segmentation has made significant progress in sub-symbolic perception [9, 8, 10, 1, 2, 3], yet the semantics of classes remains informal: there is no explicit DL-based ontology aligned with the taxonomies used in these works, nor is reasoning systematically applied to segmentation outputs to support navigation decisions.

4. Tiny-ME and its predecessors have shown that non-standard reasoning services (CA, CC, CCov, CB, CD) are effective for context-aware decision-making in mobile robots and IoT [24, 11]. To date, these services have mostly been applied to symbolic descriptions of capabilities or contexts, rather than to semantics derived from deep visual perception in planetary exploration.

Taken together, these observations highlight the need for lightweight, explainable and embedded neuro-symbolic architectures that connect modern perception models with formally grounded ontologies and reasoning services, under the computational and timing constraints typical of autonomous planetary rovers.

Chapter 2

Datasets and Simulation for Planetary Exploration

The chapter analyses the datasets and simulation environments considered in this thesis for the design and evaluation of a neuro-symbolic pipeline for autonomous robot navigation in planetary scenarios. The focus is on the joint viewpoint of *data, symbols, and simulation*: semantic class coverage, sensing modalities, spatial structure, symbolic compatibility, and simulability in USD-based environments such as *Omniverse* [16], *IsaacSim* [15], and *Omniverse Lunar Robotics Simulator (OmniLRS)* [14]. The cataloging work is guided by a common template, explicitly designed to capture, for each dataset, (i) perception-oriented metadata, (ii) symbolic-oriented metadata, and (iii) simulation-oriented metadata relevant for USD-based digital twins.

Implementation details of neural models, ontologies, and reasoning services are deferred to later chapters. Here, datasets are evaluated only in terms of their suitability as the backbone of a neuro-symbolic pipeline and as a reference for reconstructing lunar-like scenes in Omniverse.

The chapter is organized as follows. **Section 2.1** derives requirements for semantic datasets at the intersection of autonomous navigation, neuro-symbolic reasoning, and edge deployment. **Section 2.2** describes the cataloging methodology adopted to compare ten datasets (DS01–DS10). **Section 2.3** summarises their main properties, with particular emphasis on how they score along the three axes of perception, simulation and symbolic reasoning. **Section 2.4** motivates the choice of LuSNAR as the reference benchmark for this work. **Section 2.5** refines its characterisation from the viewpoint of neuro-symbolic metadata and ABox/TBox mappability, without reproducing the original paper. **Section 2.6** introduces OmniLRS and IsaacSim as the simulation back-end aligned with LuSNAR and with the USD/KG-based reasoning strategy. Finally, **Section 2.7** recalls the metrics used for 2D and 3D perception, which will be reused in the experimental chapter.

2.1 Requirements for Semantic Datasets in Neuro-Symbolic Navigation

The overall framework described in **Chapter 4** combines: (i) deep models for semantic segmentation, (ii) reasoning on an $\mathcal{ALN}(\mathcal{D})$ -based ontology executed by Tiny-ME, and (iii) a USD-native simulation environment. From this perspective, the reference dataset must satisfy more constraints than those typically imposed for pure perception or pure Simultaneous Localisation and Mapping (SLAM) benchmarks. In particular, it must provide data and metadata that can be used *coherently* along the whole pipeline: from pixel/point-level semantics, to symbolic instances, to USD prims in Omniverse.

The following dimensions are particularly relevant.

2.1.1 Semantic Coverage, Class Imbalance, and Environmental Variability

Semantic class coverage. Semantic labels must capture the terrain and object categories that matter for traversability, hazard detection, and scientific interest. Reviews of planetary terrain segmentation consistently report classes such as regolith/soil, sand, bedrock, rocks/boulders, craters, ridges, rover body, tracks, and sky as central for navigation and geomorphology [1, 2]. Martian datasets like **AI4Mars** [35] and **S5Mars** [9] adopt compact taxonomies (soil, sand, bedrock, rock, sky, rover, trace, hole), while **LuSNAR** focuses on lunar regolith, rocks, impact craters, mountains and sky, explicitly tuned to navigation and 3D reconstruction [4].

The semantic space must be:

- *navigation-aware*: classes distinguish safe vs. hazardous terrain (*e.g.*, flat regolith vs. steep crater walls);
- *ontology-friendly*: each class has a clear conceptual meaning, so that it can be mapped to DL concepts in the SPEx ontology (*e.g.*, `SpEx:Mountain`, `SpEx:Rock`, `SpEx:Crater`);
- *simulation-friendly*: each class corresponds to assets that can be instantiated in USD scenes (meshes, materials, collision shapes).

Class imbalance. Real and realistic environments are strongly imbalanced: most pixels/points belong to traversable terrain (regolith, soil), while hazards such as sharp rocks, holes or crater rims are sparse. This is documented for Martian datasets [1, 9] as well as for terrestrial off-road datasets such as **RELLIS-3D** [26]. A usable dataset must:

- expose this long-tail structure, so that learned models reflect the true statistics of the domain;

- provide enough samples for rare but safety-critical classes (holes, steep slopes, cluttered rocks);
- enable loss re-weighting and sampling strategies to mitigate imbalance during training.

The catalog therefore records not only the list of classes, but also the qualitative distribution of frequent vs. rare categories and, when available, statistics from the reference papers.

Environmental variability. Generalization beyond a single landing site or analogue field is a major challenge for planetary rovers [1]. Datasets are thus evaluated in terms of:

- *terrain diversity*: flat vs. rugged topography, sparse vs. dense rocks, presence or absence of craters and ridges;
- *visual variation*: changes in illumination, cast shadows, camera pose, and sensor noise;
- *scene diversity*: different scenes or sites, possibly spanning distinct geological units.

Synthetic datasets such as **TartanAir** [21] and **LuSNAR** [4] are explicitly designed with domain randomisation and multiple scenes, whereas real datasets like **RELLIS-3D** or **M2DGR** explore variability via multi-sequence field campaigns [26, 27]. In LuSNAR, nine lunar scenes are organized along two axes—topographic relief and object density—to span environments from gently undulating, rock-sparse plains to steep, rock- and crater-rich terrains [4].

2.1.2 Symbolic Compatibility and ABox/TBox Mappability

The proposed pipeline requires that perceptual outputs (pixel-level or point-level labels) can be lifted into a formal Knowledge Base. The underlying $\mathcal{ALN}$ -based ontology distinguishes a **TBox** (conceptual schema) and an **ABox** (assertions about individuals), and Tiny-ME supports both standard and non-standard reasoning services over this structure [5, 11]. This motivates three further requirements.

Conceptual clarity and ontological alignment. Labels must be well-defined enough to correspond to domain concepts. Datasets such as AI4Mars and S5Mars document clear definitions for bedrock, sand, rock, sky, rover, trace and hole classes [35, 9], enabling their direct codification as DL concepts (*e.g.*, disjointness constraints between terrain types). Similarly, LuSNAR partitions the lunar scene into regolith, rocks, impact craters, mountains and sky, informed by Apollo and Yutu-2 observations [4]. Datasets with loosely defined or application-specific labels are harder to integrate into a general ontology.

Instance granularity and structural information. For symbolic reasoning it is often useful to talk about specific entities—`:rock_42`, `:crater_7`—rather than just classes. Datasets differ widely in the granularity they offer:

- 3D indoor datasets (HM3D, Replica, HSSD-200) provide instance-level annotations, making ABox construction straightforward [18, 19, 20];
- voxel or grid-based datasets (SSCBench) are conceptually rich but require bespoke processing to extract object candidates from voxel labels [28];
- planetary datasets such as AI4Mars and S5Mars are 2D-only, so instance discovery requires additional segmentation on top of pixel-wise labels [35, 9];
- LuSNAR offers semantically labeled point clouds and dense depth maps, which can be clustered into object instances and enriched with spatial attributes (centroid, bounding box, local slope) [4].

Support for relational predicates. Reasoning about navigation requires more than unary concepts: the ontology must encode relations like *adjacent-to*, *inside*, *on-slope-of*, *near-rover*, and so on [24]. These relations can be computed from:

- 3D geometry (meshes or dense point clouds);
- calibrated sensor extrinsics and intrinsics;
- time-synchronized poses, IMU, and Odometry.

Datasets lacking 3D structure (*e.g.*, AI4Mars, S5Mars) can still support partial relational reasoning, but at lower fidelity and only under strong assumptions on camera pose. Multi-modal datasets such as RELIS-3D, TartanAir, and especially LuSNAR provide the necessary information to derive a 3D scene graph suitable for alignment with the ontological TBox and ABox population [26, 21, 4].

2.1.3 Simulation and USD/KG Integration

A final requirement is compatibility with a USD-based digital twin and with symbolic representations derived from USD scene graphs [17]. In this thesis, simulation and reasoning are tightly coupled and share the same semantic backbone:

- environments are represented as USD stage graphs in IsaacSim/OmniLRS, with *prims* (primitives) corresponding to terrain tiles, rocks, craters, rover body parts, and sensors;
- these prims are annotated with semantic and relational metadata, which can be translated into OWL/ $\mathcal{ALN}$ -based knowledge graphs and queried by the reasoning engine;

Datasets with existing 3D assets in Graphics Language Transmission Format (glTF), OBJ or similar formats (Replica, HM3D, HSSD-200) are naturally compatible with this strategy [19, 18, 20], while purely 2D datasets require manual

scene reconstruction. LuSNAR occupies an intermediate position: its scenes are authored in Unreal Engine 4 (UE4) with AirSim (AirSim), so the same geometric structure can be conceptually re-instantiated or approximated in USD, with LuSNAR data acting as semantic and geometric reference [4].

2.2 Comparative Analysis and Cataloging Methodology

2.2.1 Dataset Template and Qualitative Indicators

To support a systematic comparison of candidate datasets, a unified cataloging template was defined and applied to ten datasets (DS01–DS10). The template is explicitly designed around the co-presence of three families of attributes, corresponding to the three stages of the neuro-symbolic pipeline:

1. **Perception-oriented properties:** domain (Martian, lunar, terrestrial), data source (real vs. simulated), sensor modalities (RGB, depth, LiDAR, IMU, Global Positioning System (GPS)), scene complexity, number of images/frames, type and granularity of semantic annotations (pixel-, voxel-, instance-level), class taxonomy and its relation to navigation.
2. **Symbolic-oriented properties:** semantic richness, label consistency, suitability for symbolic reasoning, and ease of mapping to an $\mathcal{ALN}$ -based TBox/ABox.
3. **Simulation-oriented properties:** presence of 3D assets (meshes, point clouds), data formats (*e.g.*, ROS 2 bag, glTF, USD), feasibility of conversion to USD, simulability in Omniverse, and expected integration effort.

Each catalog entry includes: ID, dataset name, publication year, target platform, domain, modalities, presence of proprioception, scenario complexity, number of samples and labels, semantic classes, primary objectives, data format, simulability and integration effort in Omniverse, suitability for symbolic reasoning and KG construction, license, and URL/DOI. The fully filled-in catalog for DS01–DS10 is reported in Appendix A (see in particular Section A.1 and the per-dataset tables Table A.2–Table A.11). In the main text, only a condensed comparative view is provided.

2.2.2 Omniverse Simulability and Integration Effort

Because the proposed neuro-symbolic pipeline is ultimately evaluated in a USD-native simulator (OmniLRS on IsaacSim), an explicit notion of *simulability* is required. Three qualitative indicators are used.

Presence of 3D assets. Datasets are grouped according to the type of 3D information available:

- full textured meshes with semantic mapping (HM3D, Replica, HSSD-200) [18, 19, 20];
- dense point clouds and depth maps suitable for meshing (LuSNAR, RELIS-3D) [4, 26];
- only 2D images and masks (AI4Mars, S5Mars) [35, 9].

Conversion to USD. For each dataset, we assess whether existing toolchains (*e.g.*, glTF $\rightarrow$ USD importers, ROS 2 bag $\rightarrow$ USD, UE4 $\rightarrow$ USD exporters) can be used:

- *Yes* - when assets can be imported with limited effort (Replica, HSSD-200) [19, 20];
- *Partial* - when substantial pre-processing is required (LuSNAR, TartanAir, RELIS-3D) [4, 21, 26];
- *No* - when scenes would effectively have to be hand-modelled from scratch (AI4Mars, S5Mars).

Simulability and integration effort. Based on the above, two qualitative scores are assigned:

- *Simulability in Omniverse* (High/Medium/Low), reflecting the faithfulness and ease with which scenes can be recreated;
- *Integration effort* (High/Medium/Low), reflecting the engineering workload needed to obtain a working USD simulation aligned with the dataset.

These scores are deliberately project-specific: for example, indoor assets that are trivial to import into IsaacSim may still have low relevance if the thesis focuses on lunar terrains.

2.2.3 Suitability for Symbolic Reasoning and KB Construction

The third axis of the analysis concerns how easily each dataset can be plugged into the $\mathcal{ALN}$ -based KG supporting Tiny-ME. The following dimensions are considered:

- **Semantic richness and domain relevance:** number of classes, their coherence, and their alignment with planetary navigation and geology (*e.g.*, rocks, craters, regolith, ridges, rover body) [1, 2].

- **Structural information:** availability of 3D geometry, sensor poses, and temporal continuity to infer spatial relations (*e.g.*, *rock inside crater*, *rock near slope*, *hazard in front of rover*).
- **Label consistency:** annotation quality (expert vs. crowd-sourced), intra-dataset consistency, and compatibility with concepts used in ontology-guided segmentation and classification [25, 29, 7].
- **Alignment with $\mathcal{ALN}$ and Tiny-ME:** ability to encode classes and relations within $\mathcal{ALN}$ with tractable reasoning [5, 11]; suitability for non-standard inferences (CA, CC, CCov) to cope with noisy perception [24].

Each dataset is given qualitative ratings for *Suitability for Symbolic Reasoning* and *KG mappability*. These ratings are then used, together with simulability and perception-related metrics, to select a reference benchmark. The detailed ratings and textual justifications are provided in the per-dataset catalog entries in Appendix A, Table A.2–Table A.11.

2.3 Analyzed Datasets (DS01–DS10)

This section briefly summarizes the ten datasets evaluated. The focus is on properties that directly affect semantic segmentation, symbolic reasoning, and simulation. Detailed catalog sheets are reported in Appendix A, where each dataset has its own entry (Table A.2–Table A.11). For datasets that are not ultimately selected, only a concise analysis is provided, emphasising why they are not used as primary benchmarks in this thesis.

2.3.1 DS01 – SSCBench

SSCBench is a large-scale benchmark for 3D Semantic Scene Completion (SSC) in autonomous driving, aggregating KITTI-360, Waymo, and nuScenes into a unified voxel grid representation [28]. It provides voxel-wise semantic labels for urban scenes, targeting the reconstruction of occluded geometry and semantics from sparse observations. A detailed catalog entry for DS01, including all perception-, symbolic-, and simulation-oriented fields, is provided in Appendix A (Table A.2).

Relevance for this thesis. SSCBench is conceptually relevant for 3D semantic reasoning and as a model for voxel-based abstractions. However, its urban domain, lack of explicit meshes, and voxel-only format make it poorly aligned with space exploration and with USD-based simulation. Simulability is rated low and integration effort high; symbolic suitability is medium due to the absence of explicit objects. It is therefore used only as a reference for 3D completion strategies, not as a training or evaluation dataset.

2.3.2 DS02 – AI4Mars

AI4Mars provides pixel-wise semantic annotations for images from the Curiosity, Spirit, and Opportunity rovers, specifically targeting terrain-aware autonomous driving on Mars [35, 1]. It includes four main terrain classes (soil, sand, bedrock, big rock) plus a null class, with annotations obtained by crowd-sourcing and expert validation. The full catalog for DS02, including taxonomic and simulability details, is reported in Appendix A (Table A.3).

Relevance for this thesis. AI4Mars is a key 2D dataset for planetary terrain segmentation and offers conceptually clean classes that map naturally to ontological concepts. This makes it a useful reference for the semantic layer of the KG and for cross-checking semantic design choices. However, the absence of 3D geometry, proprioception, and native simulation assets limits its role to 2D perception benchmarking and ontology inspiration rather than as the backbone of a full neuro-symbolic pipeline.

2.3.3 DS03 – S⁵Mars

S⁵Mars extends the Martian segmentation landscape with high-resolution Mastcam imagery and a semi-supervised learning framework for sparse annotations [9]. It defines nine classes (sky, ridge, soil, sand, bedrock, rock, rover, trace, hole), with approximately half of each image annotated. The associated catalog sheet, with full metadata for DS03, is given in Appendix A (Table A.4).

Relevance for this thesis. S⁵Mars offers a richer taxonomy than AI4Mars and has direct implications for ontology design and for handling class imbalance. It is particularly relevant as a methodological reference for semi-supervised learning under annotation scarcity. As with AI4Mars, the lack of 3D data and of a direct link to simulation confines its role to 2D segmentation and conceptual support, not to 3D neuro-symbolic navigation.

2.3.4 DS04 – HM3D

HM3D is a large-scale dataset of scanned indoor environments for embodied AI, providing textured meshes, panoramic RGB-D, and instance-level semantic labels [18]. Its scenes are delivered as glTF meshes with object annotations. The full HM3D catalog entry (DS04) is reported in Appendix A, Table A.5.

Relevance for this thesis. HM3D is exemplary for object-centric reasoning and for USD-compatible 3D assets. It shows how instance-rich meshes can be mapped into a knowledge graph and queried by a reasoning engine. However, its indoor, anthropogenic domain and lack of proprioception make it orthogonal to lunar surface navigation. It is used here as an existence proof of USD/KG integration, not as a data source for training.

2.3.5 DS05 – TartanAir

TartanAir is a synthetic dataset generated in UE4 and AirSim, with multi-modal data (stereo RGB, depth, LiDAR, optical flow, poses, and IMU in newer releases) across diversified indoor, outdoor, and sci-fi scenes [21]. The detailed catalog fields for DS05 are collected in Appendix A, Table A.6.

Relevance for this thesis. TartanAir demonstrates how synthetic environments can provide multi-modal data with domain randomisation for robust perception and SLAM. Its design philosophy directly informs LuSNAR and the simulation strategy in this thesis. However, TartanAir does not include lunar-specific terrains nor a ready-made bridge to USD, and thus acts primarily as a methodological reference rather than a direct training dataset.

2.3.6 DS06 – RELIS-3D

RELIS-3D is a real off-road dataset for ground vehicles, including RGB, LiDAR, IMU, GPS, and semantic labels for both images and point clouds [26]. It is stored in ROS bag format and targets traversal in complex, unstructured terrestrial environments. The catalog entry for DS06, including its semantic taxonomy and ROS-based format, is given in Appendix A (Table A.7).

Relevance for this thesis. RELIS-3D is conceptually close to planetary rover navigation: unstructured terrain, rich multi-modal sensing, and semantics useful for traversability. It is a valuable template for the design of lunar datasets and for symbolic reasoning about off-road scenes. The domain mismatch (Earth vegetation, buildings) and the effort required to reconstruct USD scenes limit its direct use. It is therefore used mainly as a reference for evaluating multi-sensor design and annotation strategies.

2.3.7 DS07 – M2DGR

M2DGR is a multi-modal dataset for ground robotics, combining fisheye RGB, infrared, event cameras, LiDAR, IMU, and GPS-RTK, primarily aimed at SLAM and sensor fusion [27]. Its full catalog description as DS07 is provided in Appendix A, Table A.8.

Relevance for this thesis. M2DGR is excellent for studying localisation and multi-sensor fusion, but it does not provide semantic labels. As a result, it cannot directly support semantic segmentation or KG population without an additional annotation effort. For the purposes of this work, it is excluded as a primary candidate for the neuro-symbolic pipeline.

2.3.8 DS08 – Replica

Replica offers photorealistic reconstructions of indoor environments, with meshes, textures, and instance-level semantics [19]. It is distributed as glTF scenes that can

be readily imported into Omniverse and converted to USD. The detailed Replica catalog sheet (DS08) is provided in Appendix A, Table A.9.

Relevance for this thesis. Replica is highly relevant to the USD/KG story: it shows that semantic meshes and knowledge graphs can coexist and support object-centric navigation and reasoning [17, 17]. As with HM3D, its domain (indoor) is not aligned with lunar exploration. It informs the design of the USD/KG bridge but is not used as a perception dataset.

2.3.9 DS09 – HSSD-200

HSSD-200 is a synthetic dataset of 211 indoor scenes, with 18 656 unique models and 466 object categories, designed for embodied AI and ObjectNav tasks [20]. It provides glTF assets with rich semantic annotations and is easily convertible to USD. The corresponding catalog entry for DS09 is reported in Appendix A, Table A.10.

Relevance for this thesis. HSSD-200 is effectively a large-scale, fully synthetic counterpart of Replica, illustrating the benefits of artist-designed scenes and structured semantics. Its role here is analogous: a reference for large-scale USD-native, semantically annotated digital twins. Domain mismatch prevents its direct use in the lunar pipeline.

2.3.10 DS10 – LuSNAR (Overview)

LuSNAR is a synthetic multi-modal dataset specifically designed for lunar exploration tasks. It contains nine simulation scenes built in UE4 with AirSim, spanning different combinations of topographic relief and object density [4]. For each scene, LuSNAR provides stereo RGB images, dense depth maps, LiDAR point clouds, IMU data, rover ground-truth poses, pixel-wise semantic masks for images, and point-wise semantic labels for LiDAR. The complete catalog entry for DS10, including all fields of the template, is listed in Appendix A, Table A.11.

Supported tasks include 2D and 3D semantic segmentation, visual and LiDAR SLAM, stereo matching, and 3D reconstruction [4]. The image labels include five classes: lunar regolith, impact crater, rock, mountain, and sky; LiDAR labels distinguish at least regolith, rocks, and craters.

Relevance for this thesis. LuSNAR simultaneously satisfies:

- *domain alignment*: realistic lunar-like terrains informed by space mission data;
- *multi-modality*: joint RGB, depth, LiDAR, IMU, and pose;
- *simulation compatibility*: scenes authored in a game engine (UE4) conceptually close to Omniverse, facilitating replication in USD;
- *symbolic mappability*: semantically clean classes aligned with the concepts required for traversability and scientific reasoning.

For these reasons, LuSNAR is chosen as the primary benchmark for this thesis and is analyzed in more detail in Section 2.5, with the catalog sheet in Appendix A (Table A.11) acting as a structured reference.

2.3.11 Cross-Dataset Summary

Table 2.1 summarises the ten datasets along the main dimensions relevant to this work. The detailed definitions of each field and the full per-dataset compilations can be found in Appendix A, in particular in the template of Section A.1 and the entries of Section A.2.

ID	Dataset	Domain	Source	Semantics	Proprio.	OmniSim	Symb. Suit.
DS01	SSCBench	Urban driving	Mixed	3D voxel SSC	No	Low	Medium
DS02	AI4Mars	Martian terrain	Real	2D pixel-wise	No	Medium	Medium-High
DS03	S ⁵ Mars	Martian terrain	Real	2D pixel-wise	No	Medium	High
DS04	HM3D	Indoor	Real	3D instance	No	Medium	High
DS05	TartanAir	Mixed scenes	Sim.	2D/3D (subset)	Yes	Medium	Medium
DS06	RELLIS-3D	Off-road	Real	2D/3D	Yes	Low	High
DS07	M2DGR	Indoor/Outdoor	Real	None	Yes	Low	Low
DS08	Replica	Indoor	Real	3D instance	No	High	High
DS09	HSSD-200	Indoor	Sim.	3D instance	No	High	High
DS10	LuSNAR	Lunar terrain	Sim.	2D/3D	Yes	Medium	High

Table 2.1: Summary of the ten catalogd datasets (see Appendix A for the full catalog).

2.4 Selection of the Reference Benchmark

2.4.1 Comparative Evaluation of Martian, Terrestrial, and Lunar Datasets

From the catalog, three families emerge:

- **Planetary datasets** (AI4Mars, S⁵Mars, LuSNAR): domain-aligned with rover exploration, but with different levels of multi-modality and simulability [35, 9, 4].
- **Terrestrial robotics datasets** (RELLIS-3D, M2DGR, TartanAir): rich multi-sensor data, but with Earth-centric semantics and visual appearance [26, 27, 21].
- **Indoor 3D datasets** (HM3D, Replica, HSSD-200, SSCBench): excellent for object-centric reasoning and USD/KG integration, but not terrain-oriented [18, 19, 20, 28].

Planetary datasets are closest to the intended application. AI4Mars and S⁵Mars, in particular, offer real Martian imagery and high-quality 2D annotations, and have influenced the design of the semantic classes and the ontology [35, 9]. However,

they lack 3D geometry, proprioception, and direct links to simulation, making them unsuitable as the main backbone for a 3D neuro-symbolic navigation pipeline.

Terrestrial robotics datasets offer the opposite trade-off: rich multi-modal data and realistic dynamics, but domains (vegetation, buildings, traffic) that do not match lunar geology or optics. They serve as methodological references for multi-sensor acquisition and evaluation, but not as primary sources for semantic training.

Indoor 3D datasets are invaluable to understand how USD, glTF, and knowledge graphs interact in practice [17, 17]. Nevertheless, their semantic focus on man-made objects and room layout is orthogonal to lunar surface reasoning.

2.4.2 Rationale for Selecting LuSNAR

LuSNAR is selected as the reference dataset and benchmark because it simultaneously satisfies the requirements outlined in Section 2.1 and fits the cataloging template introduced in Section 2.2, with its detailed attributes recorded in Appendix A (Table A.11).

(1) Domain and semantics. LuSNAR is explicitly designed for lunar exploration and reproduces realistic lunar terrains, informed by space mission observations [4]. Its class taxonomy—regolith, rock, crater, mountain, sky—matches the concepts required for traversability reasoning and for scientific interest detection in the SPEX ontology (introduced in Chapter 3). The taxonomy is compact enough to be handled efficiently on edge devices, yet rich enough to drive meaningful symbolic abstractions.

(2) Multi-task, multi-modal structure. Unlike most previous lunar datasets, LuSNAR simultaneously supports semantic segmentation, navigation, and reconstruction tasks. It provides high-resolution stereo pairs, semantic labels, dense depth maps, LiDAR point clouds, IMU data, and rover poses across nine scenes [4]. This enables a coherent chain from raw sensor data to symbolic reasoning, using a single dataset for:

- training and evaluation of 2D semantic segmentation models using RGB + masks;
- 3D semantic segmentation and scene completion using labelled point clouds;
- SLAM and sensor-fusion experiments (vision-only, LiDAR, or visual-inertial);
- dense 3D reconstruction benchmarks based on depth fusion.

(3) Simulation and USD compatibility. Although LuSNAR does not ship USD scenes, its environments are authored in UE4 with AirSim, similar in spirit to other game-engine-based pipelines such as TartanAir [21, 4]. This means that equivalent or analogous lunar scenes can be constructed in IsaacSim or OmniLRS, using:

- terrain meshes derived from depth maps or static point clouds;
- rock and crater assets positioned to match or approximate LuSNAR geometries;
- camera and LiDAR rigs configured to mimic LuSNAR sensors.

LuSNAR thus acts as the semantic and geometric reference for constructing a USD-based lunar digital twin used in simulation.

(4) Symbolic mappability and KG integration. LuSNAR’s compact taxonomy eases ontology design and ABox population:

- each semantic class is mapped to a concept in the $\mathcal{ALN}$ TBox (*e.g.*, **Regolith**, **Rock**, **Crater**, **Mountain**, **Sky**);
- objects extracted from segmentation and point clouds become individuals with attributes (area, slope, roughness, distance to rover);
- spatial relations such as *inside*, *adjacent-to*, *on-slope-of* can be computed from 3D geometry and encoded as roles.

This structure can be reflected in USD by attaching semantic and relational attributes to prims and subsequently translating the USD stage to a knowledge graph in the style of [17, 17]. Tiny-ME can then perform non-standard reasoning over this KG to support context-aware navigation decisions [11, 24].

(5) Established baselines and metrics. The LuSNAR paper reports baselines for several tasks (2D/3D segmentation, SLAM, stereo matching, 3D reconstruction) using standard metrics such as Mean Intersection over Union (mIoU), Mean Accuracy (mAcc), and Chamfer distance [4]. In this thesis these results are not reproduced in detail, but they provide a reference for the level of perceptual accuracy that the neuro-symbolic pipeline should aim at. They also fix the evaluation metrics that will be reused in later chapters.

AI4Mars and S5Mars remain relevant as auxiliary resources: they inform alternative class taxonomies and can be used for cross-dataset validation of 2D segmentation models or for future domain adaptation from lunar to Martian terrains [35, 9]. However, LuSNAR is the only dataset in the catalog that simultaneously satisfies the requirements of domain alignment, multi-modality, simulation compatibility, and symbolic mappability.

2.5 Detailed Characterisation of the LuSNAR Dataset

This section refines the analysis of LuSNAR, focusing on those aspects that are critical for the neuro-symbolic pipeline: dataset organisation, semantic classes,

multi-sensor structure, and the type of neuro-symbolic metadata that can be extracted and mapped into TBox and ABox assertions. The goal is not to restate the original paper, but to highlight the features that enable the integration of LuSNAR with Tiny-ME and USD-based simulation. For a compact, field-by-field description of LuSNAR, the reader is referred to its catalog entry in Appendix A (Table A.11).

2.5.1 Dataset Structure and Multi-Sensor Organisation

LuSNAR is organized into nine lunar scenes (`Moon_1` to `Moon_9`), each providing two complementary subsets [4]:

1. a *multi-sensor navigation* subset, consisting of time-synchronized rover traversals with stereo RGB, depth, LiDAR, IMU, and ground-truth poses;
2. a *static* subset (Digital Elevation Model (DEM)-like), consisting of a small number of high-quality views and point clouds representing the static scene geometry.

For each scene, the navigation subset exposes a consistent structure:

- left camera (`image0/`): rectified RGB images, dense depth maps and pixel-wise semantic masks, aligned via timestamp files;
- right camera (`image1/`): rectified RGB images for stereo, with timestamps;
- `gt.txt`: rover states at discrete timestamps (position, orientation, velocity, IMU biases);
- `imu.txt`: raw IMU data at higher frequency;
- LiDAR point clouds: sequences of labelled 3D points aligned with images and poses.

The static subset for each scene, stored under `DEM/Moon_i/mav0/`, includes a small set of RGB views, corresponding semantic masks and depth maps, and point clouds with and without semantic labels. These data are particularly useful for reconstructing approximate meshes and validating 3D reasoning components.

This organisation supports:

- frame-based or sequence-based 2D segmentation on left-camera RGB frames;
- 3D semantic reconstruction by fusing depth maps and point clouds;
- visual, LiDAR, and visual-inertial SLAM using ground-truth poses and IMU as reference.

2.5.2 Semantic Taxonomy and Label Encodings

LuSNAR defines a compact taxonomy of five semantic classes for image labels [4]:

1. **Lunar regolith**: fine-grained, largely homogeneous surface material forming the primary traversable substrate.
2. **Impact crater**: bowl-shaped depressions with rim discontinuities, typically associated with non-traversable slopes.
3. **Rock**: discrete solid obstacles of varying scale, relevant for collision risk assessment and scientific sampling.
4. **Mountain**: elevated macroscopic terrain features with steep gradients that strongly constrain feasible paths.
5. **Sky**: background pixels above the horizon, used to stabilise segmentation and geometry estimation.

In 2D masks, these classes are colour-encoded; in point clouds, they appear as discrete integer labels. The exact mapping is provided in the dataset specification and is used throughout this thesis to ensure consistent correspondence between image and point-cloud domains.

From the viewpoint of the SPEx ontology, the compact taxonomy of LuSNAR is a key advantage. Its classes align directly with the structural elements that matter for autonomous navigation and scientific interpretation: macroscopic terrain forms such as mountains and craters, discrete solid objects such as rocks, and the underlying regolith as the reference substrate for surface analysis. The sky label, while included for segmentation completeness, is treated as non-physical background and is excluded from navigation-oriented reasoning.

2.5.3 Neuro-Symbolic Metadata Derived from LuSNAR

The structure and semantics of the LuSNAR dataset make it particularly suitable for the generation of *neuro-symbolic metadata*, that is, ABox assertions derived from neural predictions and coherently aligned with the SPEx TBox. Each segmented frame provides a compact and navigationally meaningful description of the scene, where a small number of classes—**rock**, **impact crater**, and **mountain** in particular—carry both operational and scientific significance.

These classes embody the essential elements required for autonomous navigation and surface analysis: mountains and craters define macroscopic geomorphological constraints that influence traversability and visual planning, while rocks represent discrete objects relevant to obstacle avoidance and pickup operations. Their simultaneous scientific value—as indicators of geological processes and sampling opportunities—makes LuSNAR uniquely appropriate for experimentation in lunar robotic exploration.

The controlled size and navigational relevance of this taxonomy offer clear advantages for a neuro-symbolic framework. A limited number of semantically dis-

tinct classes reduces perceptual ambiguity and supports consistent mappings between pixel-level labels and $\mathcal{ALN}$ -based symbolic concepts. This ensures tractable reasoning on embedded platforms, where computational resources are constrained, and allows Tiny-ME to process semantic information in real time without compromising logical completeness or consistency.

Within this context, LuSNAR serves as a bridge between perception, knowledge representation, and simulation. The dataset provides a foundation for constructing scene-level ABoxes that integrate geometric and semantic cues: each segmented element—rock, crater, or mountain—is abstracted into a logical individual with qualitative attributes such as slope, proximity, or roughness, encoded as data properties and interpreted through the SPEx ontology. The resulting neuro-symbolic metadata form a consistent interface between:

- the segmentation backbone, responsible for perceptual grounding;
- the ABox builder, which produces the formal $\mathcal{ALN}$ assertions;
- and the Tiny-ME reasoner, which performs both standard and non-standard inference (*e.g.*, CA and CC) to derive context-aware, explainable navigation decisions [24, 11].

By combining a scientifically motivated taxonomy with lightweight symbolic modelling, LuSNAR thus enables the generation of neuro-symbolic representations that are both interpretable and operationally relevant. As discussed in Chapter 3, this alignment between perception and knowledge structure underpins the design of the SPEx ontology and its application to real-time reasoning in lunar exploration scenarios.

2.6 OmniLRS and NVIDIA Isaac Sim as Simulation Environments

2.6.1 USD-based Lunar Digital Twin and ROS 2 Integration

OmniLRS (Omniverse Lunar Robotics Simulator) is an open-source simulator for photorealistic robotic simulations in lunar environments, built on Omniverse Isaac-Sim and USD [14]. It provides procedural environment generation, multi-robot capabilities, and a synthetic data pipeline for machine-learning applications, together with ROS 1 and ROS 2 bindings to control robots and environments.

From a data perspective, OmniLRS offers:

- a USD-native scene graph where each prim represents terrain patches, rocks, craters, or rover components;
- per-prim semantic tags that can be mapped to ontological classes (*e.g.*, `SpEx:Rock`);
- virtual sensors configured to approximate LuSNAR sensor parameters (resolution, field-of-view, baseline, LiDAR pattern).

This makes it possible to:

1. reconstruct in USD a set of lunar scenes consistent with LuSNAR, either by importing meshes derived from LuSNAR depth/point clouds or by authoring analogous terrains guided by LuSNAR statistics;
2. stream RGB, depth, and LiDAR data from the simulator into the same perception and reasoning pipeline used for LuSNAR offline data;
3. bind USD prims to ABox individuals, effectively treating the Omniverse scene as a live semantic digital twin.

2.7 Summary and Role in the Overall Architecture

Chapter 2 has presented a structured analysis of datasets and simulation environments relevant to neuro-symbolic autonomous navigation in planetary scenarios. Starting from requirements on semantic coverage, multi-modality, symbolic compatibility, and USD-based simulability, a cataloging template was defined and applied to ten datasets spanning planetary, terrestrial, and indoor domains, with the full catalog documented in Appendix A.

The comparative evaluation highlights LuSNAR as uniquely suited to the goals of this thesis: it combines a lunar domain, multi-modal sensing, established benchmarks, and a principled pathway to USD-native simulation and $\mathcal{ALN}$ -based knowledge representation. Its organisation and semantics, as summarized in Appendix A, allow the derivation of neuro-symbolic metadata—ABox assertions about regolith patches, rocks, craters, and mountains—that are consistent with the SPEx TBox and can be processed by Tiny-ME.

OmniLRS and Omniverse IsaacSim [14] complement LuSNAR by providing a photorealistic, ROS-integrated lunar digital twin, where semantic annotations can be aligned with the SPEx ontology and exploited by Tiny-ME. On this foundation, the next **Chapter 3** will describe the overall neuro-symbolic architecture: the structure of the SPEx ontology, the integration of SegFormer-based segmentation models with Tiny-ME, and the data flows that connect LuSNAR-derived perception, USD-based simulation, and symbolic reasoning for autonomous navigation.

Chapter 3

System Architecture and Design

This chapter presents the architecture and design of the **neuro-symbolic navigation system** developed in this thesis. The goal is to unify **perception, knowledge representation, reasoning, and semantic 3D visualisation** into a coherent pipeline capable of operating on **LuSNAR** lunar scenes and, by extension, on simulated or real rover platforms via a USD-based digital twin.

The architecture is deliberately *middleware-agnostic*. **ROS 2** and Isaac Sim are used as convenient orchestration and inspection tools on top of a core design that can run entirely in batch mode via scripts, with semantic USD scenes and ProtocOWL ABoxes as the primary machine-readable artifacts.

The chapter is organized as follows. **Section 3.1** introduces the key design objectives. **Section 3.2** presents a high-level overview of the architecture. **Section 3.3** and **Section 3.4** describe the perception and symbolic layers. **Section 3.5** explains the end-to-end operation. **Section 3.6** summarizes the optional integration with ROS 2 and USD/Isaac Sim. **Section 3.7** concludes the chapter.

3.1 Design Objectives and Constraints

The architecture was designed under a set of explicit scientific objectives and practical constraints.

The system must integrate deep perception, symbolic knowledge, and a semantic USD workflow within a rock-centric navigation scenario. It must represent rocks, craters, mountains, and lunar surface regions as explicit objects, associate them with qualitative features, and reason over task-oriented patterns such as rock pickup or obstacle avoidance. Explanations of compatibility or mismatch must be available through concept abduction and concept contraction, and the resulting knowledge must be embedded and validated inside per-frame USD assets.

The architecture must satisfy the following practical requirements:

- operate efficiently on a single workstation with mid-range consumer hardware, without requiring dedicated servers or distributed computation;
- process the LuSNAR dataset directly, without custom preprocessing stages;

- preserve the logical fragment required by Tiny-ME ($\mathcal{ALN}(\mathcal{D})$) and support both offline execution and ROS 2 replay;
- keep numeric thresholds and feature-extraction logic outside the TBox, ensuring logical safety and stable runtime behaviour;
- rely on ROS 2 only as a lightweight transport and timing layer when live streaming or replay is required, with the core pipeline executable as Python or C/C++ scripts;
- construct validated semantic USD scenes in which ProtocOWL ABoxes and reasoning outputs are embedded as attributes and checked against their source files via hash-based validation (e.g., Secure Hash Algorithm 256-bit (SHA-256)).

These constraints naturally lead to a modular design comprising perception, ABox construction, symbolic reasoning over a controlled TBox, and USD-based generation and validation of semantic digital twins.

3.2 High-Level System Overview

Figure 3.1 summarizes the proposed **neuro-symbolic pipeline** as a set of loosely coupled blocks. On the left, the **LuSNAR RGB+Depth+LiDAR+IMU** block represents the raw multi-sensor data streams for each frame. These data feed two numeric components:

- a **Perception** block implementing SegFormer-based semantic segmentation on RGB images, producing per-pixel terrain labels and confidence estimates;
- a **3D Reconstruction** block that converts depth and LiDAR measurements into point clouds and meshes.

Together, these modules provide 2D semantic masks and 3D geometry for the current frame. At the centre, an **ABox Builder** block consumes SegFormer masks and geometric cues (e.g. object size, distance, local slope) and instantiates object-centric **Scene ABoxes** for rocks, craters, surface patches, and rover state. These ABoxes are consistent with a dedicated **SPEx TBox** shown at the top-right of Figure 3.1. A second symbolic block, **SPEx ABox Goals**, contains static ABoxes encoding rover profiles and task-specific goal requests (e.g. rock pickup or obstacle avoidance patterns).

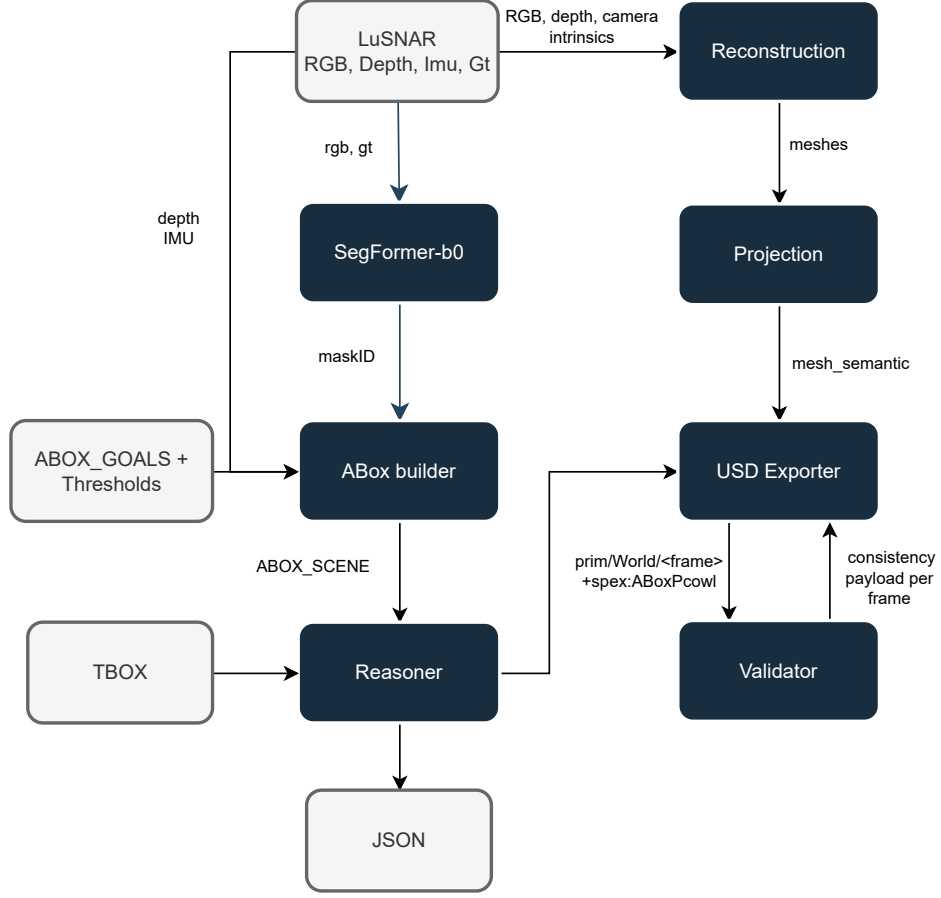


Figure 3.1: Block-level architecture of the proposed neuro-symbolic pipeline. LuSNAR sensing feeds perception and 3D reconstruction; an ABox builder connects these numeric components to the SPEX KG (TBox and goal ABoxes) and to Tiny-ME reasoning. Reasoning outputs are exported to a semantic USD scene, validated by a hash-based checker, and exposed to a navigation interface.

The right side of the diagram highlights the **Tiny-ME Reasoning** block. Tiny-ME takes as input the SPEX TBox, the goal ABoxes, and the per-frame Scene ABoxes, and performs non-standard reasoning (concept abduction and contraction) to rank candidate rocks or patches and to compute symbolic explanations of compatibility and mismatch. These decision-support outputs are forwarded to a **Navigation Interface** block that exposes ranked candidates, scores, and explanations to any downstream planner or control stack, without constraining the choice of navigation algorithm.

In parallel, the lower path shows the **Semantic USD Scene** block, which receives the reconstructed meshes together with symbolic labels and scores produced by the reasoning layer. Each frame is exported as a USD scene where geometry, semantics, and decision-support attributes are co-located. A dedicated

USD Hash Validator block re-opens the exported scenes, extracts embedded ProtocOWL payloads, compares them against the corresponding external `.pcowl` files, and performs hash checks (e.g. using SHA-256). Only frames that pass these checks are considered validated semantic USD frames.

Overall, the diagram stresses three aspects of the design: (i) a clear separation between numeric perception/geometry and symbolic knowledge; (ii) a central ABox builder that bridges these domains using the SPEx TBox; and (iii) an explicit USD path in which reasoning results are embedded, validated, and made available both for 3D inspection and for navigation.

3.3 Perception Subsystem

The **perception subsystem** extracts semantic and geometric information from LuSNAR frames and provides the quantitative input for the symbolic layer.

3.3.1 LuSNAR streams

For each scene, the system ingests **RGB images**, aligned **depth maps**, **LiDAR** scans, **IMU** logs, and ground-truth rover poses. In batch mode, these are read directly from the dataset folders. In ROS 2 mode, the same information can be published as topics and replayed, but this is orthogonal to the core processing.

3.3.2 SegFormer semantic segmentation

A **SegFormer** model (B0 or B1) fine-tuned on LuSNAR provides per-frame semantic masks for regolith, rocks, craters, mountains, and sky, together with optional confidence and entropy maps. Training follows the setup described in Chapter 5 (fixed-resolution protocol, balanced loss, class-frequency weighting). At inference, masks are exported in the layout expected by the ABox builder (integer label maps with consistent class indices and optional probability/confidence tensors).

3.3.3 3D reconstruction and semantic projection

Depth maps are back-projected into **point clouds**, meshed with Poisson or Ball Pivoting techniques, and associated with vertex- and face-level semantic labels via projection into the image plane. Confidence and entropy are aggregated at face level to support later qualitative assessment. The result is a **semantic mesh** that can be exported to USD and queried by downstream modules. The perception subsystem thus provides both 2D masks (for ABox construction) and 3D geometry (for semantic USD generation).

3.4 Knowledge and Reasoning Subsystem

The **symbolic component** is structured around a task-focused **TBox** specifically designed to validate the rock-centric neuro-symbolic pipeline on LuSNAR and to support the construction of semantic USD scenes. Its scope is intentionally minimal: it contains only the classes, qualitative bins, and relations required to describe rock morphology, rover state, and goal-capability patterns, avoiding any superfluous expressivity. This tight control over the schema ensures full compatibility with Tiny-ME’s $\mathcal{ALN}(\mathcal{D})$ fragment and guarantees predictable reasoning behaviour.

3.4.1 Design of the TBox

The TBox follows a clean, multi-layered structure. The core class hierarchy used in this work is reported in Figure 3.2, together with the data-property and object-property hierarchies that support quantitative attributes and relational structure.

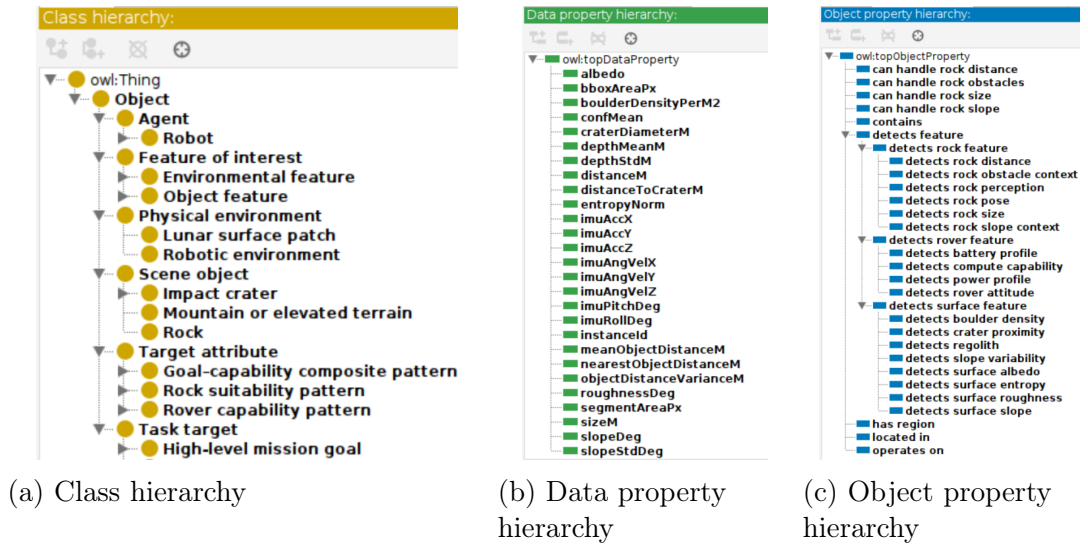


Figure 3.2: Core structure of the SpEx knowledge base.

The class hierarchy shown in Figure 3.2a is organized into three conceptual layers.

1. **Environmental and scene structure.** A scene is represented as a `RoboticEnvironment` containing one or more `:LunarSurface` patches and a set of `:SceneObject` instances located within them. IEEE 1872 and CORA classes such as `Robot`, `:RoboticEnvironment`, `PhysicalEnvironment`, `sumo:Object`, and `Agent` are reused to keep the model aligned with robotics standards, while the domain-specific part of the hierarchy is kept intentionally shallow.
2. **Object taxonomy for LuSNAR.** Rocks, mountains, and impact craters extend `:SceneObject` through a small set of domain-specific subclasses. This

reflects the coarse taxonomy of LuSNAR and avoids unnecessary fragmentation. These classes provide the bridge from **pixel-wise labels** to the object-centric individuals used in reasoning and in semantic USD scenes.

3. **Task, capability, and suitability patterns.** Under `:TargetAttribute` and `:TaskTarget`, the TBox introduces concepts for rover capability profiles, rock suitability patterns, and goal–capability composite patterns. These classes encode the qualitative structure required to model pickup and obstacle-avoidance tasks without naming any specific rock or rover instance.

The property hierarchies in Figure 3.2b and Figure 3.2c complete this view. **Data properties** attach numeric or symbolic attributes to individuals (e.g., rock distance, size, slope, roughness, entropy, IMU attitude, battery state). **Object properties** capture relational context such as spatial containment, feature detection, and association to local regions. Numeric quantities (e.g., slope, distance, variance) are computed outside the ontology; only their **qualitative interpretations** (e.g., `:Near`, `:RockInLowSlopeContext`) are incorporated into the schema as class or datatype constraints.

Figure 3.3 refines this core structure, focusing on how rock suitability and goal–capability patterns specialize the mid-level hierarchy, and how they are linked to rover-specific capability profiles.

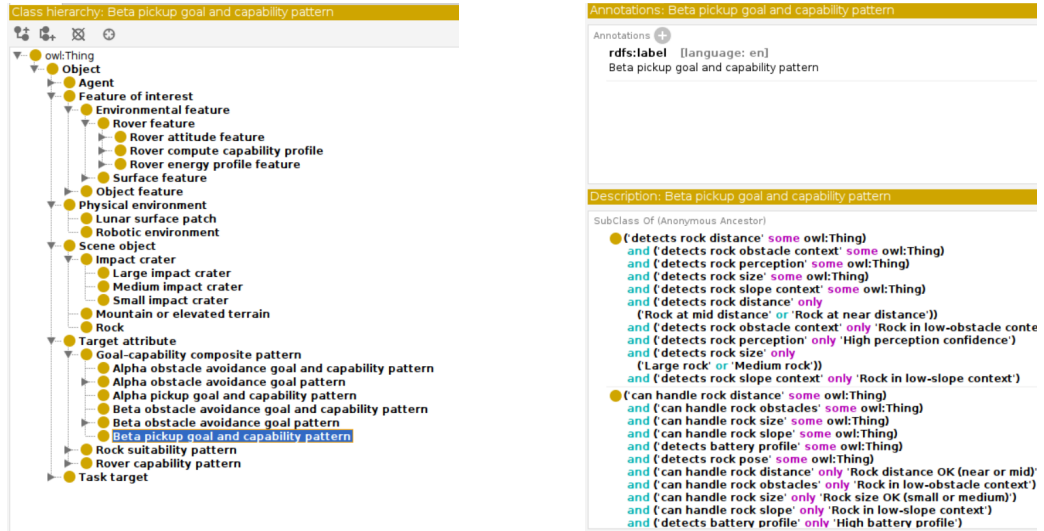


Figure 3.3: Specialization of the core hierarchy.

The left panel of Figure 3.3 shows how the `:RockSuitabilityPattern` and `:RoverCapabilityPattern` branches encode reusable templates for terrain regions and rover configurations. The right panel introduces composite classes such as `:AlphaObstacleAvoidanceGoalAndCapabilityPattern` and `:BetaPickupGoalAndCapabilityPattern`. These classes do not describe individual rocks, but rather **patterns** that must be matched by candidate rocks given a specific rover profile.

The internal logical structure of such a pattern is illustrated in Figure 3.4 for the case of the `:BetaPickupGoalAndCapabilityPattern` class.

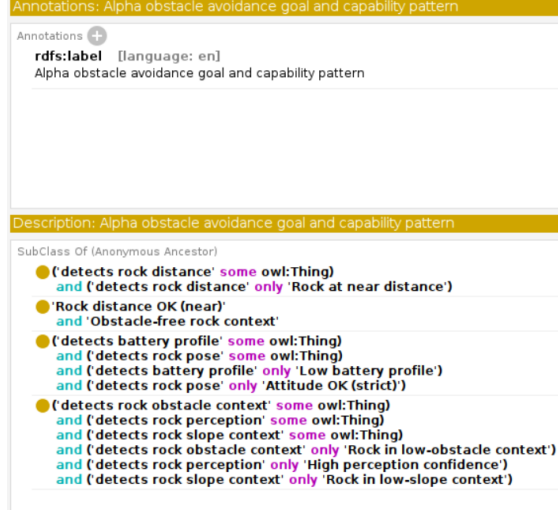


Figure 3.4: Logical structure of the `:BetaPickupGoalAndCapabilityPattern` class.

As shown in Figure 3.4, the pattern is a subclass of both a `:BetaRoverRock-CapabilityProfile` and a `:HeavyPickupCandidateRock`, thereby tying it to the Beta rover and to a specific suitability profile. Its anonymous superclass is a conjunction of two main components:

- a **detection branch**, describing what the rover must be able to sense: rock distance, obstacle context, rock size, slope context, and perception confidence, all constrained to appropriate qualitative bins (e.g. mid or near distance, low-obstacle context, high perception confidence, large or medium rocks in low-slope contexts);
- a **handling branch**, describing what the rover must be able to tolerate: acceptable rock distance and size, slope conditions, battery profile, and pose stability, again expressed through qualitative bins (e.g. distance and size categories that Beta can handle, low-obstacle and low-slope contexts, attitude constraints).

This structure is representative of all goal–capability composite patterns in the SPEx TBox. It separates the *perception side* (what the rover must detect) from the *actuation side* (what the rover can safely handle), so that concept abduction and contraction can independently relax or satisfy conditions in each branch.

3.4.2 Rover profiles and goal requests

In this thesis, the TBox is complemented by a set of **static ABoxes**, i.e., ABoxes that are defined once and remain fixed throughout the execution of the pipeline.

This modelling choice reflects the simplified configuration adopted in the current version of the proposed framework: rover characteristics and mission goals are pre-specified, rather than being dynamically updated or inferred during runtime.

- **Rover profiles.** For each rover (`:rover-Alpha`, `:rover-Beta`), a static profile ABox encodes its abstract resource characteristics—power budget, battery capacity, compute capability, and nominal attitude state inferred from IMU measurements. These fixed profiles introduce controlled variations (e.g., Alpha being more energy-constrained and Beta more capable), which are essential for validating concept abduction under realistic but well-defined conditions.
- **Goal and capability requests.** A separate static ABox defines request individuals such as `:AlphaPickupGoalReq`, `:BetaPickupGoalReq`, `:AlphaObstacleAvoidanceReq`, and `:BetaObstacleAvoidanceReq`. Each request is typed as a conjunction of task and capability patterns defined in the TBox (e.g., a `:PickupCandidateRock` combined with an `:AlphaRoverRockCapabilityProfile`). During runtime, reasoning therefore aims to match these fixed goal patterns – such as the one in Figure 3.4 – against the concrete rock individuals reconstructed at each frame.

3.4.3 Dynamic ABox construction

The **ABox builder** instantiates the TBox at the scale of a single frame. It creates individuals for the scene patch, each rock, crater, and mountain, and assigns the appropriate qualitative bins based on numeric thresholds defined outside the ontology. Rover state is inserted frame-by-frame through attitude and resource degradation patterns. The dynamic ABox contains only **assertion axioms** and is merged with the TBox and static ABoxes before reasoning.

For each frame, the builder produces:

- an OWL 2 Functional Syntax ABox for inspection and debugging;
- a ProtocOWL-encoded payload, aligned with Tiny-ME and suitable for embedding as a USD attribute.

3.4.4 Tiny-ME and the libusdsem library

Tiny-ME performs concept abduction and contraction between goal request individuals and the set of rock or patch candidates. **libusdsem** provides the glue between OWL Functional Syntax, ProtocOWL binary representation, and Tiny-ME’s internal structures. It also exposes a minimal API for the USD exporter, allowing reasoning results (compatibility scores, selected candidates, explanation tags) to be projected into 3D scenes without passing through ROS 2.

3.5 End-to-End Data and Control Flow

The pipeline operates **frame-by-frame** over LuSNAR sequences and can be executed either as a pure batch process (scripts only) or as a ROS 2 graph with the same logical stages. In all cases, semantic USD frames are the final, validated artifacts.

3.5.1 Offline training

SegFormer is trained on the LuSNAR training split, following the setup described in Chapter Chapter 5. The best checkpoint is selected based on mIoU and is then used in the neuro-symbolic processing phase. Training is entirely offline and does not involve ROS 2 or USD.

3.5.2 Neuro-symbolic processing and USD export

For each frame, the core loop is:

1. **Segmentation.** SegFormer produces a semantic mask and optional confidence/entropy maps.
2. **ABox construction.** The ABox builder fuses mask, depth, IMU, and configuration thresholds into a dynamic ABox (OWL + ProtocOWL).
3. **Reasoning.** The `libusdsem` library merges the TBox, static ABoxes, and dynamic ABox; Tiny-ME performs concept abduction and contraction, ranking candidates with symbolic explanations.
4. **3D semantics.** Depth-based meshes are reconstructed, semantics projected, and statistics computed at face level.
5. **USD export.** A USD exporter writes a per-frame scene enriched with: (i) semantic meshes and primvars; (ii) rover pose and frame timestamp; (iii) ProtocOWL ABox payloads and file-path attributes; (iv) Tiny-ME scores and explanation tags as custom attributes.
6. **Validation.** A dedicated validator re-reads the exported USD scene, extracts the embedded ProtocOWL payload and the referenced `.pcowl` file, compares them byte-wise, and computes and checks SHA-256 hashes. Only frames passing these checks are considered *validated semantic USD frames*.

These steps define the **core data and control flow**. In ROS 2 mode, they are distributed across nodes connected by topics; in batch mode, they are function calls within the same process.

3.5.3 Decision-support interface

The system outputs ranked candidates for pickup or obstacle-avoidance tasks, together with symbolic mismatch explanations (missing capabilities, violated constraints, or relaxed requirements). These structures are independent of any planner and can be consumed by different **navigation stacks**, or used purely for offline analysis of LuSNAR scenes and their semantic USD counterparts.

3.6 ROS 2 and USD/Isaac Sim Integration

ROS 2 and **USD/Isaac Sim** are used as optional integration and inspection tools on top of the middleware-agnostic core.

3.6.1 Lightweight ROS 2 graph

When needed, the pipeline is exposed in Figure 3.5 as a small ROS 2 graph whose only role is to handle timing and message transport between otherwise standalone modules. The graph is intentionally simple and avoids embedding complex logic in the middleware itself.

A typical configuration, illustrated in Figure 3.5, includes:

- `segformer_node`, wrapping the trained SegFormer model and publishing semantic masks (and optionally confidence and entropy) from RGB images;
- `abox_builder`, subscribing to segmentation, depth, and IMU topics, and publishing dynamic ABoxes fragments and associated ProtocOWL payloads;
- `tinyme_reasoner`, hosting Tiny-ME via `libusdsem`, subscribing to scene ABoxes and publishing decision-support messages and explanation tags;
- `mesh_reconstructor` and `semantic_projector`, turning depth and semantics into meshes with face-level labels;
- `usd_exporter`, subscribing to geometry, ABox payloads, and reasoning outputs and writing per-frame USD scenes;
- `usd_validator`, re-reading exported scenes and producing validation reports (hash matches, path consistency).

All these nodes are thin wrappers around the same libraries used in batch mode. Replacing ROS 2 with another middleware would not affect the logical behavior of the pipeline.

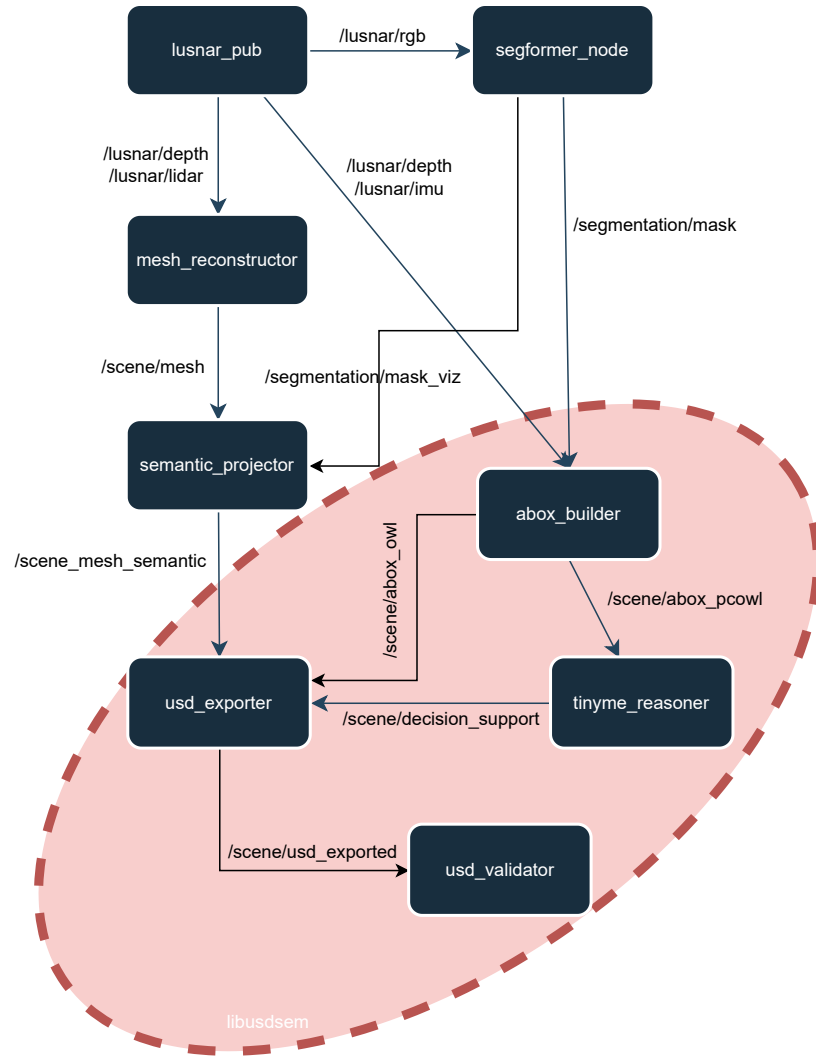


Figure 3.5: ROS 2 integration graph. Nodes wrap existing components: SegFormer inference, ABox builder, Tiny-ME reasoner, mesh reconstruction, USD export and validation.

3.6.2 USD stages and Isaac Sim

USD stages generated by the exporter include **semantic meshes**, face-level statistics, rover pose, and reasoning results encoded as custom attributes (e.g. booleans indicating goal satisfaction, compatibility scores, or explanatory tags). Protocol payloads and hash attributes make each frame self-contained and verifiable. Isaac Sim is used as an inspection tool to visualize these stages, replay LuSNAR sequences, and inspect how symbolic decisions correlate with 3D geometry. It is not part of the reasoning loop and does not affect Tiny-ME’s operation.

3.7 Summary

Chapter 3 has described the architecture of the proposed **neuro-symbolic** pipeline for LuSNAR. The pipeline integrates SegFormer-based perception, object-centric ABox construction, a minimal but expressive TBox tailored to rock-centric reasoning, Tiny-ME for non-standard matching, and USD for semantic 3D visualisation and validation. The TBox design is intentionally compact and hierarchical, with a core layout (Figure 3.2) refined into specialized suitability and goal-capability patterns (Figure 3.3 and Figure 3.4). The semantic USD workflow ensures that symbolic knowledge and 3D geometry remain tightly coupled and verifiable.

The architecture is deliberately independent of any specific middleware. ROS 2 is used only as a thin orchestration layer and can be replaced or removed without altering the logical behaviour. USD and Isaac Sim provide a convenient 3D inspection and digital-twin environment but remain outside the core reasoning loop.

The next **Chapter 4** details dataset preprocessing, SegFormer training and evaluation, ABox builder design, the `libusdsem` bridge to Tiny-ME, and the construction and validation of semantic USD scenes.

Chapter 4

Implementation of the Neuro–Symbolic Pipeline

Chapter 4 details the concrete implementation of the **neuro–symbolic pipeline** introduced in Chapter 2 and Chapter 3. The focus is on the **software stack**, the SegFormer-based perception module, the `libusdsem` library bridging USD and the SPEx KG, and a ROS 2¹ deployment architecture that mirrors the core batch pipeline as a modular graph of nodes.

The entire prototype runs on a single mobile workstation under **Ubuntu 22.04.5 LTS**, used for training, offline sequence processing, symbolic reasoning, and semantic 3D visualization. The core experiments are performed in batch mode; the ROS 2 graph is developed as a deployment-oriented architecture and partially prototyped to validate interfaces and data flow.

The chapter is organized as follows. **Section 4.1** describes the development environment and core dependencies. **Section 4.2** outlines LuSNAR dataset preparation and preprocessing. **Section 4.3** discusses the SegFormer-based semantic segmentation module. The **Section 4.4** library presents `libusdsem` and its interaction with Tiny-ME and USD. **Section 4.5** introduces the ROS 2 architecture, nodes, and message interfaces, as a deployment pathway consistent with the middleware-agnostic design. Experimental results and quantitative performance analyses are deferred to **Chapter 5**.

4.1 Development Environment and Core Dependencies

4.1.1 Hardware Platform

All software components are developed and executed on an ASUS TUF F15 laptop running Ubuntu 22.04.5 LTS (Jammy), equipped with an Intel Core i7-12700H CPU, 16 GB RAM, and a NVIDIA GeForce RTX 4070 Laptop GPU. SegFormer

¹<https://www.ros.org/>

training and inference, Tiny-ME reasoning, ROS 2 communication, and USD-based inspection all run on this single workstation. The GPU is used primarily for deep learning workloads and Isaac Sim rendering, while CPU resources handle dataset preprocessing, ROS 2 nodes, and symbolic reasoning.

4.1.2 System Environment and Reproducibility

The software environment is structured around **Python virtual environments** and standard package managers:

- a Python stack (Python 3.x, `pip`, `venv` or `virtualenv`) for dataset handling, training, and evaluation;
- a deep learning stack (**PyTorch**² with CUDA support, TorchVision³, HuggingFace Transformers⁴) for SegFormer;
- scientific and utility libraries (Numerical Python (NumPy)⁵, Scientific Python (SciPy)⁶, Panel Data and Python Data Analysis (Pandas)⁷, Open Source Computer Vision Library (OpenCV)⁸, Matlab + Plotting Library (Matplotlib)⁹, Open3D¹⁰);
- the Tiny-ME C library and Python bindings;
- the USD SDK and C/C++ bindings used by `libusdsem`;
- a ROS 2 distribution with `colcon` for building the robotics stack.

All library versions are pinned in configuration files (`requirements.txt`). Shell scripts record the active Python environment, Git commit, and hyperparameters used for each experiment, so that runs can be repeated on the same software configuration.

4.1.3 USD, Isaac Sim, and OmniLRS Environment

Simulation and inspection of USD scenes are performed on **NVIDIA Isaac Sim**¹¹, part of the NVIDIA Omniverse platform¹², running on the same workstation [15, 16]. The reference lunar scene is not built from scratch: it is loaded from the

²<https://pytorch.org/>

³<https://docs.pytorch.org/vision/stable/index.html>

⁴https://huggingface.co/docs/transformers/model_doc/segformer

⁵<https://numpy.org/>

⁶<https://scipy.org/>

⁷<https://pandas.pydata.org/>

⁸<https://opencv.org/>

⁹<https://matplotlib.org/>

¹⁰<https://www.open3d.org/>

¹¹<https://developer.nvidia.com/isaac/sim>

¹²<https://www.nvidia.com/it-it/omniverse/>

OmniLRS¹³ photorealistic lunar robotics simulator [14], which provides Isaac Sim-based environments, terrain assets, and ROS 2 interfaces tailored to off-world robotics.

Following the official workflow, OmniLRS is deployed via its Docker¹⁴ image (`omnilrs.docker`), which bundles Isaac Sim, lunar terrain assets, and configuration files. The workstation simply launches the OmniLRS container, selects a lunar environment (e.g. Lunaryard or LargeScale), and exposes the resulting scene as a USD stage and a set of ROS 2 topics. This setup offers two advantages:

- the pipeline operates on a realistic lunar surface with procedural terrain, lighting, and rover models already available, reducing scene-modelling effort;
- the same ROS 2-based pipeline is intended to run in the future on real rover data, by replacing simulated OmniLRS topics with physical sensors, without changing the neuro-symbolic stack.

USD stages are generated or modified by Python scripts that use the USD Python API together with `libusdsem`. Semantic attributes attached to prims (Section 4.4) ensure that the USD representation remains aligned with the SPEx ontology and with ABoxes. The resulting **semantic USD scenes** provide a 3D, offline view of both perception outputs and reasoning results.

4.1.4 Build System and Project Layout

The project is organized as a **monorepo** with three main components:

1. a Python package for LuSNAR dataset handling, SegFormer training and evaluation;
2. a C/C++ library, `libusdsem`, which wraps Tiny-ME and USD and exposes a small C API;
3. ROS 2 packages defining nodes, message types, and launch configurations.

C and C++ components use **CMake** as build system. Compilation is performed with conservative flags (e.g. `-O2` and `-std=c11` or `-std=c++17`) to ensure portability. Unit tests cover:

- parsing and conversion of LuSNAR masks;
- OWL Functional syntax generation for ABoxes;
- `check_aln()` validation on representative ontologies;
- basic Tiny-ME queries on synthetic KGs fragments;
- round-trip conversion between USD stages and ABox fragments.

¹³<https://github.com/OmniLRS/OmniLRS>

¹⁴<https://www.docker.com/>

4.2 LuSNAR Dataset Preparation

4.2.1 Scene Selection and Frame Indexing

The LuSNAR dataset (Table A.11) is organized into nine scenes, each containing time-synchronized multi-sensor data and annotations. As in Chapter 2, scenes `Moon_1`, `Moon_2`, `Moon_4`, `Moon_6`, `Moon_8`, `Moon_9` are used for training and validation, while `Moon_3`, `Moon_5`, `Moon_7` form the test set.

Within each scene, only the left camera stream (`image0`) is used for supervised segmentation, as it provides aligned RGB, depth, and semantic labels. For each frame, a **global index** stores:

- the path to the RGB image;
- the path to the depth map;
- the path to the semantic mask;
- the frame timestamp and mapping to pose and IMU entries.

This index is stored in a simple tabular Comma-Separated Values (CSV) or Apache Parquet¹⁵ format and serves as the ground truth registry for both training and neuro-symbolic processing.

4.2.2 Semantic Mask Conversion and Normalization

LuSNAR semantic labels are encoded as color Portable Network Graphics (PNG) images. A fixed color palette maps RGB triplets to five terrain classes: **Regolith**, **Crater**, **Rock**, **Mountain**, and **Sky**. A preprocessing script converts each color mask into a single-channel label image with integer identifiers in $\{0, \dots, 4\}$. Pixels whose color does not match any palette entry are assigned an *ignore* label (value 255). This ensures robustness with respect to minor palette inconsistencies and allows the loss function to exclude uncertain pixels.

RGB images are resized to a fixed resolution compatible with SegFormer checkpoints (e.g. 512×512 or 640×640) via scale-and-crop transformations. The same geometric operations are applied to masks. Channel-wise normalization uses *ImageNet* statistics to preserve compatibility with backbone pre-training.

4.2.3 Data Augmentation

During training, online **data augmentation** increases robustness to variability in illumination and viewpoint. The following transformations are applied with configurable probabilities:

- random horizontal flipping, consistent between image and mask;

¹⁵<https://parquet.apache.org/>

- random scaling in a range such as $[0.75, 1.25]$, followed by cropping to the target resolution;
- mild color jittering (brightness, contrast, saturation);
- optional additive Gaussian noise.

Augmentations are implemented in the PyTorch data pipeline and are activated only in training mode. Validation and test frames are processed without augmentation.

4.3 SegFormer-Based Semantic Segmentation

4.3.1 Architecture and Configuration

SegFormer combines a hierarchical transformer encoder with a lightweight Multi-Layer Perceptron (MLP)-based decoder. In this work, a mid-sized backbone (e.g. B1 or B2) is adopted to balance accuracy and computational cost on the available GPU.

The encoder produces multi-scale feature maps at four resolution levels. The decoder:

1. projects each feature map to a common embedding dimension;
2. upsamples and fuses features across scales via concatenation and MLP layers;
3. applies a final linear classifier to obtain per-pixel logits over the five LuSNAR classes.

The model is initialized from ImageNet or ADE20K Scene Understanding Dataset (ADE20K) pre-trained checkpoints, where available, and the output head is replaced with a 5-class classifier.

4.3.2 Loss Function and Optimisation

Training uses a composite loss function $\mathcal{L}$ with a main cross-entropy term and optional class-imbalance corrections:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{aux}} \mathcal{L}_{\text{aux}},$$

where $\mathcal{L}_{\text{CE}}$ is the pixel-wise cross-entropy with ignore index set to 255, and $\mathcal{L}_{\text{aux}}$ can be a Dice loss or focal loss term when used.

Class weights in the cross-entropy are derived from empirical class frequencies in the training set to mitigate the dominance of regolith and sky with respect to rocks and craters. The weighting strategy is kept conservative to avoid over-compensating rare classes.

Optimization uses **Adaptive Moment Estimation with Decoupled Weight Decay (AdamW)** with cosine learning-rate decay and linear warmup. Hyperparameters (Learning Rate (lr), batch size, number of epochs) are set once and applied consistently across all experiments to preserve the comparability of downstream reasoning behavior.

4.3.3 Inference and Output Products

At inference time, SegFormer produces for each image:

- the argmax label map (integer mask), used for later object extraction;
- optional softmax probability maps over classes, used to define confidence scores for rock and crater instances.

These outputs are written to a structured directory and are optionally cached in an intermediate Hierarchical Data Format 5 (HDF5) or Zarr Array Storage Format (Zarr) store. The ABox builder consumes either masks directly from SegFormer during online operation on LuSNAR sequences or precomputed masks when processing data in batch mode.

4.4 The libusdsem Library: Bridging USD, SpEx, and Tiny-ME

4.4.1 Design Goals and Interface

The `libusdsem` library is a C/C++ library that encapsulates the interaction between USD scenes, the SPEX ontology, and the Tiny-ME reasoner. Its design goals are:

- providing a minimal API that can be invoked from ROS 2 nodes written in C++ or Python;
- enforcing $\mathcal{ALN}(\mathbf{D})$ compatibility of ontologies before they are passed to Tiny-ME;
- supporting a bidirectional mapping between USD prims and SPEX individuals for debugging and semantic inspection.

At a high level, `libusdsem` exposes functions to:

- load a SPEX TBox and one or more ABoxes fragments from OWL Functional syntax or ProtocOWL;
- validate ontologies using `check_aln()`;

- run Tiny-ME standard and non-standard services on the loaded KG (classification, CA, CC, CCov);
- export selected individuals and their inferred types back into a USD stage as semantic attributes.

4.4.2 Logical Language Validation: `check_aln()`

Given an OWL Functional syntax file, `check_aln()` performs a static analysis to verify compliance with the $\mathcal{ALN}(\mathbf{D})$ fragment targeted by Tiny-ME [5]. The procedure checks:

- that concept constructors are restricted to conjunction, negation of atomic concepts, universal restrictions, and unqualified number restrictions;
- that datatype properties use only supported numeric datatypes and interval-like constraints;
- that role axioms respect Tiny-ME requirements (e.g. no role chains or complex role inclusions);
- that TBox definitions are acyclic or primitive where required by the structural algorithms.

In case of violations, `check_aln()` reports the offending axioms and rejects the KG. This prevents runtime failures and ensures that reasoning behavior matches the theoretical guarantees described in Chapter 3 and in the literature on lightweight description logics [11, 13].

4.4.3 Mapping Between USD Prims and SPEX Individuals

USD *prims* representing terrain patches, rocks, craters, and rover components are annotated with a minimal set of semantic attributes:

- `sem:class`: an IRI string of the corresponding SPEX class (e.g. `http://spex.owl#Rock`);
- `sem:individual`: an optional IRI string of the associated individual in the ABox;
- `sem:label`: a human-readable tag used for debugging.

The `libusdsem` library provides:

- a function to scan a USD stage, read semantic attributes, and generate an ABox fragment that mirrors the scene graph;

- a function to attach or update semantic attributes on prims based on Tiny-ME inference results (e.g. classifying a patch as `TraversableLight`).

This mapping permits visual inspection of reasoning results within Isaac Sim. For example, a patch inferred to be a hazard can be colored in the 3D view by a custom shader that reads `sem:class` attributes, yielding an intuitive, offline 3D representation of the **symbolic state** [17].

4.4.4 Integration with Tiny-ME

Internally, `libusdsem` encapsulates an instance of Tiny-ME and manages the KG lifecycle:

- at startup, the SPEx TBox is loaded once and normalized;
- for each new ABox fragment (e.g. for a new frame or batch of frames), assertions are added to Tiny-ME using its C API [11, 12];
- requests encoded as individuals (Section 3.4) are instantiated, and match-making services are executed between request and resource individuals;
- results (scores and explanations) are returned to the calling ROS 2 node as in-memory structures and, when appropriate, reflected into USD.

The library hides all details of ProtocOWL and Tiny-ME data structures, exposing instead a small set of plain C functions and plain `structs` that are straightforward to wrap in C++ or Python.

4.5 ROS 2 Architecture and Node-Level Design

4.5.1 Overall ROS 2 Graph

In line with the middleware-agnostic design of Chapter 3, the runtime system is also expressed as a **ROS 2 graph** of loosely coupled nodes. The graph mirrors the batch pipeline and provides a deployment-oriented view in which perception, ABox construction, reasoning, and simulation/visualization are exposed as modular components.

The conceptual ROS 2 graph in Figure 3.5 illustrates the main node types and topics. The design separates perception, ABox construction, reasoning, and simulation/visualization into distinct components, consistent with the modularity of the KG. A partial prototype instantiates the perception and ABox-building nodes, together with a Tiny-ME wrapper and a USD bridge, and is used to validate topic interfaces and message formats, while the main quantitative evaluation is conducted in batch mode.

The main functional groups are:

- *Perception*: acquisition of RGB images, depth maps, LiDAR scans, IMU measurements, and SegFormer inference;
- *Symbolic layer*: construction and update of scene ABox fragments; instantiation of request individuals; matchmaking and reasoning;
- *Simulation and visualization*: interaction with USD stages via Isaac Sim; visual feedback for debugging and analysis;
- *Navigation interface*: publication of decision support outputs that can be bound to a planner or navigation stack.

4.5.2 Perception Nodes

The perception layer is designed so that both LuSNAR sequences and Isaac Sim scenes can be presented to the rest of the system as standard ROS 2 topics. Camera and LiDAR streams originate either from bag playback of LuSNAR data or from the simulated scene and are exposed as `sensor_msgs/Image`, `sensor_msgs/CameraInfo`, `sensor_msgs/PointCloud2`, and `sensor_msgs/Imu` messages on topics such as `/sim/camera_left/image_raw`, `/sim/camera_left/camera_info`, `/sim/lidar/points`, and `/sim/imu/data`. These streams can be recorded, replayed at different speeds, and combined with other logs, providing a smooth transition between fully offline experimentation and future closed-loop operation.

On top of these raw streams, the semantic segmentation front-end is implemented as a `segformer_node` that subscribes to the left camera image and its calibration data. Each incoming frame is first passed through the same preprocessing steps described in Section 4.2, namely resizing to the network input resolution and normalization with ImageNet statistics. The preprocessed image is then forwarded to the SegFormer model, which runs on the available GPU via the PyTorch runtime. The node converts the resulting logits into an integer label mask and publishes it as a `sensor_msgs/Image` message on a topic such as `/perception/segmentation`; when needed for diagnostics or further analysis, it can also emit compressed confidence or probability maps. The associated Quality of Service (QoS) profiles are configured in a best-effort mode with small queues, privileging low latency and tolerance to accelerated playback over strict reliability.

4.5.3 ABox Builder Node

Immediately downstream of the segmentation stage, a dedicated node turns pixel-level predictions into a symbolic description of the current scene.

This `abox_builder_node` listens to the semantic masks produced by the perception layer and, when available, also makes use of the aligned RGB images, depth maps or point clouds, and the IMU and pose estimates coming either from LuSNAR ground truth or from a SLAM component.

For each frame, the node analyses non-regolith regions in the mask to isolate connected components corresponding to candidate rocks, craters, and mountains. Geometric descriptors such as centroids, approximate sizes, local slopes, and distances to the rover are extracted from the depth or point-cloud information, and these continuous values are then mapped into qualitative bins (for instance `:Near`, `:Small`, `:LowSlope`) based on thresholds defined in a JSON configuration file. With these ingredients in place, the node assembles the Scene ABox in OWL Functional: it introduces a frame individual, a collection of surface patches, specific individuals for each detected rock, crater, and mountain, and a rover-state individual whose attributes reflect the current inertial measurements. The resulting ABox fragment is serialized in ProtocOWL or in a compact custom format and published on a topic such as `/kb/scene_abox`, together with lightweight diagnostic summaries that report, for example, the number of detected instances per class.

4.5.4 Matchmaking and Reasoning Node

The symbolic reasoning step is concentrated in a node that embeds Tiny-ME through the `libusdsem` interface. During initialisation, this component loads the SPEx TBox, checks its compliance with the $\mathcal{ALN}(D)$ fragment using `check_aln()`, and constructs the internal data structures required for matchmaking. In the same phase it also imports the static profiles of the different rovers (Alpha and Beta have been defined for testing) and prepares the request individuals describing typical goals, including pickup tasks, obstacle avoidance, crater crossing, or energy-saving behaviors.

Once running, the node continuously listens to the Scene ABoxes published on `/kb/scene_abox`. Each fragment is imported into the Tiny-ME knowledge base and combined with the static information. For every request that is considered active, the node invokes the appropriate matchmaking service between the corresponding request concept and the candidate resource individuals representing objects in the scene, such as rocks, terrain patches, or rover states. Tiny-ME returns both a compatibility score and an explanation decomposed into CC and CA terms, which are then compacted into a message-friendly representation. The node gathers these results into ranked lists of candidates and publishes them on decision-support topics such as `/dss/alpha/pickup_candidates`, `/dss/alpha/avoidance_patches`, and `/dss/beta/pickup_candidates`, where each entry is identified by a unique individual identifier, its score, and a succinct trace of the most relevant explanatory concepts. The reasoning layer therefore offers a semantic ordering of options, leaving the responsibility for concrete control actions to downstream modules.

4.5.5 USD Bridge and visualization Node

To connect symbolic reasoning with 3D visual inspection, the architecture includes a node that operates directly on USD stages. This component, referred to as the

`usd_bridge_node`, subscribes to the decision-support topics produced by the reasoning layer and, when needed, to summaries of the corresponding Scene ABoxes. Using the USD Python API or a C++ binding, it identifies the prims that represent the relevant terrain patches, rocks, craters, or rover elements and updates their semantic attributes through `libusdsem`.

In practice, the node takes classifications and scores coming from Tiny-ME and writes them back into the 3D scene as `sem:class`, `sem:individual`, or related attributes. A patch that has been recognized as a hazard, for instance, is tagged with the corresponding SPEx class such as `:HazardPatch`. Once the stage has been updated, the same scene can be opened in Isaac Sim and rendered with shaders that read these semantic attributes, producing a visual overlay of the symbolic state on top of the geometric environment. This mechanism mirrors the batch-mode export of semantic USD frames, but it is now driven by the live ROS 2 graph and can be used to analyze how reasoning outcomes evolve along a sequence.

4.5.6 Navigation Interface Node

Downstream of decision support, the navigation-facing part of the graph is handled by a node that exposes the reasoning results to conventional motion-planning stacks. This `nav_interface_node` consumes the ranked lists of pickup, avoidance, and crossing candidates produced by the reasoning layer, while at the same time accessing the current rover pose and a local cost map or occupancy grid generated by a SLAM or mapping node.

Given these inputs, the node can transform semantic preferences into artifacts that a standard navigation pipeline can directly interpret: for example, it can derive waypoints that lead towards the highest-ranked pickup targets under feasibility constraints, or it can adjust a cost map by increasing the traversal cost of cells corresponding to patches classified as hazardous. When appropriate, it also emits explicit warnings or soft constraints on dedicated topics, such as messages indicating that a certain region identified by an individual identifier should be avoided. The aim is not to replace existing planners but to provide a thin, well-defined boundary at which symbolic reasoning can influence classical navigation modules.

4.5.7 Node Life-Cycle and Synchronization

The various components in the graph follow ROS 2 managed life-cycle patterns wherever this helps with testing and deployment. Perception and ABox-building nodes can be activated or stopped independently from the reasoning and visualization layers, which simplifies experiments where only a subset of the pipeline needs to run. The reasoning component exposes services that allow the SPEx TBox or static ABoxes to be reloaded without restarting the entire system, which is useful when iterating on ontology design. Synchronization among segmentation masks, depth data, and the construction of Scene ABoxes is handled through approximate time filters if timestamps are not perfectly aligned, so that both LuSNAR-based

sequences and Isaac Sim streams can be processed consistently, whether they are played back in real time or from recorded bags. In this way, the ROS 2 graph remains as close as possible to the batch pipeline while still supporting realistic deployment scenarios.

4.6 Summary

This Chapter 4 has described the **implementation** of the neuro-symbolic pipeline for lunar navigation. SegFormer provides a lightweight yet expressive perception module trained on LuSNAR [4], generating semantic masks that are converted into object-level instances by the ABox builder. `libusdsem` bridges USD scenes, the SPEx ontology, and Tiny-ME, enforcing $\mathcal{ALN}(\mathbf{D})$ compatibility and enabling bidirectional exchange between symbolic and geometric representations. A **ROS 2** graph mirrors the same logical stages as the batch pipeline and organizes perception, symbolic reasoning, and semantic 3D visualization into modular nodes on a single workstation. By leveraging an OmniLRS lunar scene running in Docker on top of Isaac Sim, the prototype already operates on a realistic lunar environment and anticipates a future deployment in which the same ROS 2 pipeline is executed on real rover data.

The next **Chapter 5** will build on this implementation to present the **experimental scenarios, evaluation metrics**, and quantitative analysis of accuracy, runtime, and reasoning overhead of the proposed neuro-symbolic framework.

Chapter 5

Experimental Results

This chapter presents the **experimental evaluation** of the proposed neuro-symbolic navigation pipeline. The goal is to quantify (i) the **segmentation quality and efficiency** of the SegFormer module on LuSNAR lunar scenes and (ii) the **runtime cost and scalability** of the downstream symbolic and 3D components: mesh reconstruction, 2D→3D semantic projection, ABox construction, Tiny-ME matchmaking, and semantic USD generation and validation.

The chapter is organised as follows. **Section 5.1** details the dataset and pre-processing. **Section 5.2** describes the SegFormer training setup. **Section 5.3** explains the evaluation protocol and metrics, both for segmentation and for the end-to-end pipeline. **Section 5.4** reports quantitative segmentation results together with a qualitative analysis based on LuSNAR triptych images. **Section 5.5** discusses the end-to-end timings of the neuro-symbolic pipeline, the structure of the matchmaking results produced by Tiny-ME, and their implications for a future ROS 2 deployment.

5.1 Dataset and Preprocessing

5.1.1 LuSNAR Dataset and Camera Choice

Experiments are conducted on the LuSNAR dataset, which provides stereo RGB images, depth, LiDAR, IMU and semantic labels for synthetic lunar rover navigation scenarios [4]. Following the dataset guidelines, **training** uses moderately complex scenes, while **testing** is performed on more challenging ones.

The semantic segmentation network operates on the left RGB camera stream (folder `image0/color`). Depth and LiDAR are consumed downstream when enriching semantic detections with geometric attributes and when reconstructing 3D meshes used in the USD export.

Let $\mathcal{S}_{\text{train}}$ and $\mathcal{S}_{\text{test}}$ denote the sets of scenes used for training/validation and testing, respectively:

$$\begin{aligned}\mathcal{S}_{\text{train}} &= \{\text{Moon_1}, \text{Moon_2}, \text{Moon_4}, \text{Moon_6}, \text{Moon_8}, \text{Moon_9}\}, \\ \mathcal{S}_{\text{test}} &= \{\text{Moon_3}, \text{Moon_5}, \text{Moon_7}\}.\end{aligned}$$

From $\mathcal{S}_{\text{train}}$ a train/validation split is derived with a ratio of 0.85/0.15, while the three test scenes are kept completely unseen during training and early stopping.

5.1.2 Label Processing and Image Resizing

The raw LuSNAR semantic labels are provided as color-coded PNG images. Each RGB triplet corresponds to one of the five semantic classes:

- Lunar regolith,
- Impact crater,
- Rock,
- Mountain,
- Sky.

To obtain dense integer label maps suitable for learning, RGB labels are converted to class IDs using a fixed color palette and a small color tolerance in RGB space. Pixels whose color is farther than a configurable threshold from any known class color are assigned to a dedicated “*unlabeled*” ID (255). During training, these pixels are ignored by the loss functions and by the evaluation metrics.

Images and masks are spatially resized to a fixed resolution of 512×512 pixels for training and validation. This yields a compact input size for the backbone while still preserving sufficient detail on small objects such as rocks. In the operational setting, semantic inference is later performed at 1024×1024 to remain aligned with the depth/LiDAR/IMU streams and camera intrinsics used by the rest of the pipeline.

5.1.3 Final Dataset Structure

After preprocessing, the dataset is organised as:

- **train/images**, **train/masks**: resized RGB images and integer masks for training;
- **val/images**, **val/masks**: validation split derived from the same training scenes;
- **test/Moon_3**, **test/Moon_5**, **test/Moon_7**: per-scene test folders, each with its own **images/masks** subdirectories.

Quality-assurance visualizations are generated by mapping class IDs back to RGB colors and overlaying them on the original images. Representative examples are used during development to monitor data quality but are not central to Chapter 5, except when explicitly discussed in the qualitative analysis of Subsection 5.4.4.

5.2 Training Setup

5.2.1 Network Architecture and Initialization

Two variants of the SegFormer architecture are evaluated, denoted as **SegFormer-B0** and **SegFormer-B1**. Both are initialised from ImageNet/ADE20K-pretrained checkpoints and adapted to the five LuSNAR classes by replacing the segmentation head. The transformer-based encoder produces multi-scale features, which are fused by a lightweight MLP decoder to produce dense per-pixel class logits.

All models are trained with input size 512×512 , batch size 32, and mixed-precision training (AMP) on a single GPU.

5.2.2 Loss Functions and Class Balancing

LuSNAR exhibits moderate class imbalance: large regions of lunar regolith and sky dominate the images, while impact craters and small rocks cover comparatively fewer pixels. To compensate for this, an empirical class-frequency distribution is estimated on the training split and per-class weights are derived by inverting the frequencies and renormalizing.

The training objective is a composite loss:

$$\mathcal{L} = \frac{1}{2} \mathcal{L}_{\text{CE}} + \frac{1}{2} \mathcal{L}_{\text{Dice}}, \quad (5.1)$$

where:

- $\mathcal{L}_{\text{CE}}$ is a **weighted multi-class cross-entropy** loss with ignore index set to 255,
- $\mathcal{L}_{\text{Dice}}$ is a **multi-class Dice loss**, also using the same class weights and ignore index.

Cross-entropy encourages correct per-pixel classification, while Dice loss promotes good overlap between predicted and ground-truth segments, especially for small or rare classes. The ignore index ensures that unlabeled pixels do not contribute to either term.

5.2.3 Data Augmentation

The training pipeline supports a set of label-safe augmentations, including:

- random horizontal flips,
- mild color jitter in Hue Saturation Brightness (HSV) space,
- Gaussian blur as a noise proxy,
- random cropping and rescaling back to 512×512 .

In the final experiments reported in this chapter, **all online augmentations are disabled**. This choice keeps the training distribution close to the synthetic rendering conditions of the LuSNAR dataset and simplifies the analysis of the segmentation and reasoning pipeline. Generalization is mainly supported by the diversity of scenes and by class balancing in the loss.

5.2.4 Optimization and Early Stopping

All models are optimized with the AdamW optimizer (learning rate 10^{-4} , weight decay 10^{-4}). The learning rate schedule is cosine decay with a warmup phase covering 5% of the total training steps.

Each model is trained for up to 200 epochs, with early stopping based on validation mIoU. Training is stopped if no improvement larger than a tolerance $\Delta = 10^{-3}$ in validation mIoU is observed for 10 consecutive epochs. The checkpoint with the best validation mIoU is saved and used for all subsequent test and benchmarking experiments.

5.3 Evaluation Protocol and Metrics

5.3.1 Scene-Wise Segmentation Testing and Output Artifacts

Segmentation performance is evaluated through a dedicated testing and visualization pipeline operating on the LuSNAR test scenes. For each SegFormer variant, the pipeline:

- loads the selected checkpoint and rebuilds the model with the correct number of classes;
- iterates over all images of each test scene (Moon_3, Moon_5, Moon_7) with batch size 1;
- for every image, produces the predicted class-ID mask, and optionally exports colorised predictions, confidence and entropy maps, contour overlays on RGB, and triptych panels (*RGB / Ground truth / Prediction*);
- updates a confusion matrix per scene and an overall confusion matrix across scenes;
- stores per-image statistics in tabular form (per-class pixel counts, per-image mIoU and mAcc, mean confidence/entropy, and latency measures);
- writes per-scene summaries (metrics and timing distributions) and an overall summary in JavaScript Object Notation (JSON) and CSV format.

The final tables and plots are derived from these confusion matrices and summary files, not from training logs. In particular, the confusion matrices are stored both as raw counts and as row-normalized matrices, which are useful to inspect asymmetric confusions between classes.

5.3.2 Segmentation Metrics

Segmentation quality is evaluated using standard metrics for multi-class semantic segmentation:

- per-class **Intersection over Union (IoU)**,
- **Mean Intersection over Union (mIoU)** over all classes,
- per-class **pixel accuracy**,
- **mean pixel accuracy (mAcc)** across classes,
- **overall pixel accuracy** over all classes and pixels.

All metrics are computed from the confusion matrices and ignore unlabeled pixels. Per-image confusion matrices are accumulated into per-scene matrices and then into a global matrix spanning all test scenes. The values reported in Table 5.1 and Table 5.2 and in the corresponding plots are computed from the global test confusion matrix.

5.3.3 Segmentation Latency and Overhead Decomposition

Runtime of the segmentation module is characterized with a focus on reproducibility and on isolating different sources of overhead. The evaluation pipeline performs the following steps:

- a fixed number of warm-up iterations to stabilise GPU clocks and caches before timing;
- measurement of forward-pass latency using CUDA events when a GPU is available, falling back to high-resolution CPU timers otherwise;
- explicit CUDA synchronization around timed regions to avoid scheduling bias.

For every image, three latency values are recorded:

1. **Forward-only latency**: model compute time from input tensors to logits;
2. **End-to-end latency without disk I/O**: time from input tensors to post-processed predictions and metrics (including interpolation, softmax, confidence/entropy computation and confusion-matrix updates);

3. **End-to-end latency with disk I/O:** a strict upper bound including all earlier steps plus reading the input image from disk and writing all requested artefacts (masks, overlays, CSV rows, triptychs).

From these quantities the pipeline derives absolute overheads:

$$\text{overhead_no_io} = T_{\text{E2E,no-io}} - T_{\text{fwd}}, \quad \text{overhead_io} = T_{\text{E2E,io}} - T_{\text{E2E,no-io}},$$

as well as their ratios with respect to forward latency, and the shares of forward, post-processing and I/O time over the full end-to-end latency. For each latency and overhead, the pipeline reports central tendency and tail statistics (mean, median/p50, p90, p95) on a per-scene basis and globally across all test scenes.

All segmentation metrics are reported at a resolution of 512×512 , while runtime benchmarks are obtained at 1024×1024 to match the operational configuration used by the downstream consumers.

5.3.4 End-to-End Pipeline Benchmarking

Beyond the segmentation module, the thesis evaluates the *entire* neuro-symbolic pipeline on LuSNAR at the granularity of a single frame. For a selected scene and frame index, the pipeline executes the following stages in sequence:

1. **Single-frame mesh reconstruction** from depth and RGB, producing a colored triangular mesh and associated metadata.
2. **Semantic projection** from 2D SegFormer masks, confidence and entropy maps onto mesh vertices and faces, yielding face-level class IDs and statistics.
3. **Timestamp extraction** and alignment to LuSNAR time-stamps to identify the correct ABox frame and ground-truth pose entry.
4. **ABox construction** from segmentation, depth and IMU, generating a ProtocolOWL ABox fragment that instantiates the SPEx TBox for the current scene and rover state.
5. **Tiny-ME matchmaking** on the merged TBox and ABox, computing compatibility scores and explanations for two rover profiles (Alpha, Beta) and two goal templates (pickup, obstacle avoidance).
6. **USD export** of a per-frame scene containing the mesh, face semantics, frame pose, and ProtocolOWL ABox payload as USD attributes.
7. **Fix-up of ABox attributes** to enforce the canonical base64-based ProtocolOWL encoding used by the validator.
8. **USD validation** of the embedded ABoxes through byte-wise and hash-based checks, as discussed in Subsection 5.5.4.

Each stage is wrapped in a *scientific performance layer* that measures and records:

- wall-clock duration (seconds) for the stage;
- operating-system reported CPU utilization (percentage, normalized to $[0, 100]$);
- maximum resident memory usage (kB);
- elapsed wall time in human-readable form;
- a short sample of the textual output for diagnostic purposes.

These statistics are appended to a machine-readable log file containing a list of *runs*. From this log, simple analysis scripts compute descriptive statistics per stage (mean, median, variance, p90) and aggregate timings to obtain the total per-frame cost of the mesh+semantics+ABox+USD pipeline.

Importantly, this benchmark operates *downstream* of the SegFormer inference step. Segmentation masks and, when needed, confidence/entropy maps are assumed to be produced by a preceding stage, so that the pipeline-level evaluation focuses on the incremental cost of the symbolic and 3D components. The combination of segmentation latency and per-stage pipeline timings addresses Research Question **RQ3** on the feasibility of the neuro-symbolic loop under edge-like constraints.

5.4 Segmentation Results

5.4.1 Global Segmentation Performance

Table 5.1 reports the global segmentation metrics on the LuSNAR test set for SegFormer-B0 and SegFormer-B1. Both models reach high overall accuracy and strong mean IoU and mean accuracy, despite being trained on medium-complexity scenes and tested on more challenging ones.

Table 5.1: Global performance on the LuSNAR test set (Moon_3, Moon_5, Moon_7).

Metric	SegFormer-B0	SegFormer-B1	Δ (B1-B0)
mIoU	0.843	0.851	+0.008 (+1%)
mAcc	0.916	0.920	+0.004 (< 1%)
Overall Acc	0.985	0.986	+0.001

These aggregate results are visualized in Figure 5.1, which shows the direct bar-plot comparison between SegFormer-B0 and SegFormer-B1 for mIoU, mAcc and overall accuracy. The numeric values annotated on the bars make the small

performance gap explicit: B1 consistently outperforms B0, but the gains are confined to about one percentage point in mIoU and less than one percentage point in mAcc.

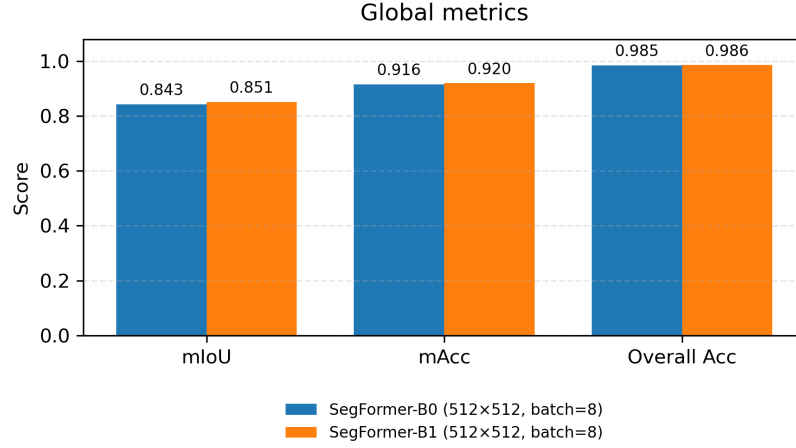


Figure 5.1: Global metrics comparison between SegFormer-B0 and SegFormer-B1 (mIoU, mAcc, overall accuracy).

5.4.2 Per-Class Analysis

Table 5.2 details the IoU and pixel accuracy for each semantic class. For all classes, performance is high, confirming that the models generalize well across surface types.

Table 5.2: Per-class IoU and accuracy on the LuSNAR test set.

Class	IoU (B0)	IoU (B1)	Acc (B0)	Acc (B1)
Lunar Regolith	0.983	0.984	0.990	0.990
Impact Crater	0.706	0.711	0.787	0.784
Rock	0.700	0.713	0.921	0.926
Mountain	0.829	0.853	0.886	0.903
Sky	0.994	0.995	0.998	1.000

Figure 5.2–Figure 5.4 provide a more compact visualization of these numbers for the most relevant non-trivial classes (Rock, Mountain, Impact crater). Each plot shows four bars: IoU and accuracy for B0 and B1.

For Rocks (Figure 5.2), both architectures obtain IoU around 0.70 and accuracy above 0.92, with B1 slightly ahead. Rocks are the most challenging class because of their small size and high variability, so these values are particularly relevant for rover safety.

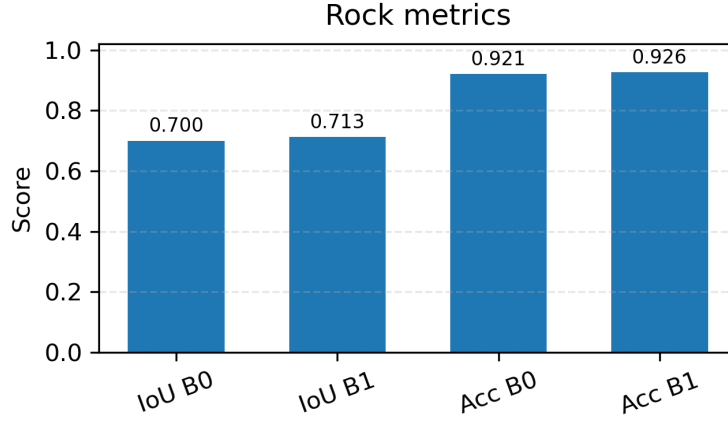


Figure 5.2: Per-class metrics for Rocks: IoU and accuracy for SegFormer-B0 and SegFormer-B1.

For Mountains (Figure 5.3), the B1 variant shows a clearer advantage, with an IoU gain of about 2.4 points and an accuracy gain of about 1.7 points. This reflects its slightly higher capacity, which helps capture extended structures with more complex geometry.

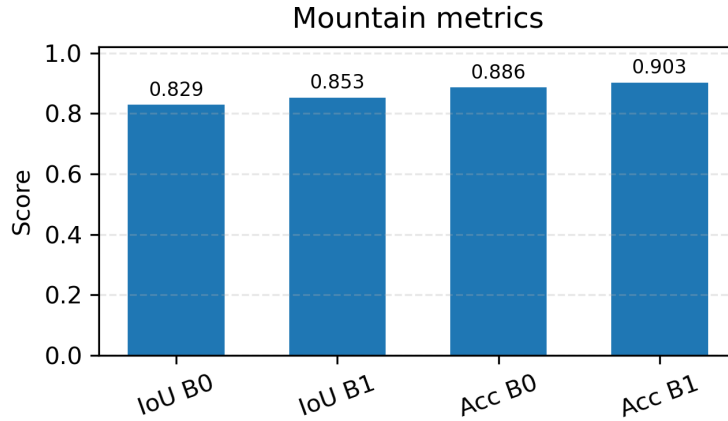


Figure 5.3: Per-class metrics for Mountains: IoU and accuracy for SegFormer-B0 and SegFormer-B1.

Impact craters (Figure 5.4) also benefit from the B1 architecture, although the improvement is modest. IoU increases from 0.706 to 0.711, while accuracy remains essentially unchanged. Since craters are relatively sparse but large-scale features, they are already well captured by both models.

Overall, the per-class analysis confirms that the composite loss with class

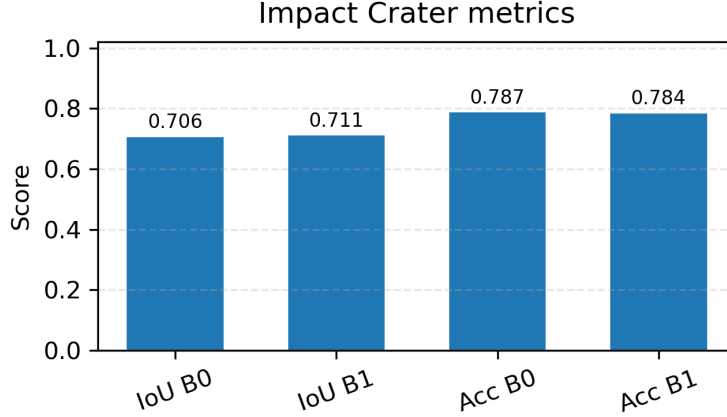


Figure 5.4: Per-class metrics for Impact craters: IoU and accuracy for SegFormer-B0 and SegFormer-B1.

weighting and the careful treatment of unlabeled pixels stabilises performance across classes, rather than concentrating capacity only on the most frequent ones (Lunar Regolith and Sky).

5.4.3 Computational Efficiency of SegFormer

Table 5.3 compares the computational cost of SegFormer-B0 and SegFormer-B1 at 1024×1024 resolution, which is the operational setting used when integrating with the rest of the navigation pipeline. The values correspond to the global means of the latency distributions collected over all test scenes.

Table 5.3: Computational efficiency at 1024×1024 resolution.

Metric	SegFormer-B0	SegFormer-B1	Δ (B1–B0)
Forward mean (ms)	3.9	5.4	+38%
E2E w/ I/O mean (ms)	190.7	192.0	+0.7%
FPS (forward)	254.8	184.8	–27%
FPS (effective)	5.24	5.21	$\approx 0\%$

Figure 5.5 and Figure 5.6 visualize these timings. The first focuses on the pure forward-pass latency (model compute only), while the second reports the end-to-end latency including I/O and post-processing. Both plots annotate the numerical timings on top of each bar to emphasize the relative difference between B0 and B1.

At the pure model level, SegFormer-B0 is substantially faster, processing roughly 255 frames per second versus 185 for B1. The difference in forward latency is about 1.5 ms per frame, corresponding to a 38% relative increase when moving from B0

to B1. This margin is significant when the segmentation module is part of a larger perception stack or must co-exist with other GPU-intensive tasks.

When considering the full end-to-end segmentation pipeline (including image loading, transfers, and disk I/O), the difference between B0 and B1 becomes almost negligible: the effective FPS is around 5.2 frames per second in both cases. This indicates that, in the current implementation, I/O and pre/post-processing dominate the timing budget. Nonetheless, the pure forward-pass latency remains a useful indicator of how the system will behave once I/O is optimized or moved to asynchronous streams, and it provides a safety margin for future extensions (*e.g.*, additional cameras or segmentation heads).

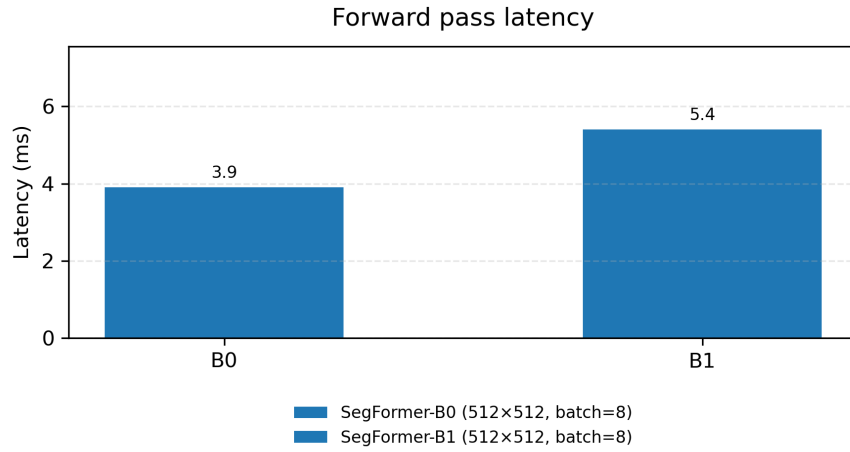


Figure 5.5: Forward-pass latency (ms) at 1024×1024 for SegFormer-B0 and SegFormer-B1.

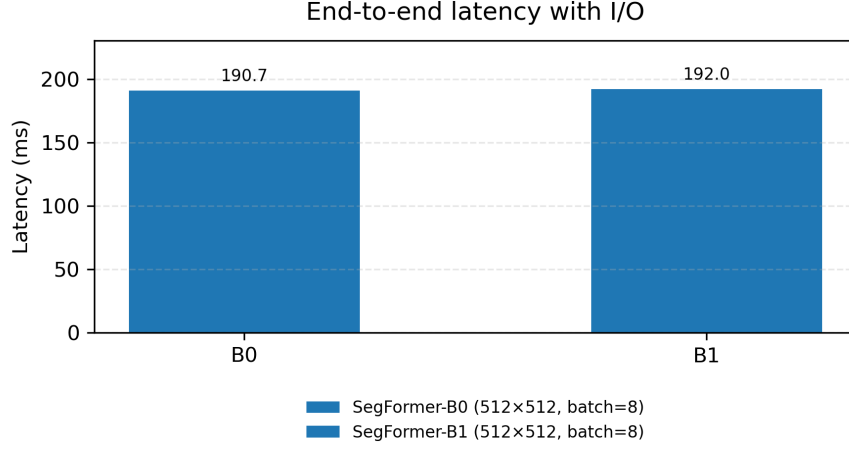


Figure 5.6: End-to-end latency with I/O (ms) at 1024×1024 for SegFormer-B0 and SegFormer-B1.

5.4.4 Qualitative Analysis and Limitations on LuSNAR Triptychs

To complement the aggregate metrics, this section analyses representative **triptych panels** of the form *RGB / Ground truth / SegFormer-B0 prediction* extracted from the Moon_3 test scene. The goal is to expose typical success modes and residual limitations that are relevant for downstream reasoning.

Baseline case: correct inference in a moderately cluttered scene. Figure 5.7 shows a frame (1693290477807724288) where SegFormer-B0 produces an almost perfectly aligned segmentation. Lunar regolith, mountain slopes and sky are correctly separated, and rocks are sharply delineated, including small objects near the rover. This panel is representative of the majority of frames and justifies the high global IoU for both Rocks and Mountains reported in Table 5.2.

Complex co-presence of classes and rock inference. Figure 5.8 corresponds to timestamp 1693290361863365888. The scene contains a challenging mixture of **rocks, impact craters, mountain, sky and cast shadows**. Despite this complexity, the predicted rock instances closely follow the ground-truth shapes, including small boulders partially occluded by craters or shadows. This example corroborates the quantitative evidence that Rocks are segmented reliably enough for safe object instantiation in the ABox.

Mild deviations along crater boundaries. A different failure mode is illustrated in Figure 5.9 for frame 1693290392722639872. Here the dominant

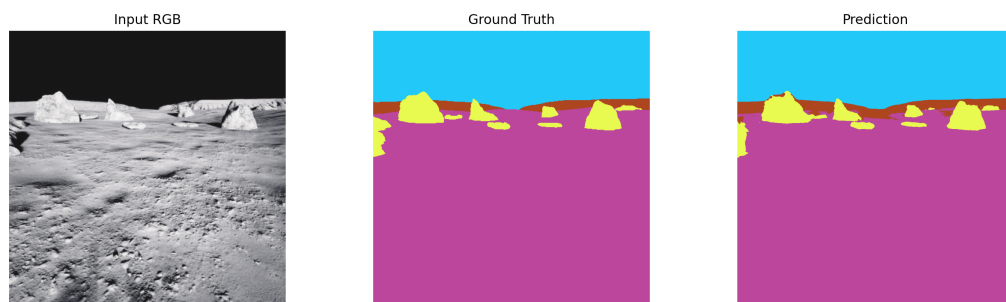


Figure 5.7: Triptych 1693290477807724288 (RGB | GT | SegFormer-B0), Moon_3 test frame.

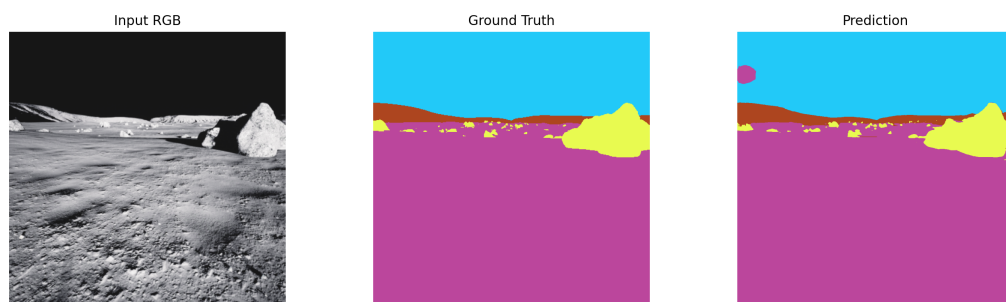


Figure 5.8: Triptych 1693290361863365888 (RGB | GT | SegFormer-B0), Moon_3 test frame.

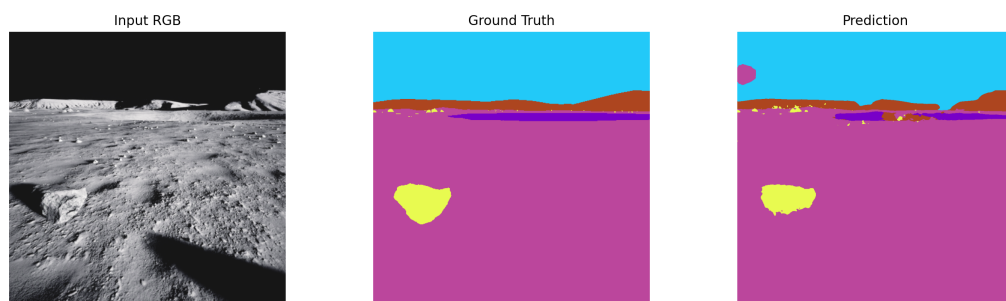


Figure 5.9: Triptych 1693290392722639872 (RGB | GT | SegFormer-B0), Moon_3 test frame.

structures are impact craters with smooth, low-contrast rims. The SegFormer prediction exhibits **slight boundary shifts** with respect to the ground truth: crater contours are sometimes eroded towards the interior or slightly dilated into the surrounding regolith. At pixel level, this manifests as thin bands of confusion along crater rims, which helps explain why Impact crater IoU (around 0.71) lags behind the very high IoU values observed for Lunar Regolith and Sky. From a reasoning perspective, these small boundary errors have limited impact because crater instances are large-scale objects and remain correctly detected, but they may affect precise geometric descriptors such as estimated crater area.

Limitations in rover self-shadow. Figure 5.10 shows a more critical situation for frame 1693290369163568640, where a significant portion of the scene is covered by the **shadow of the simulated rover itself**. In this region, texture and illumination cues are strongly degraded, and the model displays:

- local uncertainty in distinguishing rocks from shadowed regolith;
- slight fragmentation of contiguous rock regions in the vicinity of the shadow boundary;
- small misclassifications along the contact line between the rover shadow and surrounding terrain.

These phenomena are consistent with the known sensitivity of purely appearance-based segmentation to strong illumination changes. They represent the main qualitative limitation observed in the LuSNAR test scenes and highlight the importance of combining image-based predictions with depth and IMU information when building the ABox, for instance by enforcing geometric plausibility constraints on rock size and slope.

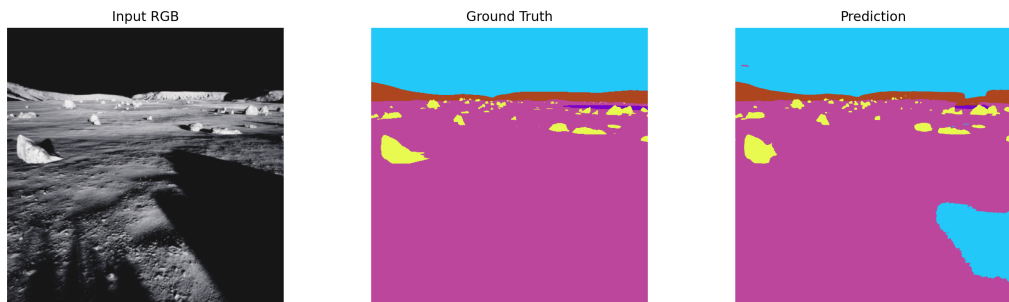


Figure 5.10: Triptych 1693290369163568640 (RGB | GT | SegFormer-B0), Moon_3 test frame.

Summary of qualitative limitations. Qualitative inspection of the triptychs confirms the quantitative trends:

- Rocks are generally segmented with high fidelity, even in cluttered scenes, which supports their use as pickup candidates in the symbolic layer.
- Impact craters suffer from small boundary shifts along low-contrast rims, which explain their slightly lower IoU and should be taken into account when interpreting crater size or shape in the reasoning process.
- Strong shadows, especially rover self-shadows, remain the main source of local segmentation errors, suggesting that additional training data or explicit shadow-aware data augmentation would be beneficial in future work.

These limitations are explicitly considered when designing the ABox builder and Tiny-ME profiles: thresholds on rock size, slope, and confidence are selected to remain robust against the failure modes observed in the triptych analysis.

5.4.5 Model Selection for Neuro–Symbolic Integration

Combining the accuracy and efficiency results, SegFormer-B1 offers slightly higher segmentation accuracy, while SegFormer-B0 is clearly more efficient in terms of forward-pass latency. The absolute accuracy gap is small (around 1% in mIoU), whereas the computational gain of B0 is more pronounced at the model level.

For the purposes of this work, semantic segmentation is not an isolated task but a **front-end for a reasoning-driven navigation pipeline**. Each frame must be processed end-to-end as follows:

1. SegFormer inference produces a dense class map.
2. The ABox builder extracts connected components, enriches them with geometric and confidence features, and instantiates individuals and role assertions in an OWL ABox.
3. Tiny-ME, operating on an $\mathcal{ALN}(D)$ TBox, performs matchmaking and constraint-based reasoning in real time to evaluate candidate resources and rover actions.

In this pipeline, the segmentation module must provide sufficiently accurate maps while leaving computational headroom for ABox construction and reasoning. The higher forward latency of B1 reduces this headroom without bringing a proportionally large gain in reasoning-relevant accuracy. In particular, Rocks and Mountains already achieve IoU around 0.70–0.85 and accuracy above 0.90 with B0, which is adequate for robust object instantiation and hazard identification.

For these reasons **SegFormer-B0 is adopted as the default semantic segmentation backbone** in all subsequent integration experiments. This choice maximises the overall system throughput and makes it easier to maintain real-time operation once ABox generation and reasoning are executed at the same frame rate.

5.5 End-to-End Pipeline Timings and 3D Semantics

5.5.1 Per-Stage Runtime Decomposition

Using the instrumentation described in Subsection 5.3.4, the neuro-symbolic pipeline is executed frame-wise on LuSNAR scenes to obtain per-stage timings and resource usage. For each stage listed above, the benchmark provides:

- the distribution of wall-clock durations over repeated runs;
- the average CPU utilization reported by the operating system (normalized to $[0, 100]$);
- the peak resident memory usage (RAM footprint);
- qualitative logs useful to correlate anomalies in timing with specific frames or configurations.

A representative experiment is conducted on the Moon_3 scene in SPEX mode, using SegFormer-B0 predictions as 2D input to the 3D and symbolic stages. Ten frames are selected along the trajectory, and the full pipeline is executed on each frame. Table 5.4 summarises the resulting per-stage timings and memory usage, averaged over the 10 runs.

Table 5.4: Per-stage timings and memory usage for the neuro-symbolic pipeline on LuSNAR Moon_3 (SegFormer-B0, SpEx mode, 10 frames).

Stage	Mean time (s)	p90 time (s)	Mean RAM (MB)
Mesh reconstruction	3.83	4.19	671.1
Semantic projection	16.92	17.40	753.3
ABox construction	1.88	1.92	463.9
Matchmaking (Tiny-ME)	0.01	0.01	2.9
USD export	4.64	4.76	621.4
USD fix (ProtocOWL)	0.14	0.16	88.0
USD validation	0.02	0.02	9.0

The total mean time per frame for the pipeline stages in Table 5.4 is approximately 27.4s on the reference workstation. The relative shares of this budget are strongly skewed:

- Semantic projection is the dominant cost, accounting for about 62% of the per-frame time. This is expected, as it requires loading dense meshes and 2D predictions, projecting hundreds of thousands of vertices back into image space, aggregating face-level statistics, and writing multiple outputs.

- Mesh reconstruction and USD export contribute a further 14% and 17% of the budget, respectively. Both are I/O-heavy and operate on large data structures (point clouds, triangle meshes, USD scenes), which explains the high peak RAM values.
- ABox construction represents roughly 7% of the total time. It traverses the 2D segmentation and depth data to instantiate a compact ProtocOWL ABox, but the schema is deliberately small and rock-centric, keeping the reasoning-relevant ABox size under control.
- Tiny-ME matchmaking, USD fix and USD validation are negligible in terms of wall-clock time, each contributing well below one second per frame. Their memory footprint is also modest compared to the 3D stages.

In terms of CPU usage, the heavy 3D and USD stages saturate a single core during their execution, while the very short reasoning and validation stages appear as brief bursts of activity. Overall, the measurements confirm that the dominant costs of the full pipeline are associated with 3D geometry processing and semantic asset generation, not with the logical reasoning itself.

Combined with the SegFormer results, these measurements answer RQ3 at a practical level: even with an unoptimized, fully synchronous implementation on a general-purpose workstation, the symbolic layer (ABox construction + Tiny-ME) remains a small fraction of the per-frame budget. The main bottleneck lies in the optional 3D semantics and USD export/validation chain.

5.5.2 Implications for ROS 2 Deployment

The per-stage timing breakdown has a direct architectural consequence for a ROS 2 deployment. A naive design in which all steps run strictly in sequence in a single node would couple:

- heavy, I/O-intensive operations (mesh reconstruction, semantic projection, USD writing and validation),
- latency-critical operations (ABox construction and Tiny-ME reasoning), which are responsible for navigation decisions.

The experimental evidence indicates that these components should be decoupled. In particular:

- The 3D semantic processing chain (depth-to-mesh reconstruction, semantic projection, USD export, and USD validation) should be executed in *parallel* with respect to the core perception+reasoning loop. In a ROS 2 architecture, this is naturally realised by a separate branch of nodes fed by the same sensor topics but publishing 3D semantic products on their own topics at a rate compatible with their higher computational cost.

- The real-time reasoning loop should remain as lightweight as possible: SegFormer inference on 2D images, ABox construction from masks, depth and IMU, and Tiny-ME matchmaking can form a compact chain of nodes operating at the desired control frequency (*e.g.*, a few Hertz (Hz)), without being blocked by 3D meshing or USD I/O.

Under this design, 3D semantics and USD export are primarily used for remote inspection and digital-twin simulation (for example in Isaac Sim), while the rover’s reactive loop relies on the 2D masks and on the symbolic state encoded in ABoxes and reasoning outputs. The executed pipeline experiments support this separation: they show that the reasoning core has a modest computational footprint compared to 3D processing, so it is feasible to keep Tiny-ME on the critical path for navigation decisions as long as mesh reconstruction and USD generation are moved to parallel ROS 2 branches or even to a remote workstation.

5.5.3 Reasoning Output Structure and Matchmaking Scores

Beyond timing, the experiments also characterize the content of Tiny-ME matchmaking results when operating on LuSNAR-derived ABoxes. For each frame, `libusdsem` passes to Tiny-ME:

- the SPEx TBox (ontology of rover capabilities, goals and rock context);
- a merged ABox containing: (i) static goal templates, (ii) rover profile individuals, and (iii) dynamically instantiated scene objects (*e.g.*, rocks) built from SegFormer+Ball-Pivoting Algorithm (BPA)+depth;
- a set of request/resource pairs, such as “AlphaPickupGoalReq vs rock-<timestamp>-023”.

Tiny-ME evaluates each pair by logic-based matchmaking in $\mathcal{ALN}(D)$ and produces a structured JSON document. At top level the file has the form

```
{
  "tbox_file": ".../TBOX_SPEX.owl",
  "abox_file": ".../ABOX_TOT_SPEX_<ts>.owl",
  "frame_id": "<ts>",
  "coherent": true,
  "matches": [
    {
      "id": 1,
      "title": "Alpha pickup vs rock-<ts>-009",
      "request_iri": "AlphaPickupGoalReq",
      "resource_iri": "rock-<ts>-009",
      "compatible": true/false,
      "abduction_penalty": ...,
      "contraction_penalty": ...,
    }
  ]
}
```

```

    "combined_penalty": ...,
    "max_penalty": ...,
    "score": ...,
    "keep": "...",          // only if incompatible
    "give_up": "...",       // only if incompatible
    "abduction_raw": "...",
    "contraction_raw": "...",
    "bonus_raw": "..."
  },
  ...
]
}

```

where each entry in `matches` corresponds to a single request/resource pair.

The fields `keep` and `give_up` appear in incompatible cases only and encode, at concept level, which part of the request description can be preserved (*keep*) and which conjuncts must be dropped (*give up*) to restore CS with the candidate resource. The fields `abduction_raw`, `contraction_raw` and `bonus_raw` store the full logical hypotheses computed by Tiny-ME (respectively for CA, CC, and surplus features), together with the corresponding numeric penalty or incentive.

Penalty-based scoring model. Following the theory of logic-based matchmaking, the experimental setup instantiates a penalty model built on two main components:

- *CA penalty* p_a : measures how much must be hypothesized in addition to the resource description to fully satisfy the request. A low p_a corresponds to high semantic similarity.
- *CC penalty* p_c : measures how much of the request must be given up to restore compatibility with the resource. A high p_c corresponds to strong semantic contrast.

These two components are combined by convex weights $w_a, w_c \in [0, 1]$ with

$$w_a + w_c = 1, \quad (5.2)$$

where w_a quantifies the emphasis on hypotheses in agreement (CA) and w_c the emphasis on parts in conflict (CC). In the experiments reported here, the following values are adopted:

$$w_a = 0.3, \quad w_c = 0.7,$$

so that inconsistencies between request and resource are penalised more heavily than missing but non-conflicting capabilities.

Let $P_{\max}$ denote the maximum penalty associated with a request, defined as the penalty of matching the most generic concept `Thing` to that request:

$$P_{\max} = \text{Penalty}(\text{Thing} \Rightarrow \text{Request}), \quad (5.3)$$

which is constant across all candidate resources for that request.

The combined penalty is then

$$p_{\text{comb}} = w_a p_a + w_c p_c, \quad (5.4)$$

and its normalized version

$$p_{\text{norm}} = \frac{p_{\text{comb}}}{P_{\text{max}}} \in [0, 1]. \quad (5.5)$$

The final match score (stored in the JSON as "score") is defined as

$$\text{score} = (1 - p_{\text{norm}}) \cdot 100 = \left(1 - \frac{w_a p_a + w_c p_c}{P_{\text{max}}}\right) \cdot 100, \quad (5.6)$$

which takes values in $[0, 100]$, with 100 corresponding to a perfect match.

Compatible vs incompatible matches. Tiny-ME first checks whether a request R and a resource S are logically compatible, i.e., whether $S \sqcap R$ is satisfiable w.r.t. the TBox. Two regimes are then distinguished:

- **Compatible case:** the conjunction $S \sqcap R$ is satisfiable. Only CA is needed, so CC is effectively zero. At implementation level, this corresponds to:

```
if compatible(res, req):
    abd = abduce(res, req)
    bonus = bonus(res, req)
    score = abd-score // no contraction weight
```

In this regime, $p_c = 0$ and the score depends solely on p_a (modulated by possible surplus bonuses).

- **Incompatible case:** the conjunction $S \sqcap R$ is unsatisfiable. Tiny-ME computes both an CA hypothesis and a minimal CC, yielding non-zero p_a and p_c , as well as explicit `keep/give_up` concepts in the JSON. The combined penalty p_{comb} and score then use the full weighted model with $w_a = 0.3$, $w_c = 0.7$.

In both cases, `bonus_raw` encodes a surplus description (features of the resource that go beyond the request) and a corresponding incentive value. This surplus is folded into the CA-based component and therefore contributes to increasing the final score without changing the penalty interpretation.

Interpretation for lunar rock selection. In the LuSNAR experiments, the requests correspond to high-level goals such as `AlphaPickupGoalReq`, which express, among others, constraints on:

- rock size (*e.g.*, small rocks only),

- rock slope (*e.g.*, low-slope surfaces),
- obstacle context (*e.g.*, low obstacle density),
- battery state and energy profile,
- perception confidence and pose stability.

The resources correspond to individual rock instances extracted from the segmentation+depth pipeline. For each rock r , the JSON describes whether the Alpha rover can safely attempt a pickup maneuver on r given the current ABox. High scores (near 100) are associated with rocks that fully satisfy the pickup profile, with minimal hypothesized information and no need to drop request constraints. Lower scores correspond to:

- rocks that are semantically close but require stronger hypotheses (moderate p_a , low p_c);
- rocks that fundamentally violate one or more hard constraints, such as being too large or lying on high slope (low p_a but high p_c).

Using the same $P_{\max}$ for all rocks under a given request ensures that scores in the JSON are directly comparable across candidates, which is essential to select pickup targets and to assess the global quality of the segmentation+reasoning loop on LuSNAR frames.

5.5.4 Hash-Based Validation of Embedded ABoxes

The end-to-end benchmark also covers the cost and robustness of validating ProtocolOWL ABoxes embedded in USD frames. As summarised in Chapter 4, a dedicated validator:

1. extracts `spex:ABoxPcowl` and `spex:ABoxPcowlFile` attributes from each frame prim in a USD scene;
2. resolves the referenced `.pcowl` file from disk, using a closest-timestamp policy consistent with the exporter;
3. decodes the payload either as base64-encoded bytes (preferred convention) or, in legacy mode, as plain UTF-8 text;
4. compares the payload against the file bytes and computes SHA-256 hashes for both sources;
5. checks that the file path stored in `spex:ABoxPcowlFile` is consistent with the resolved path.

Frames are deemed valid only if both the content and the path checks succeed. For the scenes considered in this thesis, the validator reports that all frames pass these checks, providing a strong guarantee that the ABoxes used by Tiny-ME during experimentation coincide exactly with those embedded in the semantic USD scenes.

From a runtime perspective, this validation is an offline cost: although it is included in the per-stage benchmark to characterize the full semantic USD workflow, it does not need to run in the same loop as navigation and reasoning. This further supports the separation between the real-time reasoning branch and the heavier 3D and validation tasks in an actual ROS 2 deployment.

5.5.5 Qualitative goal–resource matchmaking examples

To make the structure of the Tiny-ME output more concrete, representative goal–resource matches produced by the full pipeline on a LuSNAR frame are discussed here. In all cases, the starting point is the scene summarisation performed by the ABox builder in `libudsem`.

Consider a test frame in which the builder extracts:

- one lunar surface patch individual, annotated as a low-slope, low-obstacle, high-confidence region (*e.g.* `LowSlope`, `LowBoulderDensity`, `HighConfidence`);
- several rock instances (`rock-...`) classified as small or medium and at near or mid distance from the rover (*e.g.* `SmallRock`, `NearRock`, `MediumRock`, `MidRock`);
- a few mountains and craters (*e.g.* `mountain-...`, `crater-...`) to complete the scene context;
- the rover state, annotated with either a low-battery, strict-attitude profile (`Alpha`) or a high-battery, loose-attitude profile (`Beta`).

For each goal request individual (*e.g.* `AlphaPickupGoalReq`, `BetaPickupGoalReq`, `AlphaObstacleAvoidanceReq`, `BetaObstacleAvoidanceReq`), Tiny-ME computes:

- a *keep* conjunction: what the goal and the candidate resource consistently share;
- a *give up* part: goal-side constraints that need to be relaxed to avoid inconsistency;
- a *bonus* part: additional properties that the resource has but the goal did not explicitly request;
- a scalar *score* in $[0, 100]$, obtained as explained in Subsection 5.5.3.

Below three typical matches are summarized, using the actual penalties and scores obtained on a LuSNAR demo frame. The goal is not to list all logical details, but to show how the *ABox* scene type, the goal pattern and the resource profile interact semantically.

Example 1: Alpha pickup on a small, near rock in safe context. In this frame, the builder detects several small rocks close to the rover on a low-slope, low-obstacle patch. One such rock is classified as: `SmallRock` $\sqcap$ `NearRock` $\sqcap$ `LowSlopeRock` $\sqcap$ `LowObstacleContextRock` $\sqcap$ `RockPerceptionReliable` $\sqcap$ `RockInUnstablePose` while the rover is in an Alpha-like state with low battery and strict attitude: `LowBatteryProfile` $\sqcap$ `AttitudeOK_Strict`.

Matching this rock against `AlphaPickupGoalReq` yields a high but not perfect ranked score:

$$\begin{aligned} P_{\text{abduction}} &= 18.0, & P_{\text{contraction}} &= 2.0, \\ P_{\text{combined}} &= 10.0, & P_{\text{max}} &= 46.0, \\ \text{score} &\approx 100 \cdot \left(1 - \frac{10}{46}\right) \approx 78.3. \end{aligned}$$

The *keep* part contains all the expected structure for a “good” light pickup candidate in this scene:

- the rock is small and near (`SmallRock`, `SmallObject`, `Near`);
- the local slope and obstacle context are favourable (`LowSlopeRock`, `LowObstacleContextRock`);
- perception is reliable (`HighConfidence`, `RockPerceptionReliable`);
- the rover profile matches the Alpha pickup capability (`LowBatteryProfile`, `RoverCapabilityPattern`, `GoalCapabilityPattern`).

To reconcile the strict pose requirement in the Alpha pattern with the actual unstable rock pose, Tiny-ME places

`RockInStablePose`

in the *give up* part, with a small contraction penalty $P_{\text{contraction}} = 2.0$. At the same time, the fact that the rock is explicitly classified as unstable appears in the *bonus*:

$\sqcap \text{RockObject} \sqcap \neg \text{ImpactCrater} \sqcap \neg \text{Mountain},$

with a positive incentive. Intuitively, the scene offers almost everything that the Alpha pickup goal wants (small, near, safe, high-confidence rock), but the pose is slightly worse than ideal. The resulting score around 78 reflects this near-ideal but not exact match and is easily interpretable from the underlying symbolic explanation.

Example 2: Beta pickup on the same rock. The same ABox frame is then matched against the heavier `BetaPickupGoalReq`, intended for a more capable rover. In this case, the rock remains in a safe context and at near distance, but

the Beta goal is tuned towards medium/large rocks and a higher-energy profile. The corresponding penalties and score are:

$$\begin{aligned} P_{\text{abduction}} &= 15.0, & P_{\text{contraction}} &= 0.0, \\ P_{\text{combined}} &= 15.0, & P_{\text{max}} &= 30.0, \\ \text{score} &= 100 \cdot \left(1 - \frac{15}{30}\right) = 50.0. \end{aligned}$$

Here the *keep* part encodes the fact that the scene provides a rock in a safe, well-perceived context and that the Beta rover has sufficient capability to operate in such conditions (high battery, loose attitude). However, several rock-specific details of this instance (*e.g.* being small rather than medium/large, the precise pose class) are not required by the Beta pickup pattern and therefore appear in the *bonus* as surplus information. No explicit constraint needs to be dropped (*give up* is empty), but the combined penalty is larger than in the Alpha case because more structure must be hypothesized to fit the heavier goal pattern. The resulting score of 50 clearly indicates that this rock is usable for Beta, but less ideal than for Alpha.

Example 3: Alpha pickup on a mountain. The same frame also contains mountains and craters. When `AlphaPickupGoalReq` is matched against a mountain individual, the `ABox` builder has only annotated it as a `Mountain` in a low-slope area. From the point of view of the Alpha pickup pattern, almost everything that makes a good pickup rock (size bin, rock-specific distance feature, obstacle context, rock pose, rock perception) is missing and must be introduced abductively.

This is reflected by a much larger abduction penalty and a low final score:

$$\begin{aligned} P_{\text{abduction}} &= 38.0, & P_{\text{contraction}} &= 0.0, \\ P_{\text{combined}} &= 38.0, & P_{\text{max}} &= 46.0, \\ \text{score} &\approx 100 \cdot \left(1 - \frac{38}{46}\right) \approx 17.4. \end{aligned}$$

There is essentially no meaningful *keep* beyond generic environmental structure, because the goal expects a rock-shaped target and the resource is a mountain. No strict goal constraint needs to be retracted (*give up* is empty), and there is almost no *bonus* structure that would improve the explanation. The low score makes the ranking behavior explicit: mountains are strongly de-prioritised as pickup targets, even though they belong to the same scene and share part of the physical environment.

These examples show that, starting from a simple scene-level description extracted by the `ABox` builder (numbers and qualitative types of rocks, craters and mountains, plus rover profiles), the neuro-symbolic pipeline produces goal-resource matches whose scores can be interpreted in terms of:

- which symbolic features of the scene are preserved (*keep*);

- which goal assumptions must be relaxed (*give up*);
- which extra information is offered by the resource (*bonus*);
- and how far the match is from an ideal configuration (normalized score in $[0, 100]$).

This semantic transparency holds uniformly for both pickup and obstacle-avoidance goals and for both rover profiles, and it is directly grounded in the LUSNAR scene types produced by the upstream perception and ABox construction stages.

5.6 Summary of Experimental Evidence

The experiments in Chapter 5 establish three key points:

- SegFormer-B0 and SegFormer-B1 both achieve high segmentation performance on challenging LuSNAR test scenes, with mIoU above 0.84 and overall accuracy above 0.98; the small accuracy advantage of B1 is outweighed by the significantly lower latency of B0. Qualitative triptychs confirm that segmentation is robust across typical scenes, with residual limitations concentrated on crater rims and strong shadow regions.
- The symbolic layer (ABox construction + Tiny-ME) has a modest computational footprint compared to the 3D processing and USD stages; reasoning time is negligible with respect to mesh reconstruction and semantic projection. At the same time, the JSON-based matchmaking output exposes rich explanations in terms of CA, CC and surplus, with scores grounded in a penalty model that is consistent with DL semantics.
- The runtime decomposition supports an architecture where 3D semantics and semantic USD generation are handled by parallel ROS 2 branches, while a SegFormer-B0 + Tiny-ME loop can remain lightweight enough for integration into a resource-constrained navigation pipeline. The penalty-based scores over LuSNAR rocks indicate that, despite local segmentation errors, the neuro-symbolic chain is capable of ranking candidate pickup targets in a logically interpretable and computationally affordable way.

The broader implications of these results for the overall system design and mission profiles are discussed in the following **Chapter 6**.

Chapter 6

CONCLUSIONS AND FUTURE WORK

6.1 Summary of Contributions

This thesis presented a **neuro-symbolic navigation pipeline** that couples deep semantic segmentation with lightweight description-logic reasoning and semantic USD scene generation for synthetic lunar environments.

On the perception side, two SegFormer variants (B0 and B1) were trained and evaluated on the LuSNAR dataset under a controlled training and evaluation protocol. Quantitative results showed that **SegFormer-B0** achieves high segmentation quality on challenging test scenes ($mIoU > 0.84$ and overall accuracy > 0.98) while offering significantly lower forward latency than B1. Qualitative inspection of triptychs (*RGB / Ground truth / prediction*) on Moon_3 confirmed that B0 delivers robust segmentations in representative scenarios: rocks are consistently detected even in cluttered contexts, mountains and sky are cleanly separated, and crater shapes are largely preserved, with only local deviations along low-contrast rims. These findings justified the adoption of B0 as the default semantic backbone for integration into the reasoning loop.

On the knowledge representation side, the work introduced a **minimal SPEx TBox** tailored to LuSNAR lunar scenes, rover profiles and rock-centric goals. The TBox was deliberately kept small and focused on domain elements that are directly relevant for validating the end-to-end pipeline: terrain categories (rock, crater, regolith, mountain), rover capability profiles, and pickup/avoidance goals. A dedicated ABox builder instantiated this TBox frame-wise from 2D segmentation, depth and IMU, producing ProtocOWL ABox fragments suitable for Tiny-ME matchmaking under an $\mathcal{ALN}(D)$ profile.

The reasoning layer was characterised both structurally and quantitatively. Structurally, it relies on Tiny-ME to perform **logic-based matchmaking** between goal requests and rock resources, encoding results in a structured JSON format that exposes, for each match: compatibility, CA and CC penalties, maximum penalty, combined penalty, normalised score, and detailed logical explanations (**keep**, **give_up**, CA and surplus descriptions). Quantitatively, the experiments instantiated a human-readable **penalty-based scoring model** along with an **in-**

interpretable ranking of action candidates that is directly grounded in Description Logic semantics.

At the pipeline level, the thesis implemented and instrumented a **full frame-wise chain** from SegFormer outputs to OWL ABoxes, Tiny-ME reasoning and semantic USD export. Each stage (mesh reconstruction, semantic projection, ABox construction, matchmaking, USD export, ProtocOWL fix-up and validation) was wrapped in a performance layer that logs wall-clock timings, CPU usage and memory footprint. Experiments on Moon_3 showed that:

- semantic projection and USD handling dominate the per-frame budget (tens of seconds on the reference workstation),
- the symbolic layer (ABox construction + reasoning) accounts for a small fraction of the total time (below 10% combined),
- reasoning itself is essentially negligible in cost compared to 3D geometry processing.

This decomposition supports an architecture in which **SegFormer-B0 + ABox + Tiny-ME** can remain in the real-time loop, while 3D semantics and USD export run in parallel branches.

Finally, the work defined a **semantic USD workflow** in which per-frame ProtocOWL ABoxes are embedded into USD scenes, and a hash-based validator verifies byte-wise consistency between the embedded payloads and the corresponding .pcowl files. This bridges the neuro-symbolic pipeline with USD-based digital twins and simulation environments, aligning with recent efforts on USD-to-knowledge-graph translation for robotics [17] and knowledge-based sensing/acting in mobile robots [24]. The validator confirmed that, for the experiments considered, embedded ABoxes and disk-resident ProtocOWL files are perfectly consistent, closing the loop from perception to USD-centered semantic inspection.

6.2 Limitations

The proposed framework, while functional end-to-end, is constrained by several simplifying assumptions and design choices.

First, all experiments are conducted on the **synthetic LuSNAR dataset**. Although LuSNAR offers realistic illumination, sensor models and terrain complexity, it does not capture the full range of artefacts present in real lunar imagery, nor hardware-specific noise patterns (e.g., lens flare, sensor non-linearities, thermal drift). The SegFormer models are trained at 512×512 and evaluated at 1024×1024 mostly on a single left RGB camera, without multi-sensor fusion during training or explicit domain adaptation to real data or analog environments.

Second, despite the strong overall metrics, the qualitative analysis in Chapter 5 highlighted residual **segmentation limitations**:

- slight boundary shifts along low-contrast crater rims, leading to thin bands of misclassification between crater and regolith;

- local errors in the presence of strong shadows, particularly when the rover casts a self-shadow across rocks and regolith, causing boundary fragmentation and occasional rock/regolith confusion.

These effects are mitigated in the ABox builder by using depth and slope thresholds, but they remain failure modes of the purely appearance-based front-end and are not explicitly modeled in the current TBox.

Third, the current **SPEx TBox** is intentionally minimal. It focuses on rocks, craters, mountains, basic rover profiles and a small set of goal templates (pickup, obstacle avoidance), with relations that are mostly taxonomic or static. Richer topological, temporal and affordance-related constraints are only partially encoded. As a consequence, the expressivity of semantic matchmaking, although sufficient for rock selection and basic hazard assessment, under-approximates the complexity of realistic surface operations and longer-term mission planning.

Fourth, the **penalty and scoring model** is fixed and scenario-agnostic. The choice $w_a = 0.3$, $w_c = 0.7$ embodies a reasonable preference for penalizing semantic contrast more heavily than missing capabilities, but it is not adapted online to mission risk attitudes or to changing rover conditions (*e.g.*, low battery, degraded mobility). Similarly, the maximum penalty $P_{\max}$ is derived at request level only, without further calibration or learning from mission outcomes.

Fifth, the **ROS 2 architecture** is instantiated only at design level. The implemented pipeline is monolithic and synchronous, executed on a general-purpose workstation without an actual ROS 2 graph, without resource-aware scheduling, and without on-board constraints such as limited CPU, thermal throttling or intermittent communication. The separation between a fast reasoning loop and slower 3D/visualization branches, although strongly supported by the timing analysis, has not yet been validated in a deployed multi-node system.

Finally, the **USD layer** is used primarily as a semantic container and validation target. The embedded ABoxes are not yet exploited to reconstruct a knowledge graph directly from USD, nor is there a second, independent OWL reasoner used to cross-validate Tiny-ME in a KG reconstructed from USD, as in the USD→KG pipeline of [17]. Matchmaking is validated on a limited set of rover profiles and goal templates, rather than on a broader variety of mission types and policies.

6.3 Future Directions

Building on these results, several extensions are technically natural and scientifically promising.

SegFormer training and perception robustness. On the perception side, the current models can be strengthened by:

- performing **training with increased effective batch size** (via larger batches or gradient accumulation), enabling more stable optimisation and better use of mixed precision on modern GPUs;

- introducing carefully designed **shadow-aware and illumination-aware data augmentation**, explicitly targeting the failure modes observed in rover self-shadow and strong cast shadows;
- incorporating depth and surface-normal cues either as additional input channels or via late fusion, to reduce ambiguity in low-contrast crater rims and shadowed regions;
- investigating higher-capacity SegFormer variants or related transformer-based architectures, and calibrating the accuracy–latency trade-off explicitly for edge-class hardware.

These steps would refine the operating point of the segmentation module and tighten the interface between perception and reasoning, particularly under challenging lighting conditions.

TBox design and matchmaking expressivity. On the knowledge representation side, the SPEx TBox can be evolved beyond its minimal form. Future work includes:

- enriching the TBox with **stronger relational constraints**, such as explicit topological relations (adjacency, containment), temporal qualifiers (persistence of hazards, time-bounded goals) and context-dependent hazard models (*e.g.*, slope thresholds conditioned on regolith type and wheel condition);
- refining rover profiles and goal templates to capture **more realistic mission policies**, resource constraints and risk thresholds, including distinctions between exploratory and safety-critical maneuvers;
- aligning the TBox more closely with USD-oriented ontologies and KG models for robotic environments [17], and extending Tiny-ME matchmaking to exploit these additional axioms while preserving tractable reasoning.

This would bring the neuro–symbolic layer closer to the kind of knowledge-based sensing/acting frameworks proposed in [24], but grounded in a USD-centric scene representation and driven by LuSNAR-like data.

Adaptive scoring and policy integration. The penalty-based scoring model can also be extended. Possible directions are:

- allowing **adaptive weights** (w_a, w_c) that depend on mission mode or rover state (*e.g.*, increasing w_c when safety is critical and decreasing it in exploratory phases);
- integrating the scores with higher-level decision-making, using them as inputs to planning or scheduling modules and closing the loop between symbolic matchmaking and mission-level policies;

- collecting statistics on scores and mission outcomes over many frames to learn, at least offline, calibration mappings between penalty-based scores and empirical success rates of pickup attempts.

Such extensions would turn the current static scoring scheme into a more flexible, policy-aware component.

ROS 2 architecture and deployment. The timing analysis strongly suggests further work on the ROS 2 integration. Concrete directions are:

- implementing the proposed separation between a **real-time SegFormer-B0 + ABox + Tiny-ME loop** and slower 3D/visualization branches (mesh reconstruction, semantic projection, USD export and validation);
- exploiting ROS 2 composition, intra-process communication and QoS policies to guarantee bounded latency and predictable jitter for the reasoning loop;
- exploring deployment on embedded or heterogeneous platforms to quantify behaviour under realistic CPU/GPU and power constraints, and to evaluate the robustness of the system under degraded conditions (*e.g.*, partial sensor loss or reduced frame rates).

USD-grounded validation and KG construction. The semantic USD layer can be leveraged more deeply along two complementary lines:

- implementing a **USD→KG translator** that reconstructs an OWL KG from the semantic USD scenes, following the design of [17], and using it to cross-check Tiny-ME results against an independent OWL reasoner on a USD-grounded KG;
- extending the current validator from hash checking to full **matchmaking validation in USD**, where both the pipeline-produced ABox and a KG reconstructed from the same USD scene are used to verify consistency of resource ranking, explanations and policy decisions.

This would strengthen confidence in the correctness of the neuro-symbolic loop and make the USD-centric representation a first-class artifact for both simulation and formal analysis.

From synthetic to real and analog data. Finally, an important direction is to move beyond purely synthetic data:

- applying the proposed pipeline to **lunar analog environments** (*e.g.*, outdoor testbeds, regolith test fields) using real rover platforms, and retraining or fine-tuning SegFormer with domain adaptation techniques;

- assessing how well the minimal SPEx TBox and current scoring model transfer to real-world noise and partial observability, and revising them based on empirical failures observed in analog campaigns.

In combination, these lines of work point toward a more mature **USD-centred neuro-symbolic stack** in which SegFormer-based perception, SPEx reasoning, and semantic digital twins interact tightly, from on-board navigation to high-fidelity simulation and offline mission analysis.

BIBLIOGRAPHY

- [1] Boyu Kuang, Chengzhen Gu, Zeeshan A. Rana, Yifan Zhao, Shuang Sun, and Somtochukwu Godfrey Nnabuife. Semantic Terrain Segmentation in the Navigation Vision of Planetary Rovers—A Systematic Literature Review. *Sensors*, 22(21):8393, 2022.
- [2] Alexander M. Barrett, Matthew R. Balme, Mark Woods, Spyros Karachalios, Danilo Petrocelli, Luc Joudrier, and Elliot Sefton-Nash. NOAH-H, a Deep-Learning Terrain Classification System for Mars: Results for the ExoMars Rover Candidate Landing Sites. *Icarus*, 371:114701, 2022.
- [3] Boyu Kuang, Mariusz Wisniewski, Zeeshan A. Rana, and Yifan Zhao. Rock Segmentation in the Navigation Vision of the Planetary Rovers. *Mathematics*, 9(23):3048, 2021.
- [4] Jiayi Liu, Qianyu Zhang, Xue Wan, Shengyang Zhang, Yaolin Tian, Haodong Han, Yutao Zhao, Baichuan Liu, Zeyuan Zhao, and Xubo Luo. LuSNAR: A Lunar Segmentation, Navigation and Reconstruction Dataset Based on Multi-Sensor for Autonomous Exploration. *arXiv preprint arXiv:2407.06512*, 2024.
- [5] Franz Baader, Diego Calvanese, Deborah L. McGuinness, Daniele Nardi, and Peter F. Patel-Schneider, editors. *The Description Logic Handbook: Theory, Implementation and Applications*. Cambridge University Press, Cambridge, 2 edition, 2007.
- [6] Lauren Nicole DeLong, Ramon Fernández Mir, and Jacques D. Fleuriot. Neurosymbolic AI for Reasoning Over Knowledge Graphs: A Survey. *IEEE Transactions on Neural Networks and Learning Systems*, 36(5):7822–7842, 2025.
- [7] Abhishek Potnis, Dalton Lunga, Alexandre Sorokine, Philipe Dias, Lexie Yang, Jacob Arndt, Jordan Bowman, and Jason Wohlgemuth. Towards Geospatial Knowledge Graph Infused Neuro-Symbolic AI for Remote Sensing Scene Understanding. In *IGARSS 2023 – IEEE International Geoscience and Remote Sensing Symposium*, pages 1400–1403. IEEE, July 2023.
- [8] Junbo Li, Keyan Chen, Gengju Tian, Lu Li, and Zhenwei Shi. MarsSeg: Mars Surface Semantic Segmentation With Multilevel Extractor and Connector. *IEEE Transactions on Geoscience and Remote Sensing*, 63:1–12, 2025.

- [9] Jiahang Zhang, Lilang Lin, Zejia Fan, Wenjing Wang, and Jiaying Liu. S⁵Mars: Semi-Supervised Learning for Mars Semantic Segmentation. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–15, 2024.
- [10] Yuqi Dai, Tie Zheng, Changbin Xue, and Li Zhou. SegMarsViT: Lightweight Mars Terrain Segmentation Network for Autonomous Driving in Planetary Exploration. *Remote Sensing*, 14:6297, 2022.
- [11] Michele Ruta, Floriano Scioscia, Ivano Bilenchi, Filippo Gramegna, Giuseppe Loseto, Saverio Ieva, and Agnese Pinto. A Multiplatform Reasoning Engine for the Semantic Web of Everything. *Journal of Web Semantics*, 73:100709, 2022.
- [12] Giuseppe Loseto, Ivano Bilenchi, Filippo Gramegna, Davide Loconte, Floriano Scioscia, and Michele Ruta. *Tiny-ME Wasm: Description Logics Reasoning in Your Browser*, pages 114–126. Lecture Notes in Computer Science. Springer, Cham, 2024.
- [13] Ivano Bilenchi, Filippo Gramegna, Giuseppe Loseto, Saverio Ieva, Floriano Scioscia, and Michele Ruta. Cowl: Pushing OWL 2 over the Edge. *Internet of Things*, 29:101439, 2025.
- [14] Antoine Richard, Junnosuke Kamohara, Kentaro Uno, Shreya Santra, Dave van der Meer, Miguel Olivares-Mendez, and Kazuya Yoshida. OmniLRS: A Photorealistic Simulator for Lunar Robotics. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 16901–16907. IEEE, 2024.
- [15] NVIDIA Corporation. NVIDIA Isaac Sim. <https://developer.nvidia.com/isaac/sim>, 2024. Accessed: 2025-01-05.
- [16] NVIDIA Corporation. NVIDIA Omniverse. <https://www.nvidia.com/omniverse/>, 2024. Accessed: 2025-01-05.
- [17] Giang Nguyen, Daniel Beßler, Simon Stelter, Mihai Pomarlan, and Michael Beetz. Translating Universal Scene Descriptions into Knowledge Graphs for Robotic Environment. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 9389–9395. IEEE, May 2024.
- [18] Santhosh K. Ramakrishnan, Aaron Gokaslan, Erik Wijmans, Oleksandr Maksymets, Alexander Clegg, John Turner, Eric Undersander, Wojciech Galuba, Andrew Westbury, Angel X. Chang, Manolis Savva, Yili Zhao, and Dhruv Batra. Habitat-Matterport 3D (HM3D): 1000 Large-Scale 3D Environments for Embodied AI. In *Thirty-Fifth Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2021.
- [19] Julian Straub, Thomas Whelan, Lingni Ma, Yufan Chen, Erik Wijmans, Simon Green, Jakob J. Engel, Raul Mur-Artal, Carl Ren, Shobhit Verma,

- Anton Clarkson, Mingfei Yan, Brian Budge, Yajie Yan, Xiaqing Pan, June Yon, Yuyang Zou, Kimberly Leon, Nigel Carter, Jesus Briales, Tyler Gillingham, Elias Mueggler, Luis Pesqueira, Manolis Savva, Dhruv Batra, Hauke M. Strasdat, Renzo De Nardi, Michael Goesele, Steven Lovegrove, and Richard Newcombe. The Replica Dataset: A Digital Replica of Indoor Spaces. *arXiv preprint arXiv:1906.05797*, 2019.
- [20] Mukul Khanna, Yongsan Mao, Hanxiao Jiang, Sanjay Haresh, Brennan Shacklett, Dhruv Batra, Alexander Clegg, Eric Undersander, Angel X. Chang, and Manolis Savva. Habitat Synthetic Scenes Dataset (HSSD-200): An Analysis of 3D Scene Scale and Realism Tradeoffs for ObjectGoal Navigation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16384–16393, 2024.
- [21] Wenshan Wang, Delong Zhu, Xiangwei Wang, Yaoyu Hu, Yuheng Qiu, Chen Wang, Yafei Hu, Ashish Kapoor, and Sebastian Scherer. TartanAir: A Dataset to Push the Limits of Visual SLAM. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4909–4916. IEEE, 2020.
- [22] Paulo Gonçalves, Alberto Olivares-Alarcos, J. Bermejo, Stefano Borgo, Mohammed Diab, Maki Habib, Hirenkumar Nakawala, Veera Ragavan Sampath Kumar, Ricardo Sanz, Elisa Tosello, and Howard Li. IEEE Standard for Autonomous Robotics Ontology [Standards]. *IEEE Robotics & Automation Magazine*, 28:171–173, September 2021.
- [23] Vitor A. M. Jorge, Vitor F. Rey, Renan Maffei, Sandro Rama Fiorini, Joel Luis Carbonera, Flora Branchi, João P. Meireles, Guilherme S. Franco, Flávia Farina, Tatiana S. da Silva, Mariana Kolberg, Mara Abel, and Edson Prestes. Exploring the IEEE Ontology for Robotics and Automation for Heterogeneous Agent Interaction. *Robotics and Computer-Integrated Manufacturing*, 33(C):12–20, June 2015.
- [24] Michele Ruta, Floriano Scioscia, Giuseppe Loseto, and Eugenio Di Sciascio. Knowledge-Based Sensing/Acting in Mobile Autonomous Robots. In *2017 First IEEE International Conference on Robotic Computing (IRC)*, pages 422–427, Taichung, Taiwan, 2017. IEEE.
- [25] Yansheng Li, Song Ouyang, and Yongjun Zhang. Combining Deep Learning and Ontology Reasoning for Remote Sensing Image Semantic Segmentation. *Knowledge-Based Systems*, 243:108469, 2022.
- [26] Peng Jiang, Philip Osteen, Maggie Wigness, and Srikanth Saripalli. RELIS-3D Dataset: Data, Benchmarks and Analysis. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1110–1116. IEEE, 2021.

- [27] Jie Yin, Ang Li, Tao Li, Wenxian Yu, and Danping Zou. M2DGR: A Multi-Sensor and Multi-Scenario SLAM Dataset for Ground Robots. *IEEE Robotics and Automation Letters*, 7(2):2266–2273, 2022.
- [28] Yiming Li, Sihang Li, Xinhao Liu, Moonjun Gong, Kenan Li, Nuo Chen, Zijun Wang, Zhiheng Li, Tao Jiang, Fisher Yu, Yue Wang, Hang Zhao, Zhiding Yu, and Chen Feng. SSCBench: A Large-Scale 3D Semantic Scene Completion Benchmark for Autonomous Driving. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 13333–13340. IEEE, 2024.
- [29] Mohsen Zand, Shyamala Doraisamy, Alfian Abdul Halin, and Mas Rina Mustaffa. Visual and Semantic Context Modeling for Scene-Centric Image Annotation. *Multimedia Tools and Applications*, 76, March 2017.
- [30] Hela Yahyaoui, Fethi Ghazouani, and Imed Riadh Farah. Deep Learning Guided by an Ontology for Medical Images Classification Using a Multimodal Fusion. In *2021 International Congress of Advanced Technology and Engineering (ICOTEN)*, pages 1–6. IEEE, 2021.
- [31] Sachit Rajbhandari, Jagannath Aryal, Jonathan Osborn, and Robert Musk. Leveraging Machine Learning to Extend Ontology-Driven Geographic Object-Based Image Analysis (O-GEOBIA): A Case Study in Forest-Type Mapping. *Remote Sensing*, 11:503, March 2019.
- [32] W3C OWL Working Group. OWL 2 Web Ontology Language: Functional-Style Syntax. <https://www.w3.org/TR/owl2-syntax/>, 2024. Accessed: 2025-02-12.
- [33] Michele Ruta, Floriano Scioscia, Valerio Di Ceglie, Ivano Bilenchi, Filippo Gramegna, and Eugenio Di Sciascio. A Format and Protocol for Machine-to-Machine OWL Data Streams. In *10th IEEE International Workshop on Advances in Sensors and Interfaces (IWASI 2025)*. IEEE, 2025. to appear.
- [34] Frederik Hogenboom, Flavius Frasincar, and Uzay Kaymak. A Review of Approaches for Representing RCC8 in OWL. In *Proceedings of the 2010 ACM Symposium on Applied Computing*, pages 1444–1445, 2010.
- [35] R. Michael Swan, Deegan Atha, Henry A. Leopold, Matthew Gildner, Stephanie Oij, Cindy Chiu, and Masahiro Ono. AI4MARS: A Dataset for Terrain-Aware Autonomous Driving on Mars. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1982–1991. IEEE, 2021.

Appendix A

Dataset Catalog for Planetary and Robotic Benchmarks

This appendix reports the complete catalog for the ten datasets (DS01–DS10) analyzed in Chapter 2. Each dataset is summarized in a compact two-column table using the common cataloging template.

A.1 Dataset Cataloging Template

Field	Description
ID	Internal identifier (DS01–DS10) for cross-reference.
Dataset Name	Official name of the dataset.
Publication Year	Year of publication or release.
Data Source	Real / Simulated / Mixed.
Target Platform	Robot, sensor rig, or simulator.
Application Domain	Martian, lunar, terrestrial, indoor, etc.
Sensor Modalities	RGB, Depth, LiDAR, IMU, GPS, etc.
Proprioceptive Data	Presence of IMU, Odometry (Odometry), encoders, etc.
Scenario Complexity	Simple / Medium / Complex.
Images & Labels	Number of frames or annotated samples.
Semantic Annotations	Pixel-, voxel-, or instance-level.
Semantic Classes	Main classes and relevance.
Primary Objective	Main research purpose.
Data Format	File types and structure.
Conversion to USD	Yes / Partial / No.
Presence of 3D Assets	Meshes, point clouds, or none.
Simulability in Omniverse	High / Medium / Low.
Integration Effort	High / Medium / Low.
Suitability for Symbolic Reasoning	High / Medium / Low.
Knowledge Base (KB) Mappability	High / Medium / Low.
License	Usage conditions.
URL / DOI	Repository or reference link (Uniform Resource Locator (URL) / Digital Object Identifier (DOI)).
Notes	Additional remarks or usage in thesis.

Table A.1: Compact dataset cataloging template used throughout Appendix A.2.

A.2 Catalog Entries DS01–DS10

Table A.2: Catalog entry for DS01 – SSCBench.

Field	Description
Dataset	DS01 – SSCBench
Publication Year	2024 (IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)).
Data Source	Mixed: derived from KITTI-360 Autonomous Driving Dataset (KITTI-360), nuScenes Autonomous Driving Dataset (nuScenes), and Waymo. Data unified through voxelization and label harmonization.
Target Platform	Automotive platforms with multi-camera and LiDAR rigs (Waymo, KITTI-360, nuScenes).
Domain	Terrestrial urban environments across six cities; varied weather, lighting, and traffic conditions.
Modalities	RGB (mono/trino), LiDAR (dense/sparse depending on subset), radar in source datasets, unified voxel grids, 3D bounding boxes.
Proprioception	No (IMUs/GPS exist in source datasets but excluded in SSCBench).
Complexity	High: multi-city, dynamic traffic, occlusions, severe class imbalance, viewpoint diversity.
Images / Labels	66,913 total frames: 38,562 train, 15,798 val, 12,553 test.
Annotations	Dense voxel-wise semantic scene completion labels ($256 \times 256 \times 32$ grid, 0.2 m resolution). Dynamic objects corrected via 3D bounding-box alignment.
Classes	Unified ~ 20 -class taxonomy (road, sidewalk, building, vegetation, vehicles, traffic signs, poles, terrain, etc.).
Objective	SSC: geometry + semantics reconstruction from monocular, trinocular or LiDAR inputs.
Data Format	SemanticKITTI-style voxel grids; aggregated point clouds; KITTI-like metadata.
3D Assets	None: voxel volumes only (no meshes, no textured geometry).
Conversion to USD	No direct conversion; requires custom voxel-to-mesh reconstruction pipelines.
Simulability	Low: voxel grids not natively usable in Omniverse. Synthetic scenes must be recreated manually.
Integration Effort	High: requires mesh recovery, asset authoring, and semantic mapping.
Symbolic Reasoning	Medium: high-quality labels but no explicit objects or relations; voxel cells must be post-processed.
KB Mappability	Medium-High: voxel labels clean and consistent; good for TBox design, limited for ABoxes without object grouping.
License	Open access for research.
URL	https://github.com/ai4ce/SSCBench .
Notes	Largest SSC benchmark to date; ideal for training/evaluating monocular and LiDAR-based SSC models. Not suitable for Omniverse simulation without reconstruction.

Table A.3: Catalog entry for DS02 – AI4Mars.

Field	Description
Dataset	DS02 – AI4Mars
Publication Year	2021
Data Source	Real images from NASA Mars rovers (Curiosity, Spirit, Opportunity).
Target Platform	Mars Science Laboratory (MSL)/Mars Exploration Rover (MER) rover platforms for terrain-aware autonomous driving.
Domain	Martian surface terrain for autonomous exploration and navigation.
Modalities	Navigation Camera (NAVCAM)/Mast Camera (Mastcam) RGB; stereo-derived range/depth and camera metadata available from mission archives.
Proprioception	No (IMU, Odometry and actuator data not included in the public release).
Complexity	Medium: multiple terrain morphologies (soil, sand, bedrock, big rocks) and changing viewpoints.

Field	Description
Images / Labels	~35k images; ~326k pixel-wise labels from crowdsourcing, with expert-validated test masks.
Annotations	Pixel-wise semantic masks (4 terrain classes + Null); rover body and areas beyond 30 m are masked out.
Classes	Soil, Bedrock, Sand, Big Rock, Null; all suited to standard 3D terrain assets.
Objective	Supervised terrain semantic segmentation for traversability and hazard assessment.
Data Format	PNG images; PNG/JSON masks; auxiliary <code>info.txt</code> metadata; stereo range maps obtainable from Planetary Data System (PDS) archives.
3D Assets	None (no native meshes, DEMs or reconstructed environments).
Conversion to USD	No direct pipeline; requires terrain reconstruction (e.g., heightmaps or synthetic scenes) before USD export.
Simulability	Medium: real imagery provides strong reference for procedural Martian terrains in Omniverse.
Integration Effort	Medium-High: design of 3D terrain meshes and mapping of 2D labels to 3D regions is required.
Symbolic Reasoning	Medium: clean, simple terrain taxonomy; lacks explicit 3D structure and relations.
KB Mappability	Medium: classes map directly to TBox concepts; ABoxes population mainly at image/pixel level.
License	Open access (public NASA mission data).
URL	https://huggingface.co/datasets/hassanjbara/AI4MARS
Notes	Used as real Martian semantic reference; not chosen as primary benchmark due to missing 3D geometry and proprioception.

Table A.4: Catalog entry for DS03 – S5Mars.

Field	Description
Dataset	DS03 – S5Mars
Publication Year	2024
Data Source	Real Mastcam RGB imagery from the Curiosity (MSL) rover.
Target Platform	MSL Mastcam imaging system.
Domain	Martian surface geology and rover-centric navigation scenes.
Modalities	High-resolution RGB (1200×1200 px); no depth or 3D geometry.
Proprioception	No (no IMU, Odometry, or rover state information provided).
Complexity	High: heterogeneous geology, irregular boundaries, shadows, overlapping materials.
Images / Labels	6,000 images; sparse labels with ~49% pixel coverage; expert-annotated.
Annotations	Sparse pixel-wise segmentation; only high-confidence regions labeled after multi-stage verification.
Classes	9: Sky, Ridge, Soil, Sand, Bedrock, Rock, Rover, Trace, Hole.
Objective	High-fidelity semantic segmentation of Martian terrain with semi-supervised learning.
Data Format	RGB images (PNG), label masks (PNG), JSON splits for train/val/test; “easy” / “hard” subsets included.
3D Assets	None (no DEMs, meshes, depth maps, or point clouds).
Conversion to USD	No direct conversion; requires reconstruction or synthetic terrain generation before USD export.
Simulability	Medium: images provide strong visual priors for recreating photorealistic Mars-like Omniverse scenes.
Integration Effort	High: demands creation of full 3D terrain meshes, class-based material assignment, and scene layout.
Symbolic Reasoning	High: clear and fine-grained semantic taxonomy supports structured reasoning and ontology alignment.
KB Mappability	High: the 9-class ontology maps cleanly into $\mathcal{ALN}$ TBox concepts; ABoxes can be generated per image.
License	Open access.
URL	https://jhang2020.github.io/S5Mars.github.io/
Notes	High-quality expert annotations; ideal for sparse-label learning and conceptual modeling of Martian semantics.

Table A.5: Catalog entry for DS04 – HM3D.

Field	Description
Dataset	DS04 – HM3D
Publication Year	2021
Data Source	Real indoor 3D reconstructions from Matterport Pro2 RGB-D scans.
Target Platform	Embodied AIs simulators (Habitat), robotic perception and navigation.
Domain	Terrestrial indoor spaces: multi-floor residences, offices, shops, public areas.
Modalities	Panoramic RGB, depth, full 3D textured mesh (glTF), point clouds, semantic metadata.
Proprioception	No (no IMU, Odometry, or robot kinematics).
Complexity	High: multi-room/multi-floor buildings, clutter, obstacles, diverse layouts.
Scenes / Labels	1,000 building-scale scans; 10,600+ rooms; 112.5k m ² navigable space; ~140k instances.
Annotations	Instance-level mesh annotations with semantic textures; no pixel-level Ground Truth (GT) (derived in-sim).
Classes	~170 indoor object categories (furniture, fixtures, appliances, structural elements).
Objective	Embodied navigation, mapping, scene understanding, rearrangement tasks.
Data Format	Binary glTF meshes + textures; JSON metadata; depth maps; Habitat-ready scene bundles.
3D Assets	Yes: complete high-fidelity meshes with textures, room layouts, and navigable areas.
Conversion to USD	Feasible (Blender, Omniverse Connect, Habitat exporters); not native.
Simulability	Medium–High: complete 3D geometry; requires USD conversion and semantic mapping.
Integration Effort	High: large mesh files, multiple floors, complex texture alignment, category mapping.
Symbolic Reasoning	High: rich object taxonomy and explicit instances support ontology-based reasoning.
KB Mappability	High: structured environments and object hierarchies fit TBox/ABox representation.
License	Academic research license (Matterport-based).
URL	https://aihabitat.org/datasets/hm3d/
Notes	Large-scale, photorealistic indoor dataset; reference for high-fidelity 3D semantic environments.

Table A.6: Catalog entry for DS05 – TartanAir.

Field	Description
Dataset	DS05 – TartanAir
Publication Year	2020 (V1), updated 2023 (TartanAir V2).
Data Source	Synthetic, generated in UE4 with AirSim.
Target Platform	Generic robotic platforms (ground and aerial) and simulators.
Domain	Indoor/outdoor terrestrial scenes: urban, rural, natural, sci-fi.
Modalities	Stereo RGB, depth, optical flow, LiDAR, 6DoF pose; V2 adds IMU and event camera.
Proprioception	V1: No; V2: Yes (IMU accelerometer + gyroscope).
Complexity	High: 6DoF motion, dynamic lighting, weather, moving objects.
Images / Labels	Over 400k frames with multimodal ground truth.
Annotations	Pixel-wise segmentation (selected sequences), partial instance-level.
Classes	Scene-dependent: road, vegetation, buildings, rocks, indoor objects.
Objective	Visual odometry, SLAM benchmarking, domain randomisation for sim-to-real.
Data Format	PNG (RGB/depth), JSON (poses), PCD (LiDAR), flow maps; CSV for IMU in V2.
3D Assets	Not distributed; based on UE4 environment assets.
Conversion to USD	Not native; possible via UE4 → USD export or Blender/ROS tools.
Simulability	Medium: full scenes reproducible from UE4 but require export.
Integration Effort	High: scene regeneration or manual export needed.

Field	Description
Symbolic Reasoning	Medium: pixel-level labels usable, but no semantic relations provided.
KB Mappability	Medium: classes are mappable, but lack of ontology and spatial structure.
License	Open academic use.
URL	https://theairlab.org/tartanair-dataset/
Notes	Large synthetic benchmark with rich modalities; strong reference for SLAM under domain randomisation.

Table A.7: Catalog entry for DS06 – RELLIS-3D.

Field	Description
Dataset	DS06 – RELLIS-3D
Publication Year	2021–2022
Data Source	Real-world data collected from a Clearpath Warthog Unmanned Ground Vehicle (UGV).
Target Platform	Clearpath Warthog unmanned ground vehicle.
Domain	Outdoor off-road environments: forests, trails, hills, grasslands.
Modalities	RGB, LiDAR (OS1-64 + VLP-32C), stereo camera, IMU, GPS.
Proprioception	Yes: high-precision GPS and IMU (VN-300).
Complexity	High: irregular terrain, vegetation, natural obstacles, class imbalance.
Images / Labels	6,235 pixel-wise annotated RGB images; 13,556 point-wise LiDAR scans.
Annotations	Pixel-wise (images) and point-wise (LiDAR); 20 image classes, 16 LiDAR classes.
Classes	Grass, tree, bush, mud, puddle, water, rubble, barrier, pole, fence, vehicle, object, person, etc.
Objective	Off-road semantic segmentation, obstacle detection, traversability analysis.
Data Format	ROS bag (synchronized streams), PNG/Joint Photographic Experts Group (JPEG) (images), PCD (point clouds).
3D Assets	Partial: LiDAR point clouds, but no full meshes or scene reconstructions.
Conversion to USD	No direct conversion; possible via ROS → PCD → mesh → USD pipeline.
Simulability	Low–Medium: requires mesh reconstruction and manual scene setup.
Integration Effort	High: off-road geometry reconstruction and semantic mapping required.
Symbolic Reasoning	High: rich visual/LiDAR labels support symbolic traversability reasoning.
KB Mappability	Medium–High: classes map well to ontologies; spatial relations must be derived.
License	Open academic use.
URL	https://github.com/unmannedlab/RELLIS-3D
Notes	Strong benchmark for multi-modal off-road navigation under real environmental variability.

Table A.8: Catalog entry for DS07 – M2DGR.

Field	Description
Dataset	DS07 – M2DGR
Publication Year	2021
Data Source	Real-world acquisition (indoor + outdoor) using a ground robot.
Target Platform	Ground robots for SLAM and multi-sensor navigation.
Domain	Indoor, outdoor, and transition scenarios (corridors, streets, halls, lifts).
Modalities	RGB fisheye cameras (×6), sky-pointing camera, Infrared (IR) camera, event camera, stereo camera, LiDAR, IMU, GPS-Real-Time Kinematic (RTK).
Proprioception	Yes: IMU, wheel Odometry, GPS-RTK.
Complexity	High: diverse lighting, darkness, lifts, indoor–outdoor transitions.
Images / Labels	36 sequences (~1 TB); no semantic labels.
Annotations	None (SLAM-only, no pixel/point labels).
Classes	N/A (no semantic taxonomy).
Objective	Multi-sensor SLAM and visual–inertial–LiDAR fusion benchmarking.
Data Format	ROS bag (synchronized streams), images, point clouds, ground-truth trajectories.

Field	Description
3D Assets	None (only raw sensor recordings).
Conversion to USD	No direct support; possible via ROS → PCD → mesh → USD workflow.
Simulability	Low–Medium (requires reconstruction of 3D scenes).
Integration Effort	High due to absence of meshes and semantic data.
Symbolic Reasoning	Low (no semantic or relational information).
KB Mappability	Low (requires manual ontology construction).
License	Open academic use.
URL	https://github.com/SJTU-ViSYS/M2DGR
Notes	Rich multi-sensor dataset; strong SLAM benchmark with challenging scenarios (dark rooms, lifts, transitions).

Table A.9: Catalog entry for DS08 – Replica.

Field	Description
Dataset	DS08 – Replica
Publication Year	2019
Data Source	Real indoor scans reconstructed via high-fidelity photogrammetry.
Target Platform	Embodied AIs simulators (Habitat, Omniverse, Unity).
Domain	Indoor multi-room environments (apartments, offices, hotel rooms).
Modalities	RGB panoramas, depth, textured 3D meshes, semantic + instance labels.
Proprioception	No (no IMU/Odometry data).
Complexity	High (dense geometry, clutter, occlusions, reflective surfaces).
Scenes / Labels	18 scenes; >7k object instances.
Annotations	Instance-level 3D semantic meshes, High Dynamic Range (HDR) textures, reflectors.
Classes	88 indoor categories (furniture, appliances, small objects).
Objective	Scene understanding, navigation, segmentation, 3D mapping.
Data Format	glTF meshes, Ptex Texture Mapping Format (PTex) textures, JSON annotations, PNG/EXR images.
Conversion to USD	Yes (glTF → USD supported; Habitat export available).
3D Assets	Full photorealistic meshes with HDR textures and semantic IDs.
Simulability	High (native support in Habitat; smooth import in Omniverse).
Integration Effort	Medium (semantic metadata injection required).
Symbolic Reasoning	High (instance-level structure supports ABox/TBox alignment).
KB Mappability	High (well-defined classes, stable naming).
License	Academic research use.
URL	https://github.com/facebookresearch/Replica-Dataset
Notes	High-realism benchmark; ideal for ontology-linked USD scenes.

Table A.10: Catalog entry for DS09 – HSSD-200.

Field	Description
Dataset	DS09 – HSSD-200
Publication Year	2023
Data Source	Synthetic, human-authored 3D indoor scenes.
Target Platform	Embodied AIs simulators (Habitat, Omniverse).
Domain	Indoor multi-room environments.
Modalities	RGB-D renders, semantic instance maps, textured meshes, object poses.
Proprioception	No
Complexity	High (466 semantic categories).
Scenes / Models	211 scenes; 18,656 unique 3D models.
Annotations	Instance-level semantic labels; mesh-level and texture-level mapping.
Semantic Classes	466 categories (furniture, appliances, small objects).
Objective	Object-goal navigation, semantic scene understanding, mapping.
Data Format	glTF, PNG/EXR, JSON (poses, semantics), Habitat metadata.

Field	Description
Conversion to USD	Yes (glTF → USD importers supported).
3D Assets	Full synthetic scenes with detailed textured meshes.
Simulability	High (runs efficiently in Omniverse and Habitat).
Integration Effort	Medium (scene graph alignment + semantic metadata).
Symbolic Reasoning	High (rich hierarchy; consistent per-object semantics).
KB Mappability	High (WordNet-aligned categories; ontology-friendly).
License	Academic research use.
URL	https://3dlg-hcvc.github.io/hssd/
Notes	High-fidelity synthetic benchmark; strong transfer to real 3D scans.

Table A.11: Catalog entry for DS10 – LuSNAR.

Field	Description
Dataset	DS10 – LuSNAR
Publication Year	2024
Data Source	Synthetic (UE4 + AirSim)
Target Platform	Lunar rover simulation
Domain	Lunar surface (outdoor, planetary)
Modalities	Stereo RGB, Depth (Portable Float Map (PFM)), LiDAR, IMU, rover pose, timestamps
Proprioception	Yes (IMU + pose)
Complexity	Medium–High (9 scenes with increasing relief and object density)
Images / Labels	13,006 sequences; RGB-D pairs, 2D masks, LiDAR frames; total ~108 GB
Annotations	Pixel-wise 2D masks (PNG), point-wise 3D labels (Plain Text File Format (TXT))
Semantic Classes	2D: Regolith, Rock, Crater, Mountain, Sky; 3D: Regolith, Rock, Crater
Primary Objective	2D/3D segmentation, SLAM, stereo matching, 3D reconstruction
Data Format	PNG (RGB, masks), PFM (depth), TXT (LiDAR, IMU, poses)
Conversion to USD	No direct support (requires custom UE4/AirSim → USD pipeline)
3D Assets	Point clouds and depth; no native meshes
Simulability	Medium (UE4 scenes, but not USD-native)
Integration Effort	Medium–High (format conversion, scene reconstruction)
Symbolic Reasoning	Medium–High (clear terrain ontology; navigational relevance)
KB Mappability	High (semantic classes aligned with SPEx ontology)
License	Open access
URL	https://github.com/zqyu9/LuSNAR-dataset
Notes	Multi-modal lunar benchmark; central dataset for this thesis.